Design For Test

1. What is sequential Depth In DFT? How does it improve coverage?

Ans: FastScan performs clock sequential test generation if you specify a non-zero sequential depth.

Clock sequential identification selects scannable cells by cutting sequential loops and limiting sequential depth based on the -Depth switch. Typically, this method is used to create structured partial scan designs that can use the FastScan clock sequential ATPG algorithm.

Setting the -Sequential switch to either 0 (the default) or 1 results in patterns with a

maximum sequential depth of one, but FastScan creates clock sequential patterns only if the setting is 1 or higher.

The maximum allowable sequential depth is 255 (a typical depth would range from 2 to 5).

Coverage Improvement :

Testable faults become ATPG_untestable faults because of constraints, or limitations,

placed on the ATPG tool (such as a pin constraint or an insufficient sequential depth).

These faults may be possible-detectable, or detectable, if you remove some constraint, or change some limitation, on the test generator (such as removing a pin constraint or changing the sequential depth).

Also,(while using named capture procedure) gradually add more capture procedures with higher sequential depth until the test coverage goal is achieved or the pattern count limit is reached.

2. What is the difference between flops and scan flops?

Ans: Difference between scan flop and simple flops becomes more easy to understand when we know why do we need scan flop!!

Need of scan flop:

The goal of scan design is to make a difficult-to-test sequential circuit behave (during the testing process) like an easier-to-test combinational circuit. Achieving this goal involves replacing sequential elements with scannable sequential elements (scan cells/scan flops) and then stitching the scan cells together into scan registers, or scan chains. You can then use these serially connected scan cells to shift data in and out when the design is in scan mode.

“Before Scan” design is difficult to initialize to a known state, making it difficult to both control the internal circuitry and observe its behavior using the primary inputs and outputs of the design. In a "Scan design" scan memory elements (scan flops) replace the original memory elements (normal flops) imparting controllability and observability to the design (prime requirement for the design being testable), when shifting is enabled.

3. What is robust / non_robust test for path delay faults?

Ans: Non -Robust tests:

A pair of at-speed vectors that test a path delay fault; fault detection is not guaranteed, because it depends on other delays in the circuit.

Robust Tests:

A pair of at-speed vectors that test a path delay fault independent of other delays or delay faults in the circuit.

4. What are the techniques used to reduce pattern count without losing coverage ?

Ans: The number of test patterns that you need to achieve full coverage depends on the design size. Different ATPG tools offer different compression and pattern ordering techniques to help reduce pattern count.

fault models beyond stuck-at typically require pattern counts that are much larger than those for stuck-at only.

For Pattern reduction, first step is the chain balancing during Stitching or scan insertion. If your chains are balanced, Tool needs to insert less dummy patterns for reaching till required flop.

Also we can include compression on the chains where we have constraints on the pins of device.This means if we are having the compression factor of 2 then your 1 scan chain will get divided into 2 inside the device reducing your chain length (flops per scan chain).

5. Are the faults on the resets of the flops are detected? if so how are they detected?

Ans: Yes the faults on reset of the flop are detectable. It is detected in the following way

1) Define the reset pin as clock in the tdl

2) Let us assume that the reset of the flop is active low. Keep the reset as 'high' and scan in '1' in to the flop (during this scan enable is 'high')

3)As we have scan-in '1' the Q of the flop will have '1'

4) Make the scan-enable 'low' and toggle the reset(i.e. make it low). This is allowed because we have declared the reset as clock.

5) As reset is 'low' the flop need to get reset and whatever the value in the flop will become '0'

6) Then strobe for '0', while shifting-out the contents from the flop.

7) If the output while shifting out is '0', it means the reset pin of the flop is not stuck-at 1. Otherwise it is stuck at '1'.

Note:

1. In test mode all the flops should have a asynchronous reset.

2. The reset pins of all flops should be controllable from Top (Chip) level.

If your design contains a lot of synchronous reset flops. Then since during AC (transdly) testing these reset line faults are not targeted for AC, Constraining the reset to off state is the best way.

6. If we provide controllability and observability to the ports of area under test, will it improve the test coverage? If yes, how much %age of improvement can we see?

Ans: yes, we can see an improvement in coverage. Think about the bounding logic that we usually give for macros...

the improvement in coverage will depend on the initial coverage without bound and the no of faults in the design under test.

7. When both the Transition-delay and path_delay target for transition faults, why we should have two different methods to detect the same?

Ans: Yes both Transition and Path delay will target the transition faults, but they are different in the following ways
1) Transition-delay is related to slow-to-rise or slow-to-fall faults at a particular node. Whereas path-delay is related to slow-to-rise or slow-to-fall faults of a particular path.

2) The reason for transition-delay at a node is some manufacturing defect at that node (more resistive node). The reason for path-delay is some manufacturing defect that is distributed through out the path (more resistive path). Let me explain this in detail with an example.

Let us assume that in a path there are some 10 nodes, and the transition delay at each node is that much less which won't create any timing violation in the transition ATPG. But in path-delay these delays at each node might get accumulated and result in not meeting the timing. So it is must to have path-delay besides Transition-delay.

Transition delay is similar to stuck-at atpg, except that it attempts to detect slow-to-rise and slow-to-fall nodes, rather than stuck-at-0 and stuck-at-1 nodes. A slow-to-rise fault at a node means that a transition from 0 to 1 on the node doesn't produce the correct results at the maximum operating speed of the design. Similarly a slow-to-fall fault means that a transition from 1 to 0 on a node doesn't produce the correct results at the maximum speed of the design. Transition delay fault targets single point defects.

The Path delay fault model is useful for testing and characterizing critical timing paths in the design. Path delay fault tests exercise critical paths at-speed to detect whether the path is too slow because of manufacturing detects or variations.

Path delay fault testing targets physical defects that might affect distributed region of a chip. For example, incorrect field oxide thicknesses could lead to slower signal propagation times, which could cause transitions along a critical path to arrive too late.

If we attain 100% coverage with transition atpg test then we don't need to do the path delay test. But this is not the case in most of our designs. Then how can we GUARANTEE that all the critical paths are targeted by Transition ATPG

so we give the critical path to the atpg tool and tell it to generate patterns for it

this is called path delay.

8. What is Burnin test? Why is it done?

Ans: Burn-in is the process by which device infant mortality failures are

accelerated through application of temperature and stress voltages for

specific periods of time. The goal of burn-in is to stress the device as much

as possible thereby accelerating device infant mortality rate. Since scan

based test has higher coverage, scan test vectors are used to stress the logic

portion of the device. This can either be done by performing device level

ATPG by applying external test patterns on onchip LBIST. Memory BIST is

used for memories.

Burnin test is the technique of estimating the life time of the chip by providing stress in the form temperature , voltage and current.

High toggled atpg vectors is generated to sensitize the internal nodes of the chip so that chip will get more stress

Burnin monior pin is availabe for every SoC to check the internal status in the form of pulses.Chip will give pulses till the device burn out..

9. Why do we have different kinds of SDF's like PrimeTime SDF and Verilog SDF when there is an Open Verilog International standard ?

Ans: There are no different kind of SDF like Primetime SDF or Verilog SDF.

When a design is synthesized it creates netlist, which is made of library cells only. The library cells have path delays(Input to Output) and there are also interconnect delay in netlist. SDF means Standard Delay Format. So SDF basically contains the cell path delay and Interconnect delays between cells. The SDF format can be read/understood by all STA/simulation tools. Generarally (1) the SDF can be generated using Synthesis(dc_shell)/STA(pt_shell). This SDFs are used for initial Timing analysis and gate-simulation. (2) post-route tools also generates SDF (or spef and then you can generate SDF from spef) files. These SDF are used for final Timing Analysis and final gate simulation for sign-off/tapeout.

Each libary cell can have max,min or typical delay. so you can generated SDF based on this delay and have different SDF for max delay or min delay or typical delay, but you can not have primetime SDF or Verilog SDF.

10. If for a DFT production set we have both PATH Delay and Transition patterns which scheme should be done first ?

Ans: Its always better to do Path_delay first and then the transition delay.

Path_delay- start flop and end flop are given by the user.(i.e) transition through a well defined path-(More effective)

Transition-Start and end flop is decided by the tool,so trasition may happen through a shorter or longer path-(Less effective)

1)path delay basically targets for all the critical paths in the design.

2)Generate the path_delay patterns for all the critical paths in the design and write down the faults for the same.

3)Generate the trans delay pattern by loading the faults on the path_delay, such that the faults detected in the path_delay are not Re-targeted.

4)The major reason to follow the above sequence is in trans-delay we are not sure weather a transition has really happened through the critical path.

(A)-If we do the transition-ATPG first, we are not sure whether the transition has happened through the critical path,In such case we may not have pattern which may catch the fault through the critical path,but the fault may be detected through some other shorter paths which may add the faults to the detected list.

(B)If we run a path_delay after the above step(A) then we may have a pattern for the critical path,but it leads to the pattern redundancy because we have already a transition

pattern for the same fault.

11. What do we mean by fault simulation in DFT terminology?

Ans: Fault simulation consists of simulating a circuit in the presence of faults (logical faults).

Comparing the fault simulation results with those of the fault-free simulation of the same circuit simulated with the same applied test, we can determine the faults detected by that test.

There are several types of fault simulation techniques such as serial fault simulation , parallel fault simulation , deductive fault simulation e.t.

12. What is meant by coverage in DFT terminology?

Ans: The term coverage can be braodly classified as:

1. Fault Coverage: This is the total number of detcted faults (duing testing) divided by the total number of faults in the design.

2. Test coverage: This is the total number of detected faults divided by the number of testable faults in the design.

Testable faults are the faults in the design that can be tested and observed by the patterns used during testing.

Please note that, here the word "faults" refers to manufacturing defects, as DFT does not merely target functional faults, but rather focusses on circuit structure.

Coverage Calculations are used to measure the effectiveness of test patterns and test generation for a given set of faults.

Ex-

A Basic AND Gate has six faults

SA1/SA0----Input-A

SA1/SA0----Input-B

SA1/SA0----Output-Y

Total faults for a basic AND gate is 6

Coverage calculation goes on how many faults we are able to detect on the same.

Test Coverage is a percentage detected of all detectable faults and gives the most meaningful measure of test pattern quality.

Test Coverage=DT+(NP+AP)*PT_credit)/(total faults-UD-(AN*AU_credit).

Fault Coverage:the fault coverage is the percentage detected of all faults.it gives no credit for undetectable faults.

Fault Coverage=(DT+(NP+AP)*PT_credit)/total faults.

DT-Detected

-DR-detected robustly

-DS-detected by simulation

-DI-Detected by implication

PT-Possiblty detected

-AP-ATPG untestable possibly detected.

NP-not analyzed,possibly detected.

UD-Undetectable

-UU-undectable unused

-UT-Undectable tied

-UB undetectable tied

-UR undettable redundant

AU-ATPG untestable

-AN-ATPG untestable not detected.

ND- not detected

-NC-not controlled.

-NO-not observed.

13. What is IDDQ Testing? Why is it done?

Ans: IDDQ Testing can detect certain types of circuit faults in CMOS circuits that are difficult or impossible to detect by other methods.

IDDQ testing, when used with standard functional or scan testing, provides an additional measure of quality assurance against defective devices.

IDDQ testing refers to the integrated circuit (IC) testing method based upon measurement of steady state power-supply current. Iddq stands for quiescent Idd, or

quiescent power-supply current. Majority of IC's are manufactured using complementary metal–oxide–semiconductor (CMOS) technology. In steady state, when all switching transients are settled-down, a CMOS circuit dissipates almost zero static current. The leakage current

in a defect-free CMOS circuit is negligible (on the order of few nano amperes). However, in case of a defect such as gate-oxide short or short between two metal lines, a conduction

path from power-supply (Vdd) to ground (Gnd) is formed and subsequently the circuit dissipates significantly high current. This faulty current is a few orders of magnitude

higher than the fault-free leakage current. Thus, by monitoring the power-supply current, one may distinguish between faulty and fault-free circuits.

Why do IDDQ Testing?

For functional testing, a tester applies a sequence of input data and detects the results in the sequence of output data. Then, the output sequence is compared against the expected behavior of the device. An advantage of functional testing is that it exercises the device as it would actually be used in the target application. However, this type of testing has only a limited ability to tests the integrity of a device's internal nodes.

with functional testing only, an internal defect could slide by undetected.

The methodology for scan testing is all the sequential elements of the device are connected into chains and used as

primary inputs and primary outputs for testing purposes. Using automatic test-pattern generation (ATPG) techniques, you have the capability to test a much larger number of internal faults than with functional testing alone. The goal of ATPG is to set all nodes of the circuit to both 0 and 1, and to propagate any defects to nodes where they can be detected by test equipment.

Using both functional and scan testing you greatly increases your odds at finding an internal defect, but what if the defect is not controllable or can't be observed? That is where IDDQ testing can help.

14. If one needs to do synthesis/STA with scan replaced FF (not stitched) and need do generate timing and other reports. What should be values of SE, SI and SO pins since design is not stitched?

Ans: We need not constrain the SE, SI and SO pins for synthesis / STA of a scan replaced but not stitched design. But we will not be able to do any test related STA.

15. Can you briefly describe the points to be considered, while re-ordering the scan chain in Physical Design?

Ans: Scan chain reordering needs to respect 3 important Logical constraints.

1. Clock domain timing constraints

2. User specified scan segment positions

3. Minimizing clock domain traversals

each active edge of each clock is considered to be in a separate clock domain. Both edges of a clock and clocks with different timings may be used to control edge-triggered scan flip flops of a scan chain.

In order to construct functional scan chains, two consecutive scan flip flops A and B (A serially driving B)

1) must be clocked at the same time or

2) B must be clocked before A.

In the first case, we say that A and B have compatible clock domains.

In the second case, we say that A and B have incompatible clock domains.

The precedence relationship between scan flip-flops imposed by clock domain timings is translated at the scan segment level. Capture and launch times for a scan segment are respectively deduced from the capture time of its first scan cell (driven by its scan input) and the launch time of its last scan cell (driving its scan output). Therefore, the precedence relationship between scan segments can be established, and thus respected during scan segments reordering. User-specified scan segment positions are respected during scan reordering unless they violate clock domain timing constraints.

The last constraint, minimizing clock domain traversals, takes priority on physical design information because we want our approach to be minimally intrusive in term of adding synchronization latches. Only scan segments with compatible clock domains are reordered.

Reordering a set of scan segments with compatible clock domains consists of:
1. identifying and marking the set of clusters containing the scan segments.
2. Determining the entry and exit points between which the scan segments are going to be reordered.
3. Ordering the previously identified clusters between the entry point and exit points.
4. Reordering scan segments within each of the ordered clusters.

16. What is stuck open kind of memory fault?

Ans: Stuck open fault in memory says that a particular memory cell is not accessible. A single physical line in a circuit may be broken. The resulting unconnected node is not tied to either VCC or Gnd.

In the figure, if A=1 & B=0, then there is no path from either VDD or VSS to the output F. The output F retains its previous value for some undetermined discharge time.

17. In my project we have 3 subchips. We have three DFT modes of operation. In each mode all the pins of particular subchip will be accessible from Top level. During pattern generation I am facing chain blockage at scan cell 0 for one particular mode. Please help me in debugging the issue.

Ans: There can be many reason for this.

1. The design that you are using might not be Scan Stitched.
2. You might not have declared the Clock
3. The asynchronous reset might not be constrained during shift procedure in your spf file (TETRAMAX) or procedure file (Fastscan)
4. You might not have given proper constraints to input pins
5. Improper value of some dft mode control signals

For finding the exact issue. you have to analyze the error using the gui/schematic viewer of the tool

case 1 : You will find Q of the flop shorted with the SD of the same flop

case 2 : X at the clock port of the flop

case 3 : X at the reset port

case4 : You might see more than one input of a gate as X ; hence the tool will not know which path to follow

Case5 : you might see more than one input of a gate as X; hence the tool will not know which path to follow

Please check the I/O buffer which is used on the scan out is in output mode. else there will be a blockage at the scan cell 0. Make sure that all the I/Os on the scan out port are in the output mode and all your scan-in pins in input mode.

Check the two things first,

1. the flop where it is blocked, there clock is coming properly or not

2. Reset for the same flop

18. For my design i generated the atspeed compressed patterns. when i simulated these patterns serially then all of them were passing. i deleivered these patterns(in wgl format) to our tester team. they sent me the scan tool data and asked me y they are getting these errors. some of the errors from that log file are shown below

err nbr|cycle number|pinname|template |inst |nrmoff|frames |registers |E|R

1| 32171 |LCD_D16 |st_0_577 | 49| 35 |LCD_D160| |1|0

2| 49596 |LCD_D16 |st_0_577 | 79| 40 |LCD_D160| |1|0

i tried to debug this problem using cycle based log method of tetramax . but i dint able to figure out the exact flop. i came to know that for debugging compressed patterns through this method or through failpattern log

based method first i need to convert these adaptive scan patterns to basic scan patterns then only i can debug them.

do we have any other method to debug the compressed patterns?

is inst show scan chain number and nrmoff show the failing flop position?

Ans: various pattern generation tool provides diagnostic capability to debug the Compressed patterns, but there is a restriction based on the no of channels used at the top level and the compression factor defined while inserting the compression logic.

It is better to convert those patterns into basic patterns and do a parallel simulation, which is a easier way to debug the failures. Parallel simulation is faster and easier to debug. Tool provides the flop name, number and also the chain number too.

19. On what basis do we select LOS or LOC in DFT? Is there any major run time difference too in the execution of these two concepts?

Ans: It is necessary that the timing path should be same as the functional path. i.e., clocks should be the same in both functional & at-speed test mode. Whatever methodology (Launch On Shift / Launch On Capture) is required to meet this requirement should be implemented. There are other critical factors that will also drive to LOS / LOC implementation.

1. Whether clocks are generated by internal PLLs.

2. Whether, tester can support multiple waveforms on the same clock pin.

3. How do you handle scan enable to switch between shift & capture modes.

FYI, most of the designs that have internal clock generators implement LOC method for testing.

Below are differences between the LOC and LOS

a) For LOS the scan enable has to closed at functional frequency (which may result in gate count increase with addition of large buffers), whereas in LOC the timing on scan enable can relaxed between the last shift and launch cycle.

b) LOS atpg run time is less compared to the LOC for pattern generation.

c) Pattern count in LOS in less than the LOC.

d) Test/fault coverage for LOS is higher than the LOC.

Transition ATPG is meant for detecting slow-to-rise and slow-to-fall faults on a particular node. In order to detect the transition fault, we have to create the transition on the node. We can achieve this in two ways

I) Launch on last shift - In this method, during the last shift itself, we will shift in the required value in to the flop which will create the required transition on the intended node.

Advantages:

1) Tool has the controllability to put the required value in to the flop to cause transition.
2) We will get good coverage as we are launching the value through SD path.

Disadvantages: 1) Scan-enable need to change at-speed, we have to implement pipeline logic in order to support this.

II) Launch on capture - In this method, the flop which creates the launch on the intended node will get the value through D path. Scan-enable will be low during this time.

Advantages: 1) No need to add pipeline logic for Scan-enable.

Disadvantages: 1) We may loose some coverage as the value is launched through D path.

Its always better to go with LOS, The major advantage with LOS is increased coverage and reduction in number of patterns. When the no of patterns decreases, the test time reduces, which in turn reduces the cost for test time to a great extend.

20. In my project I am facing failures in TDL simulation. I have loaded the failing flops signals in the waveform viewer. This is chain TDL in which scan enable is always 1. Q output should follow the SI input. Whenever there is change in SI input Q should change at next clock edge. But I am seeing Q output is changing at the same clock edge when the SI changes. Only for two flops I am seeing this kind of behaviour. All other flops it is working fine.

Can anyone tell me why this behavior is happening. Please let me know anybody has faced this kind of issue.

Ans: The output of the flop should have certain clk to q delay. even in notiming models (this is done to take care of delta delay issues in rtl simulation)

now in ur case : the clock to the two flops (source and destination) may be comming at two different time. This will happen when we knowingly delay the clk to avoid delta delay issue.

Check the clk path to see if both clocks are arriving at same time or not (ideally they should come at same time) if u are using single scan clock for all flops.

When you do zero-delay simulations, the library files used should be supporting this.

If the library model is based on the udp cell definition, then it is more susceptible to race conditions as described by you. There are two options to get rid of this.

1) Use always blocks to define the flip-flop behavior. Use this model instead of the normal library model.

2) Modify the library model to have internal delay for the d and sdi signals.

21. What is the difference between coverage from dft max and coverage from tetramax?

Ans: DFTMAX is a method or a module which is used to implement compressed mode. Compressed mode means we externally we might have say 8 scan chain (8 scan in port and 8 scan out ports) but DFTMAX translate it into say 240 chains internally. The DFTMAX module will have a demux 8 to 240 and a mux 240 to 8.

This will reduce test time. as the shift time reduce by a factor of 30. This will have lower coverage numbers by a small amount. Since we are loosing certain amount of uncontrollablity as well as observability due to increased effort of the tool.

you can make up this coverage loss by loading the dftmax fault list and running a normal non compressed scheme.

TETRAMAX is a tool which is used to generate patterns. for both compressed and non compressed mode/scheme.

22. Can anybody please tell me what tester cycle is?

Can it be less than Scan clock period or capture clock period?

Ans: A tester cycle refers to the reference clock for the tester. It should be the fastest clock for any test pattern.

For a scan pattern, the tester clock can be faster than the scan clock. If clock is driven like a data pin (using NRZ) from the tester, a scan clock period is equal to 3 tester clock cycles (Refer to the waveform table for scan clock that says 0ns D, 50ns U, 80ns D).

23. What is the difference between BIST and NON-BIST tdls?

Ans: 1. BIST : Built In Self Test.

- MBIST/PBIST

- LBIST

Chip will test by itself using BIST. If you say BIST TDL, it will have programming sequence / seed to generate patterns internally from BIST instead from tester.

2. Non-BIST:-

- EDT

- DFTMAX

These TDL's are having test vectors which will be driven from tester to chip.

24. Why first negative edge flops followed by positive edge flops in the scan chain?

Ans: This is not necessary to always have negative and positive edge triggerred flops in scan chain. Actually we can have three combinations:

1) All positive
2) All negative
3) Negative followed by positive

but positive followed by negative is not taken. Since at the intersection of positive and negative flop the data will not be captured. Since at single pulse data launch and capture is not possible. We will require lock up latch.

The rule is there should not be 2 shift during one clock period. So if you put +ve edge flop followed by -ve edge flop, there is a chance of 2 shift (if the clock skew between 2 clocks is small) in one clock period. But if you put -ve edge flop then +ve edge flop, then there is no chance of that. because the +ve edge come in the next period. Or if ur design needs that +ve edge then -ve edge then you a lock up latch (if skew is small)

this depends on the nature of clock involved in your scan design.

Clock nature

1 : RTZ then chain should be : negedge ->posedge
2 : non RTZ thene it shoul be vice-versa

reason is value loaded on first flop shhuld not passed on to next FF in same cycle.

25. What is PBIST?

Ans: PBIST is the TI Internal IP used to test the memories in the design popularly called as Programmable BIST. As design is moved to deep micron, knowing the nature of defect is very challenge in silicon. Hence we want to have programmable BIST to support algorithms that have been developed post silicon.

Key feature:-
=========

1. It can able to support test 8192 memories in the design.

2. Ability to run different algorithm even post silicon.

3. Support ROM based testing.

Programmable Built-In Self-Test (PBIST) is a memory DFT feature that incorporates all the required test systems into the chip itself.

PBIST was originally adopted by large memory chips that have high pin counts and operate at high frequencies, thereby exceeding the capability of production testers. The purpose of PBIST is to avoid developing and buying more sophisticated and very expensive testers.

PBIST or programmable built-in Self Test is a technique of memory testing. The PBIST strategy, as the above responders have mentioned are basically the same as MBIST.

The difference lies in the architecture and the hierarchy.

For the PBIST, there is only a single BIST controller for a group of memories, as against one controller per memory type in MBIST. In PBIST, memories are grouped under different groups, and are treated as separate memory instances. Each memory instance is accessed by a unique RGS, RDS and CSR value. The RDS value selects the return data slice.

The PBIST controller can be accessed through 3 interfaces: VLCT (Very low cost tester), ROM and CPU. This VLCT interface makes the PBIST programmable during testing of the device, as against MBIST, where the algorithms are hard-coded and cannot be changed once fixed.

Elaborating the terms used in the above respond that will add further clarity to the concept:

RGS, RDS, CSR... etc all these are PBIST internal configuration registers used for PBIST programming. All the configuration registers are memory mapped for access by the CPU through the VBUS interface in all modes.

Chip Select Register - CSR[31:0]

PBIST implements a chip select of up to 32 RAMs with a 32-bit register. When a READ or WRITE command is executed, the corresponding Chip Select is driven active and enables that RAM. At reset, the default value of the register is all 0s.

CSR is normally the same as the 32-bit decoded value of the 5-bit RDS. For a memory to be selected, the corresponding CSR bit pointed to by the RDS register should be set to 1. More than 1 CSR could be turned on at the same time to excite more than one memory simultaneously, but the return data will always be selected from one memory pointed to by the unique RDS.

We have a 32 bit RAM Configuration Register (RAMT[31:0]) which is internally divided into 6 registers, none of which have a default value after reset.

RGS, RDS are part of RAMT.

RAM Group Select [7:0] (same as RAMT[31:24])

In order to support more than 32 RAMs, this RAM grouping address register has been provided. This extends the total number of RAMs that can be tested from 32 to 8192 RAMs of 32 bits or less. RAMs with data width larger than 32 will be tested as more than one RAM and data logged separately.

Return Data select [7:0] (same as RAMT[23:16])

This register is for selecting memory return data for selected RGS group. 8 bits of RGS and 5 bits of RDS give flexibility to test 8192 RAMs of 32 bits or less.

26. What is retention algorithm for memories?

The retention program usually involves writing a checkerboard pattern to the memory, waiting for a pre-determined amount of time (the retention spec) and then going back to read the memory, making sure the contents did not change.

This algorithm can be included in a BIST implementation - the controller does the write, and then asserts a signal telling you that it is waiting. After the specified retention time, you assert a resume signal, and the contents are read back. You would then do this for the inverse data pattern.

27. What is D2 violation and how to solve?

Mentor ATPG - the D2 indicates a copy cell that gets the same data as an upstream flop in shift mode, and may or may not get the same data during capture. It's a violation when they get the same data during capture, I believe, and will reduce fault coverage. If you set split capture cycle on, it should be resolved (consult your tool docs, and it will tell you all this).

28. How to decide the number of Scan chains?

Number of package pins available for scan I/O,

Size of the design,

EDA tools available for use (w/ or w/o compression?),

The targeted ATE (available pin count and vector memory),

The test run time desired.

They're all inter-related, so you start usually with the first two, try to figure out whether or not you're going to fit on your targeted tester, and go from there - make the trade-offs...

29. How the compression technique factor affects the Number of scan chains? Is number of Clock domains also a factor?

Clock domains are a factor, yes, but sometimes people will combine clock domains into the same scan chain. That's not uncommon, and will work if the clock skew is managed, and the tool puts lockup latches in there.

Compression affects the number of scan chains, of course, since more compression generally uses fewer external scan chains.

30. What’s the procedure to generate pattern in TFT LOC Using Fastscan tool.

Shall i generate PI to CLk, CLK to CLK & CLk to po patterns seperately.

I would say that normally, you'd just create clock to clock patterns, not PI to clock and clock to PO (so just mask pi's and po's). The clock to clock are really the main objective of transition fault tests.

31. How to meet coverage goals?

You need to review the untestable faults to determine why they are not getting covered.

The first thing to do is make sure that all the faults that should not be targeted by this test are dropped and not considered for test coverage.

Then find the logic on your chip with the most untested faults. Pick one of the faults in this area and determine why you can’t generate a test for it. You can use the interactive debugger to do this. You can manually create a test for this fault and as you do this figure out what stops you and the ATPG. The interactive debugger might be able to help you with this using it's analysis features.

32. Once a die is tested, can the pins used for scan testing need not be brought out when the die is packaged as IC? Does this have any big advantage?

No - you don't have to bring them out, but then you can't re-test w/ scan at the package level. Normally, folks don't have dedicated scan in/out pins anyway, they share them with mission-mode pins, so they end up getting bonded out anyway.

33. Why should we force the bidis to Z? Can we not use these ports as scan_in and scan_out of our various scan chains? If we use them, then how should we constrain them in the load_unload procedure? Will forcing these ports to Z affect coverage? Will it mean that we cannot put any value on these lines thereby constraining the tool to choose the other ports only for applying input stimulus?

This is normally done for just the bidis that are scan outputs. By constraining them to 'Z', it tells the tool that the pin is working in output mode, and therefore it can then trace the scan chain properly (the chains are traced from output to input).

While shifting you want to prevent conflicts on the non-scan port bidirect pins and to force the correct direction on any pins that are used as scan ports.

You can force Zs on bidirec pins to prevent conflicts.

You have to add hardware and control signals to force the scan ports in the correct direction while shifting.

You can improve test coverage by adopting the TEI methodoly that LSI logic uses. They have a pin that forces all bidirects to be off with the pin. The test are applied with strobe, wait and clock cycles (for stuck at tests).

The strobe cycle has all of the bidirects in the same direction as specified by the ATPG and the TEI pin is disabled.

The wait cycle applies Z to all the bidirect inputs and turns TEI on so all the bidirects will be in input mode.

In the clock cycle all bidirects are in input mode and you drive the values that were on the bidir busses from the input pins and you apply the capture clock.

This prevents bidirect conflicts from affecting test coverage, so your coverage is higher and require fewer vectors.

34. Can a C2 violation occur for a set/reset signal? I am getting this violation for a signal that is identified as a set/reset signal by the tool when "analyze control signlas -auto" command was used.

yes, C2 can happen for set/reset signals. For both Mentor and Synopsys tools, at least, set/reset signals are considered clocks. This DRC violation says that there is a defined clock that does not actually do any work in the circuit (maybe it is replaced in scan mode). To fix it may be as simple as taking that clock out of your ATPG scripts.

35. The time at which the scan chain is put in functional mode can vary depending on the test we are carrying. Given this, how can there be a common test mode pin for all the scan chains?

Test mode pins are typically not the same as scan-enable pins. One or more scan-enable pins (signals) are used to toggle between functional mode and scan mode. These are what you seem to be referring to. Typically different scan-enable signals are needed for at-speed testing to handle things like multi-cycle paths and inter-clock domain paths.

Test mode pins are typically used to put the circuit in test mode and are therefore generally global (static) signals. For example, a test mode pin could be used to share pins between their functional use and as scan I/O.

I am assuming that the test mode pin (irrespective of the number of scan chains) is used to control unwieldy circuits during testing.

36. One thing I am not able to completely appreciate is whether there is an issue while sharing functional pin for testing. Does it in anyway reduce the coverage?

Not if it is handled properly. You need to ensure that a test mode exists where the functional paths to the shared I/O are accessible. For example, you may have a test mode where scan testing is performed with the shared I/O connected up to scan chains and a separate test mode with the shared I/O in their normal functional setting where they can be tested say with boundary scan.

37. what is the command to be used in RTL compiler to add a mux at the PI which, is used as a shared scan enable signal, with test_mode as its select

define_dft test_mode -name test_mode -active high TM

insert_dft test_point -location -type control_node -node scanenable -test_control test_mode

38. when doing DFT scan insertion which of the following is true or is a better approach?

1. Up to three additional pins are required to implement this type of scan. Only the SCAN ENABLE pin must be dedicated; the remainder of the pins(scan in, scan out) can be shared with primary inputs and outputs.

2. Up to four additional pins are required to implement this type of scan. Only the TEST MODE pin must be dedicated; the remainder of the pins(scan en, scan in , scan out) can be shared with primary inputs and outputs.

First you will of course generally use more than one scan chain and often need more than one scan enable (SE) signal, so your 3 and 4 pin statements don't really hold true. The real question you're asking is if the SE signal(s) must be dedicated or not or can a TM signal be used so that the SE signal can be shared. The answer is that a TM signal can indeed be used to share the SE signal(s). This is generally the prefered solution as very often the design requires other internal test settings which mush be controlled by a dedicated TM signal.

39. Why Full Scan is considered combinational?

Now what I think about is that one can control the sequential part by using the scan path. This is done by clocking in the needed sequence to set up the state of the machine and then applying the PIs (primary input vectors) to the combinational inputs.

Thus you one can easily see the response of the machine without having to go through the state machine in its originally specified way. Thus we become independent of the state machine in some way.

Thus using scan we 'reduce' the sequential machine problem down to a 'combinational' problem.

By definition, Full Scan means that ALL flip-flops in the design are converted into scan flops. When the scan-enable signal is inactive, the flip-flops accept data from their functional inputs and the circuit behaves in its intended sequential nature. When the scan-enable signal is active, all flip-flops accept data from their scan input, providing full control on the values that get loaded into them. In this mode, all sequential depth is removed leaving only a combinational circuit to test.

40. It is very normal in our designs to have multiple clock domains.

In that case do we create scan chain for each clock domain?

Also, is the scan clock different from normal clock used during normal functionality?

Are there issues in scan testing when the clock is generated internally (say using PLL)

Yeah.. we need to create seperate scan chains for each clock domain..

same clocks can be used as scan clocks as this will reduce extra pins.

After going through some theory on DFT, I found the following answers:

1) the functional clock is bypassed for scan testing. So clocks in multiple domains can be clubbed into a single chain with a single clock if DC testing is the only target

2) About the pll also, the answer is same since the internal clock is bypassed and scan clock is used, the pll remains inactive during scan testing.

41. By full scan methodology do we mean that every single flop in the design is a part of the scan chain? And if we have multiple scan chains instead of one, as it is in present designs, can it still be called full scan methodology?

In a perfect world, full scan means every flip-flop, but in the real world, many flops can be unscanned, and the design is still considered full scan. In some cases, the ATPG tool can test through unscanned flops without a major impact to fault coverage. Designs using one or many scan chains are equally valid as full scan designs.

Apart from the conventional mux FF scan architecture, there are many others like the Level Sensitive scan and the clocked scan etc. How are these better or worse than the muxed FF technique?

LSSD is really a sub-specialty in the industry as a whole only a few companies use it, but it is effective. For scan purposes, it does not suffer from the hold time issues that mux-scan normally does, but area-wise, it's not as good.

Clock-scan uses a separate scan-clock for each flop - I've never seen it used in industry, but that's just me. The problem with it is that you must route two clock trees around the chip instead of one - a virtual show-stopper in these days of congested routing.

42. What are the things to be considered in scan stitching?

Scan stitching is done can be done in one of three ways:

1) Use the synthesis tool (DFT compiler or equivalent)
2) A DFT scan tool (such as DFT Architect)
3) The place and route tool

Some of the things to think about are:

1) How many scan ports do you have available? This will determine, in part, the number of scan chains. The scan chains need to be balanced in length to be as efficient as possible, for test-time/data volume.

2) Clocks - will you use flops from different clock domains in the same chain? If so, the clock domains need to be *reasonably* skew balanced for the chains to work properly. Also, lockup-latches will be necessary where the scan path crosses the clock domains for safe operation (if stitching.

3) Are there negative edge flops in the design? If so the tool will always place the negedge flops, as a group, ahead of the posedge flops.

43. why first negative edge flops followed by positive edge flops in the scan chain?

Well that's so the chain will shift data in and out properly. The idea is that each bit of the data shifting into the chain should traverse the chain one flop at a time.

Given a clock that is defined with an off-state of 0, a positive edge comes before a negative edge in time, right?

Now imagine in a scan chain, a posedge FF followed by a negedge FF. During any given clock period, the data that is latched into the posedge FF will also be latched into the negedge FF as well - in the same period. This is called a feed-through, and is generally not an optimal situation for the ATPG.

However, if the insertion tool puts the negedge flops all grouped at the front of the chain, then the boundary between the negede flops and posedge flops will be a negedge FF followed by a posedge FF. Since the positive edge of the clock comes before the negative edge, the data will be properly held up at the negedge FF until the next clock period.

I am doing scan stitching for a block which contains say two instances of prestitched blocks.

I need to connect top-level scan ports to scan ports of these blocks instances.

For example:

top block contains 5 chains in that 3 chains stitched for top-level logic.

Now for remaining scan ports I need to be connect the scan ports of stitched sub module instances

Say module is M1 and instantiated two times (M1 is stitched block)

B/C/D/M1_inst1

X/Y/Z/M1_inst2

I need to connect the top level block scan ports to scan port of the submodule inst B/C/D/M1_inst1.

As shown below

Scan_in[3] to B/C/D/M1_inst1.scan_in[0]
Scan_en to B/C/D/M1_inst1.scan_in[0]
Scan_out[3] to B/C/D/M1_inst1.scan_out[0]

Similarly for other instance.

The requirement is to maintain the hierarchy.

47. I am using RTL (RC) complier for scan stitching.

Is there any command in RC or DC for connect the ports?

I tried the below DC command but this is not supported for the available version W-2004.12-SP5

****

connect_pin [get_pins U1/Z] [find pin mid1/bot1/U3/A]

In DC, you will need a DFT compiler license to stitch is properly, as it does more than just connect the scan chains. It checks for any DRC errors, so your chains are intact. The DFT compiler documentation asks you to create a CTL model of the sub-blocks, so I am not sure if it is applicable to your implementation.

Without DFT compiler, you can try to hook it up manually, then try to extract the chain using an ATPG tool to see if the chains are intact.

48. I have in my design 3 types of flops.

1. CLK domain posedge

2. CLK domain negedge

3. RCO_CLK domain posedge only.

Is it feasible/possible to put them in 1 scan-chain ?

Yes, it is possible.

Assuming that CLK is a Return-to-Zero clock (0->1->0 pulse), you would stitch the negedge CLK domain flip-flops before posedge CLK domain flip-flops, i.e., negedge CLK FFs are closer to scan input, and posedge CLK FFs are closer to scan output.

RCO_CLK domain can be stitched at either end of the chain. However, if CLK has a significantly larger clock tree than RCO_CLK clock tree, then it is better to put RCO_CLK domain FFs at the end of the chain. Otherwise, you may need to skew your clock timing on the ATE.

This means your chain will look like this:

ScanIn -> negedge CLK -> posedge CLK -> RCO_CLK -> ScanOut

49. I would like to know about scan pin sharing with bidirectional pins.

I added control logic for these pins using or & and gates to make them work as inputs and outputs respectively. But during scan chain tracing its giving error as it is not able to trace from bidirectional pins shared as scanouts.

Constraining these pins to Z ,the dftadvisor traced the scan chains.

Why is this so even after adding control logic?

The correct way to deal with this is to constrain the bidi to Z, if u want to use it as SO or to mask the o/p of the bidi , if it has to be used as SI (as adviced by FS).

By add control logic, u'r basically modifying the design, which may not be the correct way to go about it, as it may affect the functionality.

In any case, after u added the control logic, did u constrain the control logic i/ps during atpg?

50. How to fix S29 violations (Dependent slave may not hold the same value as master) The design basically has a LOCKUP latch as master feeding a scan-flop. Could you please explain why this violation occurs and how to go about fixing it. It should not be an issue. But can you be secific which tool is giving violation?

I have used DFT Advisor to insert a scan chain, but it seems the tool screwed up my netlist. It has connected the clock inputs of some of the flip flops to the output of some combo blocks. I have heard some people say DFTA is not a good tool for the purpose of scan insertion. Can you tell me what must have gone wrong.

Another thing that I have observed is: when i take the scan inserted netlist and perform timing analysis on it using DC, I find that the tool disables some CLK to Q timing arcs of some flops in order to avoid combo loops.

At least, i never saw this kind of behavior in DFTA. i have been using DFTA for last 5years. Did you run equivalence checking between your prescan netlist vs post scan netlist. Usually DFTA is not doing anything other replacing the flops to scan. Sometimes if you already have combinational feedback loop, it tries to break that path. you can control this, by adding a test logic.

51. Could there be any issue due to floating nets or undriven nets?? in scan insertion.

After doing the equivalence checkin only I came to know how exactly the connections were in the post scan netlist(by looking at the schematics). I do not have any combinational paths in the clock tree in my pre scan netlist. This sounded very strange to me even. But I could not find any way out.

When I tried to run drc check in the dft mode there were no violations. But when I moved on to fastscan I actually encountered T3 and T5 violations in drc. And then I tried to debug using the schematics of conformal tool(LEC) since I do not have DFT Visualizer license. Thats when I came to know what happened actually.

52. In transition fault detection process there should be two vectors - for intialization and for transition. I want to understand when exactly they applied? Doesn't transition propagation vector corrupt initialization vector? Can you please explain in steps?

Here are the steps followed during transition fault testing(BRoadside)

1. load scan chains

2.Force PI

3.pulse clock

4. Force PI

5.measure PO

6.pulse clock

7. unload/load scan chains

Clearly you see, not possible for the corrupting intialization vector.

Why latches (other than lock-up and clock gating) in design are bad for ATPG? We always loose coverage by putting

them on transparent mode by constraining enable signal. How can we increase this coverage?

I think you answered your own question regarding why we lose coverage,

"putting them on transparent mode by constraining enable signal".

Any logic driving the enable signal will lose some coverage due to this constraint.

If the enable signal is driven by a large cone of logic, you can increase the coverage by adding a "force transparent on" gate close to the enable of the latch, and add an observe flip-flop just before the added gate, so that most of the faults in the logic cone can be recovered.

Without modifying the circuitry, recovering fault coverage in ATPG really depends on how these latches are used in the functional path.

54. How PLL generates two at-speed clocks during launch and capture cycles of LOC at-speed testing? Is there any circuitry which generates two pulses after scan loading?

Yeah, you have to design that circuitry, but it's design dependent.Many EDA companies now provide solutions for this. Commercial on-chip solutions for LOS (launch-on-shift) based testing are also available.

I understood that there should be seperate clock control circuitry to select two at-speed clock pulses from free-running PLLs for transition fault testing. What about stuck-at testing, in this case we need only one pulse from PLLs to capture response. Will there be any signal to control this behavior of clock control circuitry?

Well, it's not strictly necessary to have on-chip control. You can source the clock from an I/O, just as long as your ATE can handle the speed you need, and the device can distribute the clock well enough. The advantage of having on-chip control is that you can use a slow tester.

As far as the stuck-at clock goes, remember, you're using the slow external clock to shift data through the scan chains. For stuck-at, you can bypass the at-speed clock control and use only the slow external clock. Yes, you'd need a control signal. You can also, if you want to increase your flexibility, and get better fault coverage, design your clock control to be able to output 2, 3, or even 4 pulses, to catch faults that only get caught with sequential patterns. I've not done it myself, but I've read about such designs.

Transition delay and path delay fault models both seek to describe defects which cause slow transitions on gate inputs or outputs. The difference is that a path delay is more specific - the whole path, from point a to point b is specified. It's a more stringent case than a transition fault. To detect a transition fault, it is only required that the ATPG generates a pattern to propagate that fault to a scanned flop. The path selected may not be the worst case path.

Most people in the industry begin with transition faults, because the ATPG can generate patterns with decent coverage more easily. Then if desired, those patterns are augmented with a much smaller set of path delays that are determined to be critical (maybe those with the least margin as determined by the timing tool).

55. I am very new to the field of DFT and ATPG. I have generated test patterns using Synopsys TetraMax. And i don't how to validate these patterns. What should i do ? Does anyone has any tutorials to verify these patterns again the netlist using simulators like modelsim or vcs ?

I normally output Verilog Testbenches of the chain tests and parallel-load (broadside) ATPG patterns, and simulate them against the netlist w/ SDF.

There shouldn't be a lot to this, since the testbenches are automatically generated. You may have to manually add the sdf annotation command (although some tools have ways of allowing you to automatically add that).

56. I need to know how to implement simulation about Bridge faults?
I mean is the same way like stuck simulation (for example to activation the stuck line with the opposite value) If no, then how to do the simulation?

For bridging fault simulation, there are two things that are different from stuck-at:
1. Bridging fault list
2. Bridging fault behavior

It is impossible to use all pairs of nets/nodes in the gate level netlist as a bridging fault list in practical circuits, so people typically extract physically adjacent nets (with perhaps some minimum length of adjacency) to be fault sites. In absence of physical information, some folks tried logical proximity with varied degrees of success.

When two nets are bridged together, and the two nets were driven to different logic levels, the result logic depends on:
1. The drive strength of the cells driving the 2 bridged nets
2. The bridge resistance
3. The logic threshold of the cells inputs driven by the bridged nets

Some simplistic models that are often used to model the faulty behavior are wired-AND, wired-OR, A-dominate, B-dominate, or some other combination of these models.

You can read more about this in the following paper:
"A Comparison of Bridging Fault Simulation Methods", ITC 1999.
http://ieeexplore.ieee.org/xpls/abs_all ... ber=805783

57. what is the difference between sequential and combinational atpg?

Combinational ATPG simulate one clock pulse on the circuit. Sequential ATPG simulate multiple pulses.

58. What is parallel scan chain loading for simulation? Do we force scan in values for parallel loading? How it works? How it is different from serial loading?

parallel scan chain loading means forcing the scan in port of the flops parallely.

serial loading means Serial shift register. first serially shift in the values into the registers by pulsing the clocks(no of flops)

When running atpg pattern on to a scan inserted netlist in a commerical testing tools. Is there any I can find out which atpg pattern is sent into which scan chain of the scan-netlist.

When dealing with test data compression, it's hard to know exactly which chain is being tested at any one time. Some tools may be able to report which chain(s) are tested for a given pattern, but then the answer may come back that several chains were being tested at once.

It would be different for straight-ahead scan, because you know exactly what path the data takes as it goes in and comes out of the scan chains.

59. How does "scan compression logic (XPAND + X-Compact)" work ? Can it generate any arbitrary test vector ?

Both XPAND and X-Compact techniques are based on combinational logic(XOR gate). It can use test patterns generated from any atpg tool. XPAND works on Illinois scan idea. The problem is that when there are X's present at the X-Compact end, test coverage gets reduced.

Reference:

[1] Subhasish Mitra, Kee Sup Kim, " XPAND: An Efficient Test Stimulus Compression Techniques", IEEE Transactions on Computers, vol 55, pp 163 -173, Feb 2006.

[2] Subhasish Mitra, Kee Sup Kim, " X-Compact: An Efficient Response Compaction Technique", IEEE Transactions on CAD of Circuits and Systems, Vol 23, pp 421-432, March 2004.

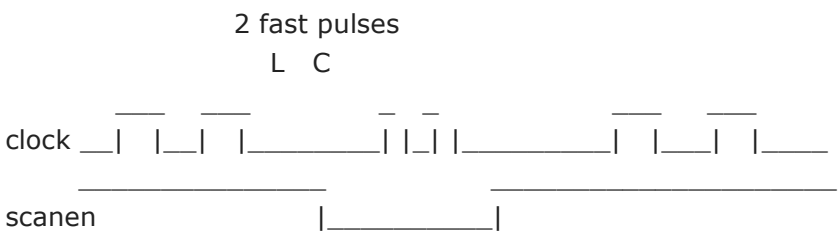
60. During the process of ATPG, I encountered a term called clocked POpattern. Could someone throw some light on what are these patterns ?

Clock PO patterns are special patterns meant to test primary output values when those primary outputs are connected, directly or indirectly, to one of the scan clocks (usually through combinational logic or just buffers).

61. What is difference between launch on shift and launch on capture?
Advantage and disadvantage of each?

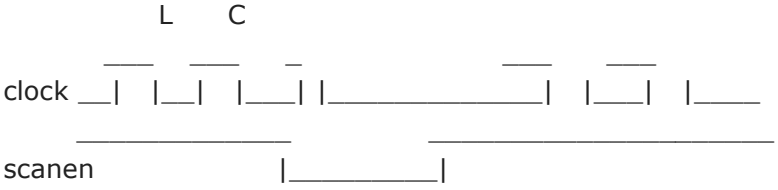
AC scan test is a 2 pattern test, the first pattern launches a transition at the soure flip-flop(s), and the second captures the transition at the destination flip-flop(s). Hence, we need two clock pulses for each AC test.
There are two ways to achieve this. The first way is to add another clock pulse during scan capture (while your scan enable is inactive). This is called Launch on Capture (LOC).

Code: Launch on Capture



The other way is to rely on the last shift clock (while scan enable is still active) to launch the transition (1st pattern), and use 1 clock pulse in the capture cycle as the 2nd pattern to capture the transition. This is called Launch on Shift (LOS).

Code: Launch on Shift



In general, to make sure the transition reaches the capture flip-flop in time, the delay between the launch and capture cycles should be your cycle time (or the actual path delay, for those who run transition tests faster than at-speed).

As you can see, to run LOS at-speed, your scan enable must switch at-speed also. This is usually problematic in layout, since you need to either treat the scan enable signal as a clock net (require clock tree synthesis with accurate delay/skews), or pipeline the scan enable signal, which increases the area/complexity of your scan enable.

I have seen publications that claim either LOS gives you higher transition fault coverage than LOC, or vice versa. I believe this is design dependent, and it depends on the complexity of the logic cone(s) driving the source flip-flop(s). If the logic cone(s) are simple, it gives ATPG a greater degree of freedom to generate the appropriate 2nd pattern in LOC. Notice that the 2nd pattern in LOS is always 1 bit shifted from the 1st pattern. On the other hand, if the cone is complex, it may be hard to generate the appropriate 2nd pattern through the logic, making LOS coverage numbers more attractive.

62. What all needs to be taken care in scan stitching to get the good coverage?

If you are using Mentor DFTAdvisor or Synopsys DFT Compiler, cleaning up pre-stitch drc errors and most of the warnings (especially clock warnings) will generally lead to good fault coverage.

If coverage is still low after cleaning drc errors/warnings, then there may be issues inherent to the design that causes low coverage (redundant logic, complex reconvergent fanouts, black boxes, constrained nets, etc.)
Both Mentor and Synopsys tools provide ways to analyze low fault coverage in their ATPG tools. Also, some RTL analysis tools may be useful to find these kinds of problems (see http://www.dftdigest.com/miscellaneous/ ... -for-test/)

62. How to toggle reset to get coverage ?

If the reset is asynchronous (and properly bypassed during scan), you can declare the reset pin as a clock during ATPG, and ATPG will toggle it accordingly to get faults on reset pin.
If the reset is synchronous, you can treat the reset pin as a normal data pin, and ATPG should be able to cover faults on the reset.

Be careful, however, if you run transition fault ATPG. Reset usually cannot toggle at-speed, so you may not want to declare the reset as a clock when running transition fault ATPG.
You can also try to run the patterns that toggle the reset as a clock pin at a reduced speed on the tester, if you worry about transition fault coverage on reset.

63. What are Pattern Faults? I have seen the definition in cadence ET manual but didn't understand.

A pattern fault is a fault model created by IBM Testbench folks (later acquired by Cadence and became Test Encounter). Instead of using a standard fault model like stuck-at, transition, etc., you can use truth tables to describe what the fault free and faulty behavior is for each gate type.
The advantage is that you can use pattern faults to guide ATPG to generate patterns that would not be generated with other fault models. For example, in a 2 input AND gate, to cover single stuck-at faults, you need only 3 patterns, 01, 10, and 11. If you want to force ATPG to generate a pattern for the 00 case in every AND gate, you can define a pattern fault with 00 as its sensitization.

A pattern fault is a mechanism used by Encounter Test to model static or dynamic defects that are not easily, or may be impossible to be represented by stuck-at pin faults. A pattern fault is basically a statement of 1) the requirements to excite the defect being modeled; and 2) the effect the defect being modeled has on the circuit once it has been excited. For the current release of Encounter Test, fault excitation requirements are specified as logic values required to be on circuit pins or nets of a (sub-) circuit. For dynamic pattern faults, the required values include a (two) pattern sequence or required values to excite the defect.

Encounter Test automatically generates pattern faults to represent defects not possible to represent as pin stuck-at pin faults on LATCH, TSD and XOR primitives. The customer or technology library provider can add additional pattern faults for any sub-circuit used in the circuit design being processed. These user defined pattern faults will be repeated in the Encounter Test fault model for each instance of the sub-circuit.

64. What is the difference between test pattern and test vector?

The terms are sometimes used interchangeably. Me, I tend to say that a test pattern is a set of test vectors.
However, the ATPG tools generally give you stats that claim a certain number of 'patterns', but personally, I would call them 'vectors'.

65. What is a BUS Primitive and clock_PO pattern?

A bus primitive is just a DFT model of a bus - a net that has more than one driver. It's important that you constrain it during test.
A clock PO pattern is a pattern that measures a primary output that has connectivity to a clock. So if a clock signal propagates through combinational logic to a primary output (PO), an ATPG vector can be created to measure the results of that propagation.

66. Why do we go for toggle fault modeling? I think suck-at model also detects the faults which are detected by toggle fault model. How tester applies patterns during toggle fault detection?

Toggle faults are used mostly for code coverage metrics (for verification). You are right that stuck-at is a super-set of that. I myself have never put 'toggle' patterns on a tester.
Toggle coverage is also used to measure the effectiveness of burn-in patterns.
In the old days, where CMOS technology is not as leaky, some folks used toggle patterns for IDDQ testing. However, pseudo stuck-at faults were proven to be far superior than toggle patterns for IDDQ.

67. Effect of C4 violation on ATPG?

If you don't declare rst as a clock, and constrain it to its inactive state, can you get Tetramax to pass drc?

The C4 violation is telling you that it can't make a difference in the state of any flop when it tries to toggle rst - so maybe it's not a reset after all. Or maybe it's gated off in scan mode?

In this case, like Siyad said, maybe remove it from consideration as a clock, and things will clear up. And then you might want to look into the design and see if you intended to render it ineffective - it will result in a loss of coverage...

When I did observe the fanout of the reset, it indeed is getting gated in the design.

Now, I am trying to bypass this gated logic of rst manually by inserting a mux with one input of mux as rst and other input as gate output and the test mode signal as the select line of mux and then connect the mux output to the flop rst pin. Is it okay to do this. Can this solve the C4 violation at the post dft drc stage of the design.

Now when I utilize this scan inserted netlist with the above modification (mux) and the .spf to generate coverage with Tetramax. Will the effect of C4 violation be subsided.

68. What is the difference between FastScan and Flex Test?
On what basis do we select between these two tools?
FlexTest is a limited capacity 'sequential' ATPG tool - originally targeted for use with small, non-scan or limited scan blocks. It can also give you fault coverage of a given pattern set for the same. It's not used widely anymore, that I know of. There's really no decision to make 'between' these two tools, because they are for a different purpose. FastScan is for scanned circuits.

69. Why we loose coverage when we constrain pins?
In general, whenever you constrain any pins of your device, you take away the ability of the ATPG to toggle that pin and check it (and its effects) in both states. Sometimes when you constrain a pin, it will have negligble effect. Sometimes it will have far ranging effects on fault coverage.

70. My design contain more clock gating logic, how it affect DC and AC (At speed) coverage?
A general mode of Clock Gating Cell contains two pins enable(EN) and test enable(TE) pins. (Enable of latch = EN or TE)

We will be working on TE pins in testmode.
If TE is connected to Scan Enable then entire Clock gating cell will be covered during ATPG. (All faults will be covered)

So I dont think there is any coverage loss during ATPG

Please see follwing link for more details
http://www.vlsichipdesign.com/clockondemand.html

71. What will be the test coverage of a design before creating patterns by ATPG(immediately after changing to atpg mode).

When you change from setup mode to ATPG mode in pattern generation, the tool performs learning analysis on the design and identify some of the faults in the design (classifies themas DI faults- Detect Implication), most probabily these faults located in scan chain path. At this time ATPG tool generates no patterns.
My guess is that the fault coverage is basically all stuck at faults along the scan path, plus half the scan enable, scan mode, and some clock and reset faults, right? Basically anything that would be detected by the scan shift pattern, right?
If you write out a complete fault list before starting to create patterns and analyze it, maybe that would give you some clues.

This is typically the coverage that you would get along scan-shift-path (SI, Q pins of flop, part of clock, reset, scan-enable, some part of iopads used scan scanin/scanout). I have seen it be roughly between 12% and 16% depending on design size.

Also, you nomenclature indicates that you are referring to Mentor ATPG tool. All other ATPG tools do the same (btw).

It is not exactly correct to say that we will get coverage without generating any patterns (or before creating patterns).
If we observe the coverage, it will be on scan shift path as you mentioned. TheMentor tool classifies all these faults as DI -Detected by Implication only after generating Chain patterns. So we need at-least one pattern (ie Chain pattern) to detect thses DI faults and get some covergae number.

72. I am trying to improve coverage by appending functional vectors.

What tool are you using? I heard fault simulation time is almost intolerable.

Fault grading is a "grade" on the quality of the fault detection provided by functional vectors. I think you fault grade functional vectors against any kind of fault models. But i have done stuck at only.

Say in my design some flops work at low frequency, in that case,
How can we take care of flops of lower frequency when we do an at speed testing?

It depends upon whether you have independent scan clocks to control the different clock domains. If so you can generate patterns that cover all the domains, and you just need to mask the boundaries between domains.

But that's not the normal case. Many times people will use one scan clock to drive the whole circuit - and in this case, you will need to generate patterns for each clock domain separately, while masking or black boxing all the other domains.

First thing, you have make sure that there is no launch in lower frequcny and capture at high frequency domain. if you have it, add a observable flop and capture it in low frequecy. use a named capture procedure, to launch and capture one clock domain at a time. shift is not a problem bcoz you may have separate scan clocks to shift out.

73. Why at-speed(LOC) pattern count is more than stuck-at pattern count?

For a full scan design, stuck-at ATPG looks at generating patterns in the combinational logic blocks in between scan flip-flops. This is because every scan flip-flop can be treated as a primary input and a primary output for stuck-at ATPG purpose.
However, when testing for at-speed failures, 2 patterns are needed to launch a transition and capture the effect into a scan flip-flop. Therefore, ATPG needs to trace back beyond one level of scan flip-flops to figure out how to get the appropriate "2nd pattern" in a 2-pattern test, which means it has less degree of freedom in how to assign the scan flip-flops to detect a fault, which leads to higher pattern count.

LOC does indeed result in many more patterns that stuck-at patterns. However at-speed patterns can also be applied using a launch-on-shift (LOS) timing protocol. This does not suffer from the sequential ATPG drawbacks as the LOC protocol and therefore results in little to no pattern increase over stuck-at. LOS can be more tricky to implement due to the need for at-speed scan-enable signals (these are necessary as LOS requires going from shift-mode to functional mode within one system clock cycle).

74. I am facing issue w.r.t coverage in compression as it is lesser than bypass mode. However by decreasing the abort limit we are getting better results.But the same is not applicable when I start my initial run with default abort limit. Any reason why this is happening in compression alone?
I do know that increase in abort limit increases runtime as well as pattern count.

It seems to me that fault coverage w/ compression will always be less than bypass mode. In bypass mode there is no masking at the compactor.

Decreasing the abort limit to get better coverage doesn't seem right though, but I'm not an expert at what goes on behind the scenes of ATPG tools. Is that what you meant by 'better results'?

75. What is the difference between redundant, undetectable and untestable faults?
How will a ATPG tool identify a fault to be redudant or untestable or undetectable?
How to analyze redundant faults?

write the fault list of each fault class separately and save it in a file. Now load back the same for which you want to analyze like UD,UR,AU..etc. There is a command option in almost all the tools to analyze the fault which will show the schematic of the fault site.You can use graphical interface options to proceed further.Syntax depends on which tool you are using.

ex- analyze fault /path of fault site/ -stuck 0|1

Now you can start analyzing how many ever faults you want. Its better to have a hierarchical/modular coverage numbers so that you will know which block is giving lesser coverage and analyze those fault sites accordingly.

76. I have been told that any logic in the reset path creates testability problems. I would like to know exactly the problem that would create.
I did a bit of analysis myself. If the internal reset is OR'ed with POR and then issued as one single reset to rest of the logic, still the internal reset comes from some flop or through a cloud that can again be controlled through the scan chain. So we are in complete control of how and when the internal reset is generated and can be disabled during testing. Is the issue because, the combination that is required to disable the internal reset might clash with combination required for testing itself.
Any thoughts?

The bottom line is that the resets to all the flops in scan chains need to be controlled during the scan shift operation. If not, these resets that come from other flops will be randomly resetting others flops duringg this time, which
makes it impossible to scan data in and out...

Depending on the implementation, some people use scan_enable to control the internal resets, and some use scan_mode.

I have couple of comments

1) Internally generated reset is like any other logic during test mode.
If we can control them using our scan pattern, so can we do for the internal reset. I guess the issue here that pattern that is used to mute the internal reset might collide with pattern that is required to test.

2) Even though we can disable such control signal using scan signals, it is probably preferable to keep such signals at bare minimum. This one is a pure guess

You're right - an internally generated reset should be valid during test mode. However, if it emanates, in any way, from a scanned flip-flop, then it must be gated during scan shift. If not, the integrity of the scan chain is broken.

So the bare minimum, as you say, is probably scan_enable.

1) I believe fault simulation is carried out to check the effectiveness of the vector. I believe the vector generated by the ATPG tool is by a deterministic process and should be able to bring out the fault. Why is then fault simulation required. Also, can the ATPG tool itself cannot determine if vector is not an effective one

2) Are there special tools for fault simulation? i.e. fault simulator.
Or the regular simulators provide this option.

Fault simulation is not required to validate ATPG output. However, as deepaky correctly points out, an ATPG tool will internally call a fault simulation tool. This is because once a pattern is generated to cover a specific fault, it is fault simulated to see if it covers any other faults that were not explicitely targeted. If other faults are covered they can be dropped from further consideration (unless a specific N-detect level is a requirement, but that's a different discussion....)

Stand-alone fault simulators do also exist. They are typically used to fault grade functional vectors. Fault simulators are also included in LBIST solutions to fault grade the pseudo random patterns generated by the LBIST.

77. If Fault Simulation is not required to validate ATPG then why do we have an option to perform fault simulation in tools like fastscan.

Fastscan is a versatile test pattern generation and fault simulation tool. It can be used to fault grade any set of scan patterns against many types of user definable fault types. Again, for the fault types than Fastscan is capable of targeting, it is not necessary to perform a separate fault simulation run on the patterns Fastscan generates. However you can for example fault simulate those generated patterns on other fault types.

Many people will use their functional verification test for manufacturing test. They need to fault simulate these vectors to determine what faults are covered and left uncovered to be targeted by an ATPG. It also allows them to determine the quality of these tests.

78. How to avoid C4 violation in Synopsys DFT compiler?

First thing you cannot get C4 in the pre_dft drc, it will be some D violation.

Second, you have not used the command

set_dft_configuration -fix_reset enable (may be you miss to write that here)

Further, I dont think that autofix will solve this problem. It is a capture issue and not scan shift issue. What I think is happening here is that , your reset pin before reaching the flop is gated with testmode and then reaches the flop. The flops are not failing the drc and coming on the scan chain because the reset on the pin of the flops is in inactive state.

You can trace the top level reset pin in your design to see if it is getting gated some where.

In DC you can use the command

all_fanout -from reset

79. debugging compressed patterns failing in tester

well, for debugging its always easier with basic scan patterns corresponding to these compressed patterns. However if you wish to debug with these compressed patterns then you need to find the cycle failing and then find all the corresponding flops in all the chains compressed. This way you will have huge number of flops which could be failing e.g. if you have compression ratio of 100 then for each failure you will have 100 flops which could be contender to this failure.

80. C6 violation during ATPG

set clock_off simulation on
set split capture_cycle on
And if the two commands by kunal does not resolve these issue, then you need to mask these flops.
Normally such violation occurs when Clock is seen as Data by ATPG tool.
For Fastscan the commands would be :
add cell constraint -drc C6

What do you mean by typical scan frequencies?

Typical scan frequency is the frequency that most of the ATE's use when scanning in the data to the scan flops. it is typically around 1-10 Mhz.

Why is Hold time neglected while calculating Max Frequency?

To calculate the maximum frequency of the digital circuit, you will use the following formula.

tCLOCK=tCHQV+tPD(max)+tSETUP.

Here, in all FFs, normally tHOLD < tCHQV, means Clock to Q output delay includes the hold time. So we are not considering the hold time for calculating the max. Frequency.

In scan chains if some flip flops are +ve edge triggered and remaining flip flops are -ve edge triggered how it behaves?

For designs with both positive and negative clocked flops, the scan insertion tool will always route the scan chain so that the negative clocked flops come before the positive edge flops in the chain. This avoids the need of lockup latch.

For the same clock domain the negedge flops will always capture the data just captured into the posedge flops on the posedge of the clock.

For the multiple clock domains, it all depends upon how the clock trees are balanced. If the clock domains are completely asynchronous, ATPG has to mask the receiving flops

Why we avoid latches in the design, even if they provide only cell delay.
Is there any time related issues??

Whenever latch is enabled it will pass whatever is there on its D inputs to Q output. If suppose any glitch is coming on D and latch is enabled it will pass it to q. Glitch always create problem u would be knowing this.

Latches are fast, consumes less power, less area than Flops but Glitches can also come along with these advantages, that’s why we go for flops.

Also Latches are not DFT friendly... It is very difficult to perform Static timing analysis with latches in your design...

While attempting to do scan insertion for a design, I am finding many uncontrollable clock and uncontrollable asynchronous signals like set and reset violations.
Can anyone guide me on how to avoid these violations?

Uncontrollable clocks are those which come out from a combo logic (dividers ...etc)
when doing DFT, all the controls of the FF must be with respect to the TEST EN signal,
in the sense once if TEST EN =1, then automatically the entire chip should be in scan mode .... for this all the clock of the FF should be controllable externally (at pin level).

wherever u get a "uncontrollable clock" or uncontrollable set-reset please do the following
1) MUX the functional CLK with the "test clock" with the TEST EN as the select line and then give the o/p of mux as the input to the FF (so that when TEST EN =1, the test clock will be given to all the FF and when TEST EN=0, the chip functions with normal clock)

What is the difference between a latch and a flip-flop?

- Both latches and flip-flops are circuit elements whose output depends not only on the present inputs, but also on previous inputs and outputs.
- They both are hence referred as "sequential" elements.
- In electronics, a latch, is a kind of bistable multi vibrator, an electronic circuit which has two stable states and thereby can store one bit of of information. Today the word is mainly used for simple transparent storage elements, while slightly more advanced non-transparent (or clocked) devices are described as flip-flops. Informally, as this distinction is quite new, the two words are sometimes used interchangeably.
- In digital circuits, a flip-flop is a kind of bistable multi vibrator, an electronic circuit which has two stable states and thereby is capable of serving as one bit of memory. Today, the term flip-flop has come to generally denote non-transparent (clocked or edge-triggered) devices, while the simpler transparent ones are often referred to as latches.
- A flip-flop is controlled by (usually) one or two control signals and/or a gate or clock signal.
- Latches are level sensitive i.e. the output captures the input when the clock signal is high, so as long as the clock is logic 1, the output can change if the input also changes.
- Flip-Flops are edge sensitive i.e. flip flop will store the input only when there is a rising or falling edge of the clock.
- A positive level latch is transparent to the positive level(enable), and it latches the final input before it is changing its level(i.e. before enable goes to '0' or before the clock goes to -ve level.)
- A positive edge flop will have its output effective when the clock input changes from '0' to '1' state ('1' to '0' for negative edge flop) only.
- Latches are faster, flip flops are slower.
- Latch is sensitive to glitches on enable pin, whereas flip-flop is immune to glitches.
- Latches take fewer gates (less power) to implement than flip-flops.
- D-FF is built from two latches. They are in master slave configuration.
- Latch may be clocked or clock less. But flip flop is always clocked.
- For a transparent latch generally D to Q propagation delay is considered while for a flop clock to Q and setup and hold time are very important.

Synthesis perspective: Pros and Cons of Latches and Flip Flops

- In synthesis of HDL codes inappropriate coding can infer latches instead of flip flops. Eg.:"if" and "case" statements. This should be avoided sa latches are more prone to glitches.

- Latch takes less area, Flip-flop takes more area (as flip flop is made up of latches) .
- Latch facilitates time borrowing or cycle stealing whereas flip flops allows synchronous logic.
- Latches are not friendly with DFT tools. Minimize inferring of latches if your design has to be made testable. Since enable signal to latch is not a regular clock that is fed to the rest of the logic. To ensure testability, you need to use OR gate using "enable" and "scan_enable" signals as input and feed the output to the enable port of the latch. [ref]
- Most EDA software tools have difficulty with latches. Static timing analyzers typically make assumptions about latch transparency. If one assumes the latch is transparent (i.e. triggered by the active time of clock, not triggered by just clock edge), then the tool may find a false timing path through the input data pin. If one assumes the latch is not transparent, then the tool may miss a critical path.
- If target technology supports a latch cell then race condition problems are minimized. If target technology does not support a latch then synthesis tool will infer it by basic gates which are prone to race condition. Then you need to add redundant logic to overcome this problem. But while optimization redundant logic can be removed by the synthesis tool! This will create endless problems for the design team.
- Due to the transparency issue, latches are difficult to test. For scan testing, they are often replaced by a latch-flip-flop compatible with the scan-test shift-register. Under these conditions, a flip-flop would actually be less expensive than a latch.
- Flip flops are friendly with DFT tools. Scan insertion for synchronous logic is hassle free.

What do you mean by scan chain reordering?

Answer1: Based on timing and congestion the tool optimally places standard cells. While doing so, if scan chains are detached, it can break the chain ordering (which is done by a scan insertion tool like DFT compiler from Synopsys) and can reorder to optimize it.... it maintains the number of flops in a chain.

Answer2: During placement, the optimization may make the scan chain difficult to route due to congestion. Hence the tool will re-order the chain to reduce congestion.

This sometimes increases hold time problems in the chain. To overcome these buffers may have to be inserted into the scan path. It may not be able to maintain the scan chain length exactly. It cannot swap cell from different clock domains.

Because of scan chain reordering patterns generated earlier is of no use. But this is not a problem as ATPG can be redone by reading the new netlist.

What is software reset and hardware reset?

Software reset is a reset managed by software by writing a special register bit dedicated for this purpose; it's usually a synchronous reset.

Hardware reset is the traditional reset activated by setting the IC reset pin to '1' or '0' (depending on the convention: active low or active high reset), it can be also generated inside the IC.

In a Design which type of RESET (asyn or syn) will you prefer??

Best Solution is "Assert RESET asynchronously and De-assert Synchronously"

Active Low or Active HIGH?
If active high reset is used, any noise on the reset line can cause a spike and these results in a reset. But if active Low reset is used, this can never happen. So active low is preferred. This is the case where the

reset comes from external to chip, but if reset is internally generated; then this may not be big issue.

Power point of view:
In cases where leakage is significant, active Low reset causes more leakage; as in in-active state the reset is active high and the transistor can leak slowly from high causing more leakage. So active high is preferred for low power.

What is your design style for scanning?

If your design is predominantly edge-triggered, use the multiplexed flip-flop, clocked scan, or clocked LSSD scan style.

If your design has a mix of latches and flip-flops, use the clocked scan or LSSD scan style.

If your design is predominately level-sensitive, use the LSSD scan style.

Is it possible to reconvert the scan flip flop to normal flip flop?

You can convert the unused flops back to normal flops in DC , and the tool does that automatically. The command for the same is "set_dft_insertion_configuration-unscan true".

All the flops which are not on the scan chains will be converted back to normal flop.

can anyone tell me for a design how can we decide how many scan chains we require?... another question is in a particular scan chain how many flops should be put? is it dependent on the technology or is it dependent on designer or on the customer requirement?

scan chian number depend on the clock domain and max flipflops. Don't mix different clock domains and clock edges to aviod timing issue.
and longer chian will increase test time, < 1000k ff's per chain.

There are a number of factors to decide the number of scan chains and length of each scan chain.

1. Look for your ATE specification to find how many scan chains are supported. If length of scan chain is very long, then the tester time increases and hence your cost

2. If you are using scan compression, then your scan compression methodology will determine the maximum length of each scan chain

Whether Positive skew is good for Setup? How?

positive skew is similar to useful skew , it is nothing but delaying capture clock by skew value correct than the normal launch clock value when ever you delay the capture clock surely you gain that my margin which can reduce in your setup violation. If you want to visualize properly, you can think like if you have more period then surely setup violation will reduce correct. it is similar concept.

Negative skew is bad for setup?

similar to my earlier answer here you are reducing the capture clock.

Positive skew is bad for hold? Negative skew is good for hold?

for example: FF1 --> FF2 --> FF3

case 1: All FF[flip flop] has zero skew.
analysis: You have a setup violation path from FF1 to FF2.
Work around : Trying to optimize the datapath from FF1 to FF2 , you cannot do it , as the path is already optimized. Now what is the only option left is playing safely rather marginally with clock.
So you are planning for skewing the capture flip flop clock that is FF2, delaying , so assume you met with the setup violation across FF1 to FF2.

Another scenario comes, you had delayed your FF2 more, so you have a hold violation to FF3.
hope the concept is clear.

In STA report, if we find any hold violation, it can be remedied by adding buffers? Whether it will cause setup violation? Then how much to add and on what basis?

Assuming you have hold violation, you cannot go on apply on adding buffers on the data path. Before doing this you need to check the setup margin across the endpoint and see.
There could be scenario's where in path passing through a cell A you have setup and path passing through cell B input you have a hold violation, you have to apply specify back-trace and apply to that specific inputs only and not on the endpoint register straight away.

Uncertanity you are adding is for your margin's / jitter / process variations and things like that, reducing uncertanity is not a correct option to reduce your violation count. It is like taking from your saving's account and spending!!!

What are the various ways to reduce Clock Insertion Delay in the Design

1. Number of Clock sinks
2. Balancing two different clock frequencies
3. Placement of clock sinks.
4. Placement of Clock gating cells
5. Clock tree buffers/inverters drive strength's
6. Clock Transition
7. Placement of Clock gating cells and the clock sinks
8. Combinational cells in the path of clocks (say clock dividers, muxes, clockgates) .

What are the measures taken in the Design achieving better Yield?

Better yield could be achieved by reducing the possibility of manufacturability flaws. Guarantying the circuit performance, by reducing parametric yield, with process variations playing a major role is a big-challenge.

- Create more powerful stringent run set files with pessimistic spacing/short rules.
- Check for the areas where the design is prone to lithographic issues, like sharp cuts and try to re-route it.
- For via-reliability issues, use redundant vias, to reduce the chances for via-breakage.
- In order to design for yield-enhancement, design systems, this could have optimal redundancy, like repairable memories.
- Optimal placing of de-coupling capacitances reduces the power-surges.
- Doubling the width of the non-critical nets, clock-nets can increase the yield parameter.
- Ensure that the poly-orientation is maintained.

What are the measures or precautions to be taken in the Design when the chip has both analog and digital portions?

Designing for Optimal integration of Analog and Digital

- As today's IC has analog components also inbuilt, some design practices are required for optimal integration.
- Ensure in the floor planning stage that the analog block and the digital block are not sitting close-by, to reduce the noise.
- Ensure that there exists separate ground for digital and analog ground to reduce the noise.
- Place appropriate guard-rings around the analog-macro.
- Incorporating in-built DAC-ADC converters, allows us to test the analog portion using digital testers in an analog loop-back fashion.
- Perform techniques like clock-dithering for the digital portion.

What is local-skew, global-skew, and useful-skew mean?

Local skew: The difference between the clock reaching at the launching flop vs the clock reaching the destination flip-flop of a timing-path.

Global skew: The difference between the earliest reaching flip-flop and latest reaching flip-flop for a same clock-domain.

Useful skew: Useful skew is a concept of delaying the capturing flip-flop clock path, this approach helps in meeting setup requirement with in the launch and capture timing path. But the hold-requirement has to be met for the design.

What is the difference between latches and flip-flops based designs?

Latches are level-sensitive and flip-flops are edge sensitive. Latch based design and flop based design is that latch allows time borrowing which a tradition flop does not. That makes latch based design more efficient. But at the same time, latch based design is more complicated and has more issues in min timing (races). Its STA with time borrowing in deep pipelining can be quite complex.

How many minimum modes should qualify STA for a chip?

1. Scan Shift mode
2. Scan Capture mode
3. MBIST mode
4. Functional modes for Each Interface
5. Boundary scan mode
6. scan-compression mode

How many minimum process lots, should STA be qualified?

1. Fast corner
2. Slow corner
3. Typical corner

How many minimum Timing, Should STA be qualified?

1. Normal delay mode (with out applying duration)
2. On-chip variation mode (duration applied)
3. SI mode (Signal integrity cross talk impact on STA)

What are the various Timing Paths in a Chip?

1. Path starting from Input Data pad and Ending at Register pin.
2. Path starting from clock-pin of a Register and Ending at Data pin of a Register (Reg-to-Reg path).

3. Path starting from output of a Register and Ends at output pad of a chip.
4. Path starting from input data pad and ends at output data pad (pure combo path).

What is meant by virtual clock definition and why do i need it?

Ans: Virtual clock is mainly used to model the I/O timing specification. Based on what clock the output/input pads are passing the data.

What are the various timing-paths which i should take care in my STA runs?Ans:
1. Timing path starting from an input-port and ending at the output port(purely combinational path).
2. Timing path starting from an input-port and ending at the register.
3. Timing path starting from an Register and ending at the output-port.
4. Timing path starting from an register and ending at the register.

How to solve setup & Hold violations in the design?
Ans: To solve setup violation
1. optimizing/restructuring combination logic between the flops.
2. Tweak flops to offer lesser setup delay [DFFX1 -> DFFXx]
3. Tweak launch-flop to have better slew at the clock pin, this will make CK->Q of launch flop to be fast there by helping fixing setup violations
4. Play with skew [tweak clock network delay, slow-down clock to capturing flop and fasten the clock to launch-flop](otherwise called as Useful-skews)

To solve Hold Violations
1. Adding delay/buffer[as buffer offers lesser delay, we go for special Delay cells whose functionality Y=A, but with more delay]
2. Making the launch flop clock reaching delayed
3. Also, one can add lockup-latches [in cases where the hold time requirement is very huge, basically to avoid data slip]

What is the implication of a combinatorial feedback loops in design testability?

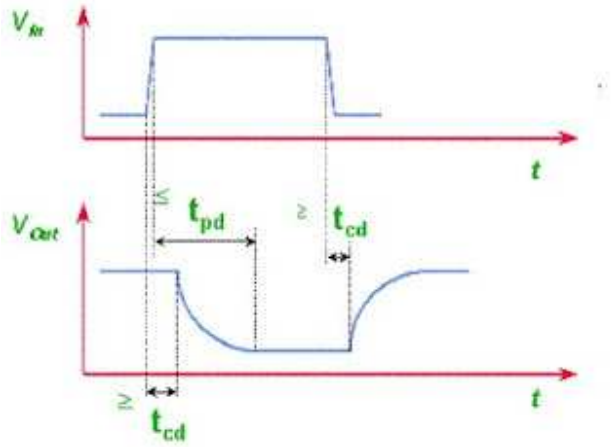
The presence of feedback loops should be avoided at any stage of the design, by periodically checking for it, using the lint or synthesis tools. The presence of the feedback loop causes races and hazards in the design, and 104 RTL Design
leads to unpredictable logic behavior. Since the loops are delay-dependent, they cannot be tested with any ATPG algorithm. Hence, combinatorial loops should be avoided in the logic.

When are DFT and Formal verification used?

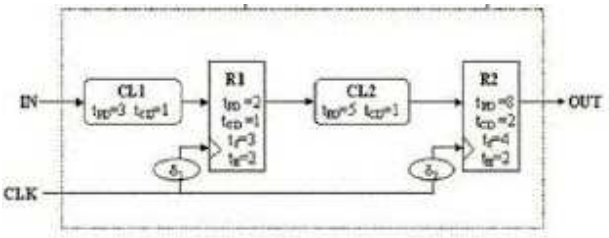
Ans: DFT:
· manufacturing defects like stuck at "0" or "1".
· Test for set of rules followed during the initial design stage.

Formal verification:
· Verification of the operation of the design, i.e, to see if the design follows spec.
· gate netlist == RTL ?
· using mathematics and statistical analysis to check for equivalence.

Ques : What is the significance of contamination delay in sequential circuit timing?
Ans: Look at the figure below. tcd is the contamination delay.



Contamination delay tells you if you meet the hold time of a flip flop. To understand this better please look at the sequential circuit below.



The contamination delay of the data path in a sequential circuit is critical for the hold time at the flip flop where it is exiting, in this case R2.
Mathematically, th(R2) <= tcd(R1) + tcd(CL2)
Contamination delay is also called tmin and Propagation delay is also called tmax in many data sheets.

Ques: Given two ASICs. one has setup violation and the other has hold violation. how can they be made to work together without modifying the design?

Ans: Slow the clock down on the one with setup violations..
And add redundant logic in the path where you have hold violations.

Ques: Suggest some ways to increase clock frequency?

Ans: Check critical path and optimize it.
· Add more timing constraints (over constrain).
· pipeline the architecture to the max possible extent keeping in mind latency req's.

Ques: What are different types of timing verifications?

Ans: Dynamic timing:

a. The design is simulated in full timing mode.
b. Not all possibilities tested as it is dependent on the input test vectors.
c. Simulations in full timing mode are slow and require a lot of memory.
d. Best method to check asynchronous interfaces or interfaces between different timing domains.

Static timing:

a. The delays over all paths are added up.
b. All possibilities, including false paths, verified without the need for test vectors.
c. Much faster than simulations, hours as opposed to days.
d. Not good with asynchronous interfaces or interfaces between different timing domains.

Ques : Can you explain what stuck at zero means?

Ans : These stuck-at problems will appear in ASIC. Some times, the nodes will permanently tie to 1 or 0 because of some fault. To avoid that, we need to provide testability in RTL. If it is permanently 1 it is called stuck-at-1 If it is permanently 0 it is called stuck-at-0.

Ques : What is "Scan" ?

Ans : Scan Insertion and ATPG helps test ASICs (e.g. chips) during manufacture. If you know what JTAG boundary scan is, then Scan is the same idea except that it is done inside the chip instead of on the entire board. Scan tests for defects in the chip's circuitry after it is manufactured (e.g. Scan does not help you test whether your Design *functions* as intended). ASIC designers usually implement the scan themselves and occurs just after synthesis. ATPG (Automated Test Pattern Generation) refers to the creation of "Test Vectors" that the Scan circuitry enables to be introduced into the chip. Here's a brief summary:

- Scan Insertion is done by a tool and results in all (or most) of your design's flip-flops to be replaced by special "Scan Flip-flops". Scan flops have additional inputs/outputs that allow them to be configured into a "chain" (e.g. a big shift register) when the chip is put into a test mode.
- The Scan flip-flops are connected up into a chain (perhaps multiple chains)
- The ATPG tool, which knows about the scan chain you've created, generates a series of test vectors.
- The ATPG test vectors include both "Stimulus" and "Expected" bit patterns. These bit vectors are shifted into the chip on the scan chains, and the chips reaction to the stimulus is shifted back out again.
- The ATE (Automated Test Equipment) at the chip factory can put the chip into the scan test mode, and apply the test vectors. If any vectors do not match, then the chip is defective and it is thrown away.
- Scan/ATPG tools will strive to maximize the "coverage" of the ATPG vectors. In other words, given some measure of the total number of nodes in the chip that could be faulty (shorted, grounded, "stuck at 1", "stuck at 0"), what percentage of them can be detected with the ATPG vectors? Scan is a good technology and can achive high coverage in the 90% range.
- Scan testing does not solve all test problems. Scan testing typically does not test memories (no flip-flops!), needs a gate-level netlist to work with, and can take a long time to run on the ATE.

- FPGA designers may be unfamiliar with scan since FPGA testing has already been done by the FPGA manufacturer. ASIC designers do not have this luxury and must handle all the manufacturing test details themselves.

Ques : What is an LFSR .List a few of its industry applications.?

Ans: LFSR is a linear feedback shift register where the input bit is driven by a linear function of the overall shift register value. Coming to industrial applications, as far as I know, it is used for encryption and decryption and in BIST(built-in-self-test) based applications..

Ques : what is false path? how it determine in ckt? What the effect of false path in ckt?

Ans: By timing all the paths in the circuit the timing analyzer can determine all the critical paths in the circuit. However, the circuit may have false paths, which are the paths in the circuit which are never exercised during normal circuit operation for any set of inputs.
An example of a false path is shown in figure below. The path going from the input A of the first MUX through the combinational logic out through the B input of the second MUS is a false path. This path can never be activated since if the A input of the first MUX is activated, then Sel line will also select the A input of the second MUX.
STA (Static Timing Analysis) tools are able to identify simple false paths; however they are not able to identify all the false paths and sometimes report false paths as critical paths. Removal of false paths makes circuit testable and its timing performance predictable (sometimes faster)

Ques : Consider two similar processors, one with a clock skew of 100ps and other with a clock skew of 50ps. Which one is likely to have more power? Why?

Ans: Clock skew of 50ps is more likely to have clock power. This is because it is likely that low-skew processor has better designed clock tree with more powerful and number of buffers and overheads to make skew better.

Ques: What are multi-cycle paths?

Ans: Multi-cycle paths are paths between registers that take more than one clock cycle to become stable. For ex. analyzing the design shown in fig below shows that the output SIN/COS requires 4 clock-cycles after the input ANGLE is latched in. This means that the combinatorial block (the Unrolled Cordic) can take up to 4 clock periods (25MHz) to propagate its result. Place and Route tools are capable of fixing multi-cycle paths problem.

Ques: You have two counters counting upto 16, built from negedge DFF , First circuit is synchronous and second is "ripple" (cascading), Which circuit has a less propagation delay? Why?

Ans: The synchronous counter will have lesser delay as the input to each flop is readily available before the clock edge. Whereas the cascade counter will take long time as the output of one flop is used as clock to the other. So the delay will be propagating. For Eg: 16 state counter = 4 bit counter = 4 Flip flops Let 10ns be the delay of each flop The worst case delay of ripple counter = 10 * 4 = 40ns The delay of synchronous counter = 10ns only.(Delay of 1 flop)

Ques : Explain about setup time and hold time, what will happen if there is setup time and hold tine violation, how to overcome this?

Ans: Set up time is the amount of time before the clock edge that the input signal needs to be stable to guarantee it is accepted properly on the clock edge.
Hold time is the amount of time after the clock edge that same input signal has to be held before changing it to make sure it is sensed properly at the clock edge.
Whenever there are setup and hold time violations in any flip-flop, it enters a state where its output is unpredictable: this state is known as metastable state (quasi stable state); at the end of metastable state, the flip-flop settles down to either '1' or '0'. This whole process is known as metastability

Ques: What is skew, what are problems associated with it and how to minimize it?

Ans: In circuit design, clock skew is a phenomenon in synchronous circuits in which the clock signal (sent from the clock circuit) arrives at different components at different times.
This is typically due to two causes. The first is a material flaw, which causes a signal to travel faster or slower than expected. The second is distance: if the signal has to travel the entire length of a circuit, it will likely (depending on the circuit's size) arrive at different parts of the circuit at different times. Clock skew can cause harm in two ways. Suppose that a logic path travels through combinational logic from a source flip-flop to a destination flip-flop. If the destination flip-flop receives the clock tick later than the source flip-flop, and if the logic path delay is short enough, then the data signal might arrive at the destination flip-flop before the clock tick, destroying there the previous data that should have been clocked through. This is called a hold violation because the previous data is not held long enough at the destination flip-flop to be properly clocked through. If the destination flip-flop receives the clock tick earlier than the source flip-flop, then the data signal has that much less time to reach the destination flip-flop before the next clock tick. If it fails to do so, a setup violation occurs, so-called because the new data was not set up and stable before the next clock tick arrived. A hold violation is more serious than a setup violation because it cannot be fixed by increasing the clock period.
Clock skew, if done right, can also benefit a circuit. It can be intentionally introduced to decrease the clock period at which the circuit will operate correctly, and/or to increase the setup or hold safety margins. The optimal set of clock delays is determined by a linear program, in which a setup and a hold constraint appears for each logic path. In this linear program, zero clock skew is merely a feasible point.
Clock skew can be minimized by proper routing of clock signal (clock distribution tree) or putting variable delay buffer so that all clock inputs arrive at the same time

Ques: What is slack?
Ans: 'Slack' is the amount of time you have that is measured from when an event 'actually happens' and when it 'must happen'.. The term 'actually happens' can also be taken as being a predicted time for when the event will 'actually happen'.
When something 'must happen' can also be called a 'deadline' so another definition of slack would be the time from when something 'actually happens' (call this Tact) until the deadline (call this Tdead).
Slack = Tdead - Tact.
Negative slack implies that the 'actually happen' time is later than the 'deadline' time...in other words it's too late and a timing violation....you have a timing problem that needs some attention.

Ques: What is glitch? What causes it (explain with waveform)? How to overcome it?
Ans:

The following figure shows a gated clock. The gated clock's corresponding timing diagram shows that this implementation can lead to clock glitches, which can cause the flip-flop to clock at the wrong time.

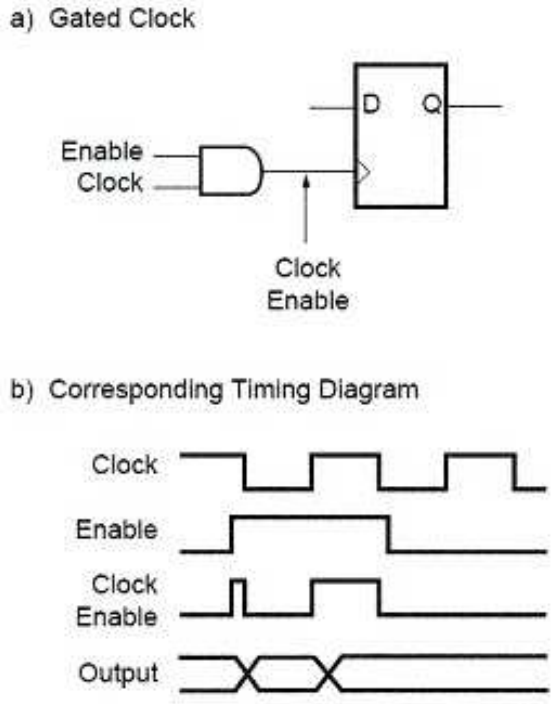
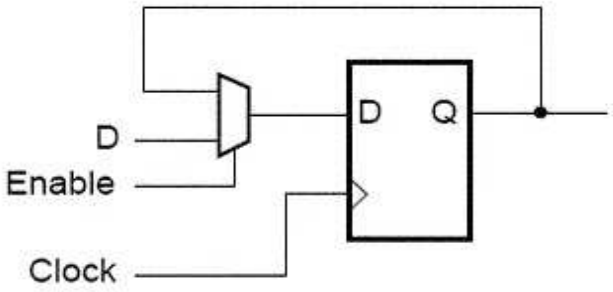


Figure 2-12: **Gated Clock**

The following figure shows a synchronous alternative to the gated clock using a data path. The flip-flop is clocked at every clock cycle and the data path is controlled by an enable. When the enable is Low, the multiplexer feeds the output of the register back on itself. When the enable is High, new data is fed to the flip-flop and the register changes its state

a) Using a Feedback Path



b) Corresponding Timing Diagram

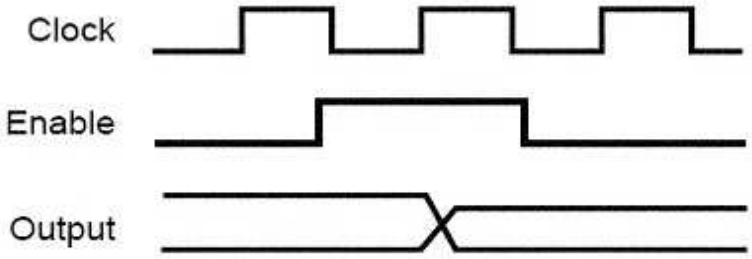


Figure 2-13: **Synchronous Design Using Data Feedback**

Ques : A very good interview question... What is difference between setup and hold time. The interviewer was looking for one specific reason , and its really a good answer too..The hint is hold time doesn't depend on clock, why is it so...?
Ans: Setup violations are related to two edges of clock, i mean you can vary the clock frequency to correct setup violation. But for hold time, you are only concerned with one edge and does not basically depend on clock frequency

Ques :What is Stuck-at fault ?
Ans: A Stuck-at fault is a particular fault model used by fault simulators and Automatic test pattern generation (ATPG) tools to mimic a manufacturing defect within an integrated circuit. Individual signals and pins are assumed to be stuck at Logical '1', '0' and 'X'. For example, an output is tied to a logical 1 state during test generation to assure that a manufacturing defect with that type of behavior can be found with a specific test pattern. Likewise the output could be tied to a logical 0 to model the behavior of a defective circuit that cannot switch its output pin.

Ques : What is Clock Gating ?
Ans: Clock gating is one of the power-saving techniques used on many synchronous circuits including the Pentium 4 processor. To save power, clock gating refers to adding additional logic to a circuit to prune the clock tree, thus disabling portions of the circuitry where flip flops do not change state. Although

asynchronous circuits by definition do not have a "clock", the term "perfect clock gating" is used to illustrate how various clock gating techniques are simply approximations of the data-dependent behavior exhibited by asynchronous circuitry, and that as the granularity on which you gate the clock of a synchronous circuit approaches zero, the power consumption of that circuit approaches that of an asynchronous circuit.

Ques : How to fix the hold/setup violation in scan path (Q to SI path)?
Ans : If the violations are small, as in most designs with well balanced clock trees, the place and route tools can generally fix these by adding small buffers/inverters. Some smart scan reordering tools will take a slightly farther scan flip-flop to avoid hold problems.
If the violations are big, due to large clock skews, the best way to solve this is to add a data lockup latch. I have always wondered why scan chain reordering tools don't take clock skew information into consideration. Not sure if the latest tools uses this vital information.
I think the tools look first to reduce congestion, by reducing the number of clock buffers in the scan path, which will minimize hold time violations, but there are always some left, so your process has to include a step where the post-route netlist is re-analyzed for timing (both mission and scan mode), and any hold violations left in the scan path are manually fixed.

Ques : I have 3 types of flops in my design.
1. CLK domain posedge
2. CLK domain negedge
3. RCO_CLK domain posedge only.
Is it feasible/possible to put them in 1 scan-chain ?

Ans: Yes, it is possible.
Assuming that CLK is a Return-to-Zero clock (0->1->0 pulse), you would stitch the negedge CLK domain flip-flops before posedge CLK domain flip-flops, i.e., negedge CLK FFs are closer to scan input, and posedge CLK FFs are closer to scan output.
RCO_CLK domain can be stitched at either end of the chain. However, if CLK has a significantly larger clock tree than RCO_CLK clock tree, then it is better to put RCO_CLK domain FFs at the end of the chain. Otherwise, you may need to skew your clock timing on the ATE.
This means your chain will look like this:
ScanIn -> negedge CLK -> posedge CLK -> RCO_CLK -> ScanOut

Ques : What are the thinks to be considered in scan stitching? I know little bit about this can you explain me more in details?
Ans : Scan stitching is done can be done in one of three ways:

1) Use the synthesis tool (DFT compiler or equivalent)
2) A DFT scan tool (such as DFT Architect)
3) The place and route tool

Some of the things to think about are:

1) How many scan ports do you have available? This will determine, in part, the number of scan chains. The scan chains need to be balanced in length to be as efficient as possible, for test-time/data volume.
2) Clocks - will you use flops from different clock domains in the same chain? If so, the clock domains need to be *reasonably* skew balanced for the chains to work properly. Also, lockup-latches will be necessary where the scan path crosses the clock domains for safe operation (if stitching.

3) Are there negative edge flops in the design? If so the tool will always place the negedge flops, as a group, ahead of the posedge flops.

Ques : why first negative edge flops followed by positive edge flops in the scan chain?

Ans : Well that's so the chain will shift data in and out properly. The idea is that each bit of the data shifting into the chain should traverse the chain one flop at a time.
Given a clock that is defined with an off-state of 0, a positive edge comes before a negative edge in time, right?
Now imagine in a scan chain, a posedge FF followed by a negedge FF. During any given clock period, the data that is latched into the posedge FF will also be latched into the negedge FF as well - in the same period. This is called a feed-through, and is generally not an optimal situation for the ATPG.
However, if the insertion tool puts the negedge flops all grouped at the front of the chain, then the boundary between the negede flops and posedge flops will be a negedge FF followed by a posedge FF. Since the positive edge of the clock comes before the negative edge, the data will be properly held up at the negedge FF until the next clock period.

Ques : Apart from the conventional mux FF scan architecture, there are many others like the Level Sensitive scan and the clocked scan etc. How are these better or worse than the muxed FF technique?
Ans : LSSD is really a sub-specialty in the industry as a whole only a few companies use it, but it is effective. For scan purposes, it does not suffer from the hold time issues that mux-scan normally does, but area-wise, it's not as good.
Clock-scan uses a separate scan-clock for each flop - I've never seen it used in industry, but that's just me. The problem with it is that you must route two clock trees around the chip instead of one - a virtual show-stopper in these days of congested routing.

Ques : By full scan methodology do we mean that every single flop in the design is a part of the scan chain? And if we have multiple scan chains instead of one, as it is in present designs, can it still be called full scan methodology?
Ans : In a perfect world, full scan means every flip-flop, but in the real world, many flops can be unscanned, and the design is still considered full scan. In some cases, the ATPG tool can test through unscanned flops without a major impact to fault coverage. Designs using one or many scan chains are equally valid as full scan designs.
Ques : What is a BUS Primitive and clock_PO pattern?
Ans : A bus primitive is just a DFT model of a bus - a net that has more than one driver. It's important that you constrain it during test.
A clock PO pattern is a pattern that measures a primary output that has connectivity to a clock. So if a clock signal propagates through combinational logic to a primary output (PO), an ATPG vector can be created to measure the results of that propagation

Ques : What is the difference between test pattern and test vector ?
Ans : The terms are sometimes used interchangeably. Me, I tend to say that a test pattern is a set of test vectors.
However, the ATPG tools generally give you stats that claim a certain number of 'patterns', but personally, I would call them 'vectors'.

Ques : During the process of ATPG, I encountered a term called clocked POpattern. Could someone throw some light on what are these patterns ?

Ans : Clock PO patterns are special patterns meant to test primary output values when those primary outputs are connected, directly or indirectly, to one of the scan clocks (usually through combinational logic or just buffers).

Ques : Say in my design some flops work at low frequency, in that case,
How can we take care of flops of lower frequency when we do an at speed testing?
Ans : It depends upon whether you have independent scan clocks to control the different clock domains. If so you can generate patterns that cover all the domains, and you just need to mask the boundaries between domains.
But that's not the normal case. Many times people will use one scan clock to drive the whole circuit - and in this case, you will need to generate patterns for each clock domain separately, while masking or black boxing all the other domains.

Ques : What all needs to be taken care in scan stitching to get the good coverage?
Ans : If you are using Mentor DFTAdvisor or Synopsys DFT Compiler, cleaning up pre-stitch drc errors and most of the warnings (especially clock warnings) will generally lead to good fault coverage.
If coverage is still low after cleaning drc errors/warnings, then there may be issues inherent to the design that causes low coverage (redundant logic, complex reconvergent fanouts, black boxes, constrained nets, etc.)
Both Mentor and Synopsys tools provide ways to analyze low fault coverage in their ATPG tools. Also, some RTL analysis tools may be useful to find these kinds of problems
(see http://www.dftdigest.com/miscellaneous/rtl-design-for-test/)

Ques : How to toggle reset to get coverage ?

Ans : If the reset is asynchronous (and properly bypassed during scan), you can declare the reset pin as a clock during ATPG, and ATPG will toggle it accordingly to get faults on reset pin.
If the reset is synchronous, you can treat the reset pin as a normal data pin, and ATPG should be able to cover faults on the reset.
Be careful, however, if you run transition fault ATPG. Reset usually cannot toggle at-speed, so you may not want to declare the reset as a clock when running transition fault ATPG.
You can also try to run the patterns that toggle the reset as a clock pin at a reduced speed on the tester, if you worry about transition fault coverage on reset.

Ques : What are Pattern Faults?
Ans : A pattern fault is a fault model created by IBM Testbench folks (later acquired by Cadence and became Test Encounter). Instead of using a standard fault model like stuck-at, transition, etc., you can use truth tables to describe what the fault free and faulty behavior is for each gate type.
The advantage is that you can use pattern faults to guide ATPG to generate patterns that would not be generated with other fault models. For example, in a 2 input AND gate, to cover single stuck-at faults, you need only 3 patterns, 01, 10, and 11. If you want to force ATPG to generate a pattern for the 00 case in every AND gate, you can define a pattern fault with 00 as its sensitization.

A pattern fault is a mechanism used by Encounter Test to model static or dynamic defects that are not easily, or may be impossible to be represented by stuck-at pin faults. A pattern fault is basically a statement of
1) the requirements to excite the defect being modeled; and
2) the effect the defect being modeled has on the circuit once it has been
excited. For the current release of Encounter Test, fault excitation requirements are
specified as logic values required to be on circuit pins or nets of a (sub-) circuit. For
dynamic pattern faults, the required values include a (two) pattern sequence or required values to excite the defect.

Encounter Test automatically generates pattern faults to represent defects not possible to represent as pin stuck-at pin faults on LATCH, TSD and XOR primitives. The customer or technology library provider can add additional pattern faults for any sub-circuit used in the circuit design being processed. These user defined pattern faults will be repeated in the Encounter Test fault model for each instance of the sub-circuit.

Ques : What is the use of compressor? Why can't we use compressor for RAM?
Ans : You're getting your test methodology mixed, I think, but you can test a ROM with a compressor - also known as a MISR. That's done all the time. The idea is that the contents of the ROM are simply read out, and fed into the MISR (multiple input shift register), and the resulting signature is read out and compared with the expected signature.
It's not so common to do that with RAMs, since both the write and read data are generated algorithmically.
However if your question is more general, the use of a compressor is usually to turn a large set of data into a smaller one, which is very handy for when access to the circuit, or bandwidth, is limited.

Ques : may I know what is the major difference between transition and path delay fault models and which of them is an industrial standard?
Ans : Transition delay and path delay fault models both seek to describe defects which cause slow transitions on gate inputs or outputs. The difference is that a path delay is more specific - the whole path, from point a to point b is specified. It's a more stringent case than a transition fault. To detect a transition fault, it is only required that the ATPG generates a pattern to propagate that fault to a scanned flop. The path selected may not be the worst case path.
Most people in the industry begin with transition faults, because the ATPG can generate patterns with decent coverage more easily. Then if desired, those patterns are augmented with a much smaller set of path delays that are determined to be critical (maybe those with the least margin as determined by the timing tool).

Ques : What is memory redundancy & what is a repairable memory?

Ans : Since memory is much more dense than standard logic, it is more likely to fail in the presence of manufacturing defects. In addition, many SoCs have a significant amount of memory on-board, so the device's yield will be determined by the memory's yield.
To combat this yield limitation, designers will implement memories with extra rows and/or columns. Then if a defect causes there to be just a small number of addresses to fail, an analysis algorithm is run to determine how one or more of the extra rows or columns can be substituted for the defective ones.
So the extra rows/columns are the redundancy, and the fact that they have been implemented makes it so that you can 'repair' a defective memory. It's not so much 'repair' as 'replace'

Ques : Can we have same BIST controller for different memories?
If yes what are the requirements?
Ans : Sharing one BIST controller with several memory instances is very common in the industry. Chips with hundreds of memory instances are very common these days, so BIST sharing is almost a given with these kinds of devices.
The most important factors with BIST sharing are:
1. Sharing similar types of memories, e.g. embedded DRAM require different BIST algorithms from SRAM, thus sharing may not be very economical.
2. Physical location of the memories on the chip. If you have a large chip, you may want to consider using several BIST controllers to cover different sections of the chip.

There are many levels of sharing BIST. You can share the basic FSM or test sequencer but not the rest of BIST, or you can share the full address/control/write data generation portion but not the response comparison. You can also share the response comparison, but once you do that, the memories that share the same reposnse comparator must be tested serially.

Ques : What is a seed for Logic BIST?
What is re-seeding?
How re-seeding is done in SocBIST and DBIST?
Ans : A 'seed' for an LFSR is the starting value of the register that provides stimulus. Remember an LFSR generates a pseudo-random value (which repeats itself, given enough clocks) - but if the register is large enough, there's not enough time to go through the whole repeating sequence. So if you go part way through, and then 'reseed' the LFSR, you can get fault coverage improvement in a more efficient manner.

Ques : What is the relationship between Cost and Yield of a chip?
Is there any derivation for the same?
Ans : The relationship between cost and yield is pretty complex stuff, and depends on the technology node, the manufacturer, and many other factors.
The best one can say is that they are normally inversely proportional - the better your yield, the lower the cost, because the fixed cost of putting wafers through the fab will result in more devices going to the customer.
However, I can write a test that gives me 100% yield (no test at all), but I'll get a large percentage of my devices returned, and lose a ton of respect (and market share).

Ques : Is Logic BIST going to take over the conventional scan methodology in industrial applications in near future?
Ans : Very unlikely. There are too many advantages that scan has over Logic BIST that it cannot be usually replaced.
For example, Logic BIST lack the full controllability and observability that scan provides, which may be essential for debug, or coverage improvement.

Ques : What problems could be expected on silicon in scan mode? I am talking about the problems that could hinder running chip in scan mode?
like I have heard that sometimes testing in scan mode doesn't work at the intended frequency it is designed for?

Ans : You can avoid having problems in the silicon by simulating your scan vectors with back-annotated timing information (SDF). However, even then you may have some problems, because normally, simulation is not run with all the same timing corners that are used to close timing.
When someone says that they are not able to run scan at-speed, it usually means that the ATPG was run without considering all the false- and multi-cycle-paths that are present in the design. These paths don't generally run at-speed. This happens a lot in multi-clock domain circuits.
Also very common is that the scan chains don't shift properly - they may have hold-time violations. This is hard to overcome, but a partial set of patterns may be possible if you can figure out where the problem is. Again, if you simulate the scan shift pre-silicon, you may avoid this.

wht is the diffrence between verification and dft?
difference between defect, fault and failure?
how can we perform scan operation?
wht is serial and parallel loading?
wht is the difference between sequential and combinational atpg?
wht is atpg?
wht is drc violation?
wht is fault model?
how many fault models are there?
wht is scan stiching?
different command option....
wht is bist?
wht is bisa?
formulas for test data volume, scan chains in data compression mode..
What you mean by scan chain reordering?
How logic transition fault is different from memory transition fault.
What are RAM sequential patterns?
Diff b/w Named Capture Procedures and Clock Procedures
How much is your design count? Complexity?
What is possible cause of simulation mismatches when you simulate the generated ATPG patterns? What is right way to debug them?
How do you solve coverage issues?
what is normal mode and at-speed mode?
what is mbist?
what is dft coverage? how to get high dft coverage?
normal flow for dft ?

What is the difference between structural and functional vectors?
Ans: http://www.eetasia.com/ARTICLES/2004DEC/B/2004DEC01_ICT_ST_TA.pdf

What the major problem faced in dft with tri-state buffers and how is it resolved.
Ans: 1. The major problem is from the tester end, not all testers are able to measure Z.
2. For the IDDQ vectors, there can be no Z in the design, there is quite a lot of current when a pin is in Z state. A floating bus that is a bus with z on it will drain too much of current and hence loosing the objective of the iddq vectors.
3. Next these tri-state buffers are generally used for sharing the bus, so there has to be a dft logic so there is no contention on these bus during test.

Give three conditions where bus contention can occur.

Ans: 1. During the shift (i.e load of the scan chains)
2, During the forcing of the PIs
3. After the capture clock, the flops may capture any thing which may lead to the contentions.

Which is advantageous, launch at shift or capture launch.

Ans:
http://www.stridge.com/Stridge_articles/ac_scan_testing.htm
http://scholar.lib.vt.edu/theses/available/etd-02062003-145930/unrestricted/etd.pdf
Also a new technique is being used now is to pipe line the scan enable, and negating the scan enable in advance so that by the time of capture is to be done, the scan enable is low.

what is P1500 funda?

Ans: It is similar to BSDA but at the chip level , instead of the board level. The major difference is that in the BSDA
we are sure that the chips are OK and then do the board testing. But in the case of P1500 , we are not sure of anything, each and every core has to be tested.

How to achieve high fault coverage. How to increase it.

Ans: 1. 100% scan design
2. More number of test points
3. No X's in the design
4. Use sequential patterns
5. Completely defined netlist, i.e there should be no floating outputs, or un connected inputs.
6. There should be logic to certain that there would be no contentions on the bus
7. Avoid floating bus using bus keepers.

Latch - how is it used in dft for sync two clock domains.

Ans: Latches are used as lockup latches in the scan chains. These are inserted in the chains where ever there is a change in the clock domain. By clock domain we mean, two clocks or the same clock with phase difference.
Let us have a condition here to explain the things; we have a design with 2 clocks CLK1 and CLK2. There is a single chain in the design, which means that the scan chain have flops which can be clocked by either of the clock.
The tool by default will order the flops in the scan chain such that first we have one clock domain's flop followed by the other domain flops. Let us consider that the CLK2 flops follows CLK1 flops.
Now consider the flop which is at the boundary that is the one where the output of the CLK1's flop is going to the CLK2's scan_in. Clock skew between these successive scan-storage cells must be less than the propagation delay between the scan output of the first storage cell and the scan input of the next storage cell. Otherwise, data slippage may occur. Thus, data that latches into the last flop of CLK1 also latches into the first flop of CLK2. This situation results in an error because the CLK2's flop should latch the CLK1's "old" data rather than its "new" data.
To overcome this issue we add the lock up latch where ever there are clock domain crossing. In our example we would add a lock-up latch which has an active high enable and is being controlled by inverted of CLK1. Thus becomes transparent only when CLKA goes low and effectively adds a half clock of hold time to the output of the last flip-flop of clock domain CLK1.

Explain Fault types ?

Ans: The different fault types are
1. Stuck at fault model : The node is modeled to be stuck at some value 0 or 1 depending on what we are targeting.

2. Iddq fault model : This is similar to the stuck at fault model but here instead of measuring the voltage we measure the current . In a CMOS design at the quiescent state, ideally there is suppose to no current in the silicon, if there is current then some node has either shorted to ground or to the power.
3. Transition fault model : This is considered to stuck at fault model within a time window. The value of the node changes but not within the time ,at which it should change .For detecting such faults we have two vector for each pattern one for launching the fault and the other to capture the fault. The time between the launch and the capture is supposed to be equal to the time at which the chip would normally function. This is the reason it is all called at-speed test.
4. Path delay fault model : In this fault model ,instead of concentrating on a single gate of the netlist ,we generally are concern with a collection of gates which forms a valid path in the design. These are generally the critical paths of the design. Here again we have two vectors for each pattern. Do let me know if you know what is a valid path (don't feel offended I am just writing this because you are out of touch with all these technical jargons since long , otherwise I hope you must be knowing them).
The transition faults are also measured at the paths ends, but the major difference between the transition and the path delay is that in the path delay we give the path where as in the case of transition the tool itself selects the path for give fault.
The fault location for IDDQ, stuck-at and transition are same.
5. Bridging fault model : this is a new model which is gaining importance . In this case any two close lying net may effect the value of each other. There is generally a victim and another is a aggressor, so an aggressor forces some value on the victim . We first find the coupling capacitance of each net pair, then depending on the specs we may select some nets which have coupling capacitance more then specified value, these are selected and then these become the fault locations for the ATPG.

Does the dft vectors test the functionality of the design also?
Ans: No the dft vectors does not test the functionality of the design. It can be otherwise that is we can use the functional vector to test fault grade them and use the same for finding the fault coverage using these vectors. The dft vectors are generated keeping the design in test mode , so they won't be beneficial for the functional mode. But note this that there may always be an overlap in the patterns.

How do u break combinational loops. (*)How does introducing TIEX will eliminate combinational loop. [I told him by forcing known value we can break the loop]
Ans: By adding a tiex gate we can break the combinational loop. First what is a combinational loop. The value is not stabilized, there are oscillation. So if we place a X gate at some place in the loop, we are not propagating the deterministic value which was causing the oscillations.

Adverse Effect : Any X in the design would reduce the coverage.

The second solution would be to place a buffer with unit delay. In this case you would require sequential patterns. Please note that we are not placing any Tiex or buffer with unit delay in the netlist, it is just that we are telling ur ATPG tool to model them for the ATPG purpose. So you won't see any tiex or buf with unit delay gates in the netlist.

What is scannability checking?
Ans: I think this relates to the scan chain integrity. The first pattern that is pattern0 in most of the ATPG tool is called the chain test pattern. This pattern is used to check the integrity of the scan chains, to see if the scan chains are shifting and loading properly; if the scan chains itself have a fault there is no use checking the full chip using this chain.

Give three Clock drc rules and how to fix them.
Ans: 1. Clock not controllable from the top. (Use mux to controll the same)

2. When all the clocks are in off state , the latches should be transparent (add logic to make them transparent)
3. A clock must not capture data into a level sensitive (LS) port (latch or RAM) if that data may be affected by new captured data. (for FASTSCAN : clock_off simulation on and for TetraMAX : set atpg -resim_basic_scan_pattern)

What does test procedure files have?
Ans: The test procedure file contains all the scan information of your test ready netlist.
1. The number of the scan chains
2. The number of scan cells in each scan chain.
3. The shift clocks.
4. The capture clocks
5. Any test setup procedure required before starting the test pattern generation
6. The timing of the different clocks.
7. The time for forcing the Primary input , bidi inputs , scan inputs etc
8. The time to measure the primary outputs, scan outputs , etc ..
9. The pins which have to be held at some state in the different procedure as load_unload, shift etc ..

What problems u faced while inserting test points.
Ans: The problems u faces while inserting test points ,
I don't think there is any problem, except
1. Selecting the best candidate location for the test points.
2. Area over head

Give three Clock drc rules and how to fix them.
Ans: 1. Clock not controllable from the top. (Use mux to controll the same)
2. When all the clocks are in off state , the latches should be transparent (add logic to make them transparent)
3. A clock must not capture data into a level sensitive (LS) port (latch or RAM) if that data may be affected by new captured data. (for FASTSCAN : clock_off simulation on and for TetraMAX : set atpg -resim_basic_scan_pattern)

What does test procedure files have?
Ans: The test procedure file contains all the scan information of your test ready netlist.
1. The number of the scan chains
2. The number of scan cells in each scan chain.
3. The shift clocks.
4. The capture clocks
5. Any test setup procedure required before starting the test pattern generation
6. The timing of the different clocks.
7. The time for forcing the Primary input , bidi inputs , scan inputs etc
8. The time to measure the primary outputs, scan outputs , etc ..
9. The pins which have to be held at some state in the different procedure as load_unload, shift etc ..

What problems u faced while inserting test points.
Ans: The problems u faces while inserting test points ,
I don't think there is any problem, except
1. Selecting the best candidate location for the test points.
2. Area over head

If enable pin of tri-state is 0, the output is Z. how does tool treat this Z as in DFT. How is Z handled.
Ans: It depends on the tester. We can customize the tool to generate the patterns

How operating voltage can be used to satisfy timing?
Ans: Higher operating voltage can cause faster slew, which sometimes can fix timing.

What is the difference between local-skew, global-skew and useful-skew?
Ans: Local-skew - skew between a reg2reg path
Global-skew - skew between all registers in a clock domain
useful-skew - to advance the clock in a reg2reg path in order to meet setup time.

What is meant by virtual clock definition and why do i need it?
Ans: Virtual clocks are defined on I/O because they real clock associated with them, however, they are considered start points and endpoints in STA, so they must have a clock. The latency is estimated to be the network latency of a clock tree, pre-cts.

Is it possible to reduce clock skew to zero
Ans: Not in a practical sense.

What are problems associated with skew and how to minimize it?
Ans: Skew is the difference in insertion delay to registers. If the skew is too large, then you fail timing.

How to solve setup and Hold violations in the design?
Ans: For hold only way to insert buffers (Correct)
For setup only way to upsize the cell and tweak the skew?
Or reduce the levels of logic from reg2reg, use fasted cells, swap pins, slow down the clock, etc...

What are the violations which prevent scan insertion?

Ans: Scan design rules require that registers have the functionality, in test mode, to be a cell within a large shift register. This enables data to get into and out of the chip. The following violations prevent a register from being scannable:

• The flip-flop clock signal is uncontrollable.

• The latch is enabled at the beginning of the clock cycle.

• The asynchronous controls of registers are uncontrollable or are held active.

Uncontrollable Clocks

This violation can be caused by undefined or unconditioned clocks. DFT Compiler considers a clock to be controlled only if both of these conditions are true:

• It is forced to a known state at time = 0 in the clock period (which is the same as the "clock off state" in TetraMAX).

•It changes state as a result of the test clock toggling.

Latches Enabled at Beginning of Clock Cycle

This violation applies only when the scan style is set to level-sensitive scan design (LSSD). For a latch to be scannable, the latch must be forced to hold its value at the beginning of the cycle, when the clock is inactive. This violation can be caused by undefined or unconditioned clocks.

This violation indicates that there are registers that cannot be controlled by ATPG. If the violation is not corrected, these registers will be unscannable and fault coverage will be reduced.

Asynchronous Control Pins in Active State

Asynchronous pins of a register must be capable of being disabled by an input of the design.If they cannot be disabled, this is reported as a violation. This violation can be caused by asynchronous control signals (such as the preset or clear pin of the flip-flop or latch) that are not properly conditioned before you run DFT Compiler.

What are the violations which prevent data capture?

Ans: The violations are described in the following sections:

• Clock Used As Data

• Black Box Feeds Into Clock or Asynchronous Control

• Source Register Launch Before Destination Register Capture

• Registered Clock-Gating Circuitry

• Three-State Contention

• Clock Feeding Multiple Register Inputs

What are the violations which reduces fault coverage?

Ans: Violations that can reduce fault coverage are

• Combinational Feedback Loops

• Clocks That Interact With Register Input

• Multiple Clocks That Feed Into Latches and Flip-Flops

• Black Boxes

Combinational Feedback Loops

An active (or *sensitizable*) feedback loop reduces the fault coverage that ATPG can achieve by increasing the difficulty of controlling values on paths containing parts of the loop.

Clocks That Interact With Register Input

A clock that affects the data input of a register reduces the fault coverage attainable by ATPG, because ATPG pulses only one clock at a time, keeping all other clocks in their off states. Attempting to fix this purely in the ATPG setup can result in timing hazards.

Black Boxes

Logic that drives or is driven by black boxes cannot be tested because it is unobservable or uncontrollable. This violation can drastically reduce fault coverage, because the logic that surrounds the black box is unobservable or uncontrollable

Which scan styles are supported in your technology library?

Ans: To make it possible to implement internal scan structures in the scan style you select, appropriate scan cells must be present in the technology libraries specified in the target_library variable.

Use of sequential cells that do not have a scan equivalent always results in a loss of fault coverage in full-scan designs.

What is your design style?

Ans: If your design is predominantly edge-triggered, use themultiplexed flip-flop, clocked scan, or clocked LSSD scan style.

If your design has a mix of latches and flip-flops, use the clocked scan or LSSD scan style.

If your design is predominately level-sensitive, use the LSSD scan style.

How complete are the models in your technology library?

Ans: The quality and accuracy of the scan and nonscan sequential cell models in the Synopsys technology library affect the behavior of DFT Compiler. Incorrect or incomplete library models can cause incorrect results during test design rule checking.

DFT Compiler requires a complete functional model of a scan cell to perform test design rule checking. The Library Compiler UNIGEN model supports complete functional modeling of all supported scan cells. However, the usual sequential modeling syntax of Library Compiler supports only complete functional modeling for multiplexed flip-flop scan cells.

When the technology library does not provide a functional model for a scan cell, the cell is a black box for DFT Compiler.

What is Design For Test (DFT)?

Ans: Designing for the testability, in which the test merges, with the design in the earlier process of the design, following what is called a design-for-test (DFT).

What is Testability?

Ans: Testability is a design attribute that measures how easy it is to create a program to comprehensively test a manufactured design's quality.

What is Functional Testing?

Ans: Functional testing verifies that your circuit performs as it was intended to perform. Testing of all possible input combinations grows exponentially as the number of input increases. Thus in real time it may not be possible to test all the input combinations.

What is Manufacturing Testing?

Ans: Manufacturing testing verifies that your chip does not have manufacturing defects by focusing on circuit structure rather than functional behavior. Manufacturing defects include problems such as

- Power or ground shorts
- Open interconnect on the die due to dust particles
- Short circuited source or drain on the transistor due to metal spike-through.

Manufacturing defects might remain undetected by functional testing yet cause undesirable behavior during circuit operation.

What Are Fault Models?

Ans: When a manufacturing defect occurs, the physical defect has a logical effect on the circuit behavior and chip may not work as intended. An open connection can appear to float either high or low. A signal shorted to power appears to be permanently high. A signal shorted to ground appears to be permanently low.

What is Stuck-at Fault Models?

Ans: The stuck-at-0 fault means a signal that is permanently low regardless of the other signals that normally control the node, as shown in Figure 1. The stuck-at-1 fault represents a signal that is permanently high regardless of the other signals that normally control the node. For example, assume that you have a two-input AND gate that has stuck-at-0 fault on the output pin. As shown in Figure 1-1, regardless of the logic level of the two inputs, the output is always 0.

Figure 1: AND Gate with stuck at 0 Fault at output pin

What is Controllability?

Ans: How easy it is to control the inputs of any cell inside the design from the input pads.

What is Observability?

Ans: How easy it is to observe the outputs of any cell inside the design from the output pads.

What is Scan design?

Ans: Scan design is the most popular DFT technique and has high fault coverage results. The idea is to control and observe the values in all the design's storage elements so you can make the sequential circuit's test generation and fault simulation tasks as simple as those of a combinational circuit.

What is Full Scan?

Ans: Full scan is a scan design methodology that replaces all memory elements in the design with their scan-able equivalents and then stitches them into scan chain. Full scan can assure the high quality of the

product. The idea is to control and observe the values in all the design's storage elements so you can make the sequential circuit's test generation and fault simulation tasks as simple as those of a combinational circuit.

Full Scan Benefits
The following are benefits of employing a full scan strategy:

- Highly automated process. Using scan insertion tools, the process for inserting full scan circuitry into a design is highly automated, thus requiring very little manual effort.

- Highly effective, predictable method. Full scan design is a highly effective, well-understood, and well-accepted method for generating high-test coverage for your design.

- Assured quality. Full scan assures quality because it tests most of the silicon.

What is Partial Scan?

Ans: The full scan design makes all storage elements scannable; it will cost in terms of silicon area and the timing. Because of these in some design we may not have the luxury of having a full scan design. There comes the idea of Partial scan, which is also a scan design methodology where only a fraction of the storage elements in the design are replaced by their scannable equivalents and stitched into scan chains. By carefully choosing which storage elements to be replaced we can increase the testability of the design with minimal impact on the design's area or timing. If the design cannot offer to accommodate the extra delay added in critical path (due to added mux in the storage element), we can exclude those critical flip-flops from the scan chain using partial scan.

Partial Scan Benefits
- Reduced impact on area.

- Reduced impact on timing.

- More flexibility between overhead and fault coverage.

What are the different dft strategies?
Ans: At the highest level, there are two main approaches to DFT: ad hoc and structured. The following subsections discuss these DFT strategies.

Ad Hoc DFT
Ad hoc DFT implies using good design practices to enhance a design's testability, without making major changes to the design style. Some ad hoc techniques include:

- Minimizing redundant logic

- Minimizing asynchronous logic

- Isolating clocks from the logic

- Adding internal control and observation points

Using these practices throughout the design process improves the overall testability of the design.

Structured DFT
Structured DFT provides a more systematic and automatic approach to enhancing design testability. Structured DFT's goal is to increase the controllability and observability of a circuit. Various methods exist for accomplishing this. The most common methods are

- The scan design technique, which modifies the internal sequential circuitry of the design.
- The Built-in Self-Test (BIST) method, which inserts a device's testing function within the device itself.

The boundary scan, which increases board testability by adding circuitry to a chip.

What is the difference between the coverage reports from dft dsm and from ttmax?

Ans: dft dsm coverage report gives the approximate test coverage while ttmax gives the actual test coverage.

What are ATPG modes?

Ans: TetraMAX offers three different ATPG modes: Basic-Scan, Fast-Sequential, and

Full-Sequential.

Basic-Scan ATPG: In Basic-Scan mode, TetraMAX operates as a full-scan, combinational-only ATPG tool. To get high test coverage, the sequential elements need to be scan elements. Combinational ROMs can be used to gain coverage of circuitry in their shadows in this mode.

Fast-Sequential ATPG : Fast-Sequential ATPG provides limited support for partial-scan designs. In this mode,multiple capture procedures are allowed between scan load and scan unload, allowing data to be propagated through nonscan sequential elements in the design such as functional latches, non-scan flops, and RAMs and ROMs. However, all clock and reset signals to these nonscan elements must still be directly controllable at the primary inputs of the device. You enable the Fast-Sequential mode and specify its effort level by using the **-capture_cycles** option of the set atpg command.

Full-Sequential ATPG : Full-Sequential ATPG, like Fast-Sequential ATPG, supports multiple capture cycles between scan load and unload, thus increasing test coverage in partial-scan designs. Clock and reset signals to the nonscan elements do not need to be controllable at the primary inputs; and there is no specific limit on the number of capture cycles used between scan load and unload. You enable the Full-Sequential mode by using the **-full_seq_atpg** option of the set atpg command. The Full-Sequential mode supports an optional feature called Sequential Capture. Defining a sequential capture procedure in the STIL file lets you compose a customized capture clock sequence applied to the device during Full-Sequential ATPG. For example, you can define the clocking sequence for a two-phase latch design, where CLKP1 is followed by CLKP2. This feature is enabled by the **–clock -seq_capture**option of the set drc command. Otherwise, the tool will create its own sequence of clocks and other signals in order to target the as-yet-undetected faults in the design.

What is Fault Coverage?

Ans: The extents to which that design can be tested for the presence of manufacturing defects, as represented by the single stuck-at fault model. Using this definition, the metric used to measure testability is fault coverage, which is defined as

Number of detected faults

(Total number of faults) - (number of undetectable faults)

What is test coverage?

Ans: Test coverage gives the most meaningful measure of test pattern quality and is the default coverage reported in the fault summary report. Test coverage is defined as the percentage of detected faults out of detectable faults.

What is ATPG effectiveness?

Ans: ATPG effectiveness is defined as the percentage of ATPG-resolvable faults out of the total faults.

Important Definitions::::::::::::::

At-speed clock: A pair of clock edges applied at the same effective cycle time as the full operating frequency of the device.

Capture clock/capture clock edge: The clock used to capture the final value resulting from the second vector at the tail of the path.

Capture vector: The circuit state for the second of the two delay test vectors.

Critical path: A path with little or no timing margin.

Delay path: A circuit path from a launch node to a capture node through which logic transition is propagated. A delay path typically starts at either a primary input or a flip-flop output, and ends at either a primary output or a flip-flop input.

Detection, robust (of a path delay fault): A path delay fault detected by a pattern providing a robust test for the fault.

Detection, non-robust (of a path delay fault): A path delay fault detected by a pattern providing a non-robust test for the fault.

False path: A delay path that does not affect the functionality of the circuit, either because it is impossible to propagate a transition down the path (combinationally false path) or because the design of the circuit does not make use of transitions down the path (functionally false path).

Launch clock/launch clock edge: The launch clock is the first clock pulse; the launch clock edge creates the state transition from the first vector to the second vector.

Launch vector: The launch vector sets up the initial circuit state of the delay test.

Off-path input: An input to a combinational gate that must be sensitized to allow a transition to flow along the circuit delay path.

On-path input: An input to a combinational gate along the circuit delay path through which a logic transition will flow. On-path inputs would typically be listed as nodes in the Path Delay definition file.

Path: A series of combinational gates, where the output of one gate feeds the input of the next stage.

Path delay fault: A circuit path that fails to transition in the required time period between the launch and capture clocks.

Scan clock: The clock applied to shift scan chains. Typically, this clock is applied at a frequency slower than the functional speed.

Test, non-robust: A pair of at-speed vectors that test a path delay fault; fault detection is not guaranteed, because it depends on other delays in the circuit.

Test, robust: A pair of at-speed vectors that test a path delay fault independent of other delays or delay faults in the circuit.

What is Adaptive Scan?

Adaptive scan is similar to scan at the chip boundary, but it inserts a combinational decompression structure between the chip scan pins and the numerous short internal scan chains. Compressed scan input values are loaded in the adaptive scan module that distributes them internally through an association between an external scan pin

and an internal scan chain. To maximize test coverage, the association adapts to the needs of ATPG to supply the required values in the scan cells — thus the name "adaptive scan."

Adaptive scan optimizes the traditional scan chains into smaller segments enabling savings in test time, while the adaptive scan module and the output compactor significantly reduce the amount of test data needed to comprehensively test the chip lowering ATE memory requirements and giving room to add DSM Test patterns.

The following key benefits and features are associated with adaptive scan technology:

• Provides 10-50X test time and test volume reduction

• Same high test coverage and ease of use as traditional scan

• No impact on design timing or physical design implementation
1-pass test compression synthesis flow

• Hierarchical Adaptive Scan Synthesis

• Boundary scan synthesis and compliance checking to the 1149.1 standard

What are the design guidelines for getting good error coverage and error free dft?

Ans:

❑·Use fully synchronous design methodology using 'pos-edge' only if possible.

Generated or gated clocks should be properly planned, documented and collected in one module at the top-level.

❑·For gated clocks or derived clocks: a test mode should be implemented that will drive all the Flip-Flop (FF) clocks from a single test clock during this test mode. Also, clock skew for this test clock should be properly balanced so there are no hold violations on any of the registers both during scan shift and normal mode.

❑·Provide proper synchronization for signals crossing clock domains, different edges of the same clock or asynchronous inputs. Such un-testable synchronization logic should be isolated in a separate module. Create patterns for this asynchronous logic separately.

❑·A lock-up latch should be inserted in a signal crossing a clock-domain or clock edge, if all the FFs are to be part of same scan chain.

❑·Don't use clocks in combinational logic or as data/set/reset input to a FF.

❑·All the asynchronous set/resets should be directly controllable from the primary input of the chip. If there are any internally derived set/resets, it should be possible to disable them using one or more primary inputs.

❑·Don't use asynchronous set/resets in combinational logic or as data input to a FF.

❑·Avoid using registers with both Set/Reset functionality.

❑·Avoid latches in your design. If there are any latches in the design, care should be

taken to make them transparent during the scan test.

❑·Avoid internal tri-state buses (instead consider a MUX architecture). If not, then

implement bus control logic to ensure one and only one driver (no bus conflicts or no

floating buses) is active on the bus during scan test.

❑·For external tri-states, bring out 3 signals for each tri-state pad: input, output and

enable.

❑·No combinational feedback loops (especially when integrating the sub-blocks).

❑·Avoid large logic cones. It increases the ATPG runtimes exponentially.

❑·Avoid redundant logic (re-convergent fan-outs are good indication of redundant logic). It creates undetectable faults in the net-list. Add transparent or observability FFs in the design to increase fault coverage.

❑·Use ‘logic_high’ and ‘logic_low’ port signals for all the sub-blocks instead of VDD/VSS or ‘tie_1/tie_0’ cells.

- Put ‘don’t_touch’ on these ports so that they will not be removed and can be used to connect the ‘tie_off’ nets generated during the synthesis. These ‘logic_high’ and ‘logic_low’ ports can be connected to a ‘tie_macro’ (these macros will contain scan-able FFs) at the top-level that make all these nets testable.

❑·Default values on the buses should use alternate ‘logic_high’ and ‘logic_low’

connections to increase fault coverage.

❑·Balance the scan chains to be of approximately equal length and limit this length

according to tester memory limitation. Here is one generic formula for calculating the

of scan patterns based on tester memory and maximum scan chain length.

#scan_patterns < (tester_mem_per_chain-max_chain_length)/max_chain_length)

❑·Make sure the ‘scan_enable’ signal is properly buffered so that scan tests can be run at higher frequencies.

❑·Use only cells (including memories) which have the appropriate DFT/ATPG models

available.

❑·Provide for debug capability of a memory failure as part of the memory test

methodology.

❑·Disable the memory during scan shift and bypass mode for allowing fault coverage around the memory I/O.

❑·Provide a power-down mode for the memory, analog and other power consuming blocks during the IDDQ test.

❑·Verify there are no hold time violations on any registers in both scan shift and normal mode after test insertion.

❑·Plan chip level scan issues before starting the block level design.

❑·Proper review of the memory, analog or other custom blocks test methodology. For analog IP blocks or hardened IP blocks (with no internal scan chains) provide a bypass option so the surrounding logic can be tested.

❑·Proper review of all the different test modes of the chip and detailed documentation of the primary pin assignments during these test modes

❑·Proper review of any additional test mode control logic.

Important formulas::::::::::

Test Application Time Reduction = (Length of longest scan chain in scan mode) / (Length of longest scan chain in ScanCompression_mode)

Scan Test Data Volume = 3 * Length of longest scan chain * number of scan chains

Scan Compression Test Data Volume = Length of longest scan chain * (Number of scan inputs + 2 * Number of scan outputs)

Test Data Volume Reduction = (Scan Test Data Volume) / (Scan Compression Test Data Volume)

Why we loose coverage when we constrain pins?

Ans: In general, whenever you constrain any pins of your device, you take away the ability of the ATPG to toggle that pin and check it (and its effects) in both states. Sometimes when you constrain a pin, it will have negligble effect. Sometimes it will have far ranging effects on fault coverage.

Why at-speed(LOC) pattern count is more than stuck-at pattern count?

Ans: For a full scan design, stuck-at ATPG looks at generating patterns in the combinational logic blocks in between scan flip-flops. This is because every scan flip-flop can be treated as a primary input and a primary output for stuck-at ATPG purpose.
However, when testing for at-speed failures, 2 patterns are needed to launch a transition and capture the effect into a scan flip-flop. Therefore, ATPG needs to trace back beyond one level of scan flip-flops to figure out how to get the appropriate "2nd pattern" in a 2-pattern test, which means it has less degree of freedom in how to assign the scan flip-flops to detect a fault, which leads to higher pattern count

Say in my design some flops work at low frequency, in that case,
How can we take care of flops of lower frequency when we do an at speed testing?

Ans: It depends upon whether you have independent scan clocks to control the different clock domains. If so you can generate patterns that cover all the domains, and you just need to mask the boundaries between domains.

But that's not the normal case. Many times people will use one scan clock to drive the whole circuit - and in this case, you will need to generate patterns for each clock domain separately, while masking or black boxing all the other domains.

What all needs to be taken care in scan stitching to get the good coverage?

Ans: If you are using Mentor DFTAdvisor or Synopsys DFT Compiler, cleaning up pre-stitch drc errors and most of the warnings (especially clock warnings) will generally lead to good fault coverage.

If coverage is still low after cleaning drc errors/warnings, then there may be issues inherent to the design that causes low coverage (redundant logic, complex reconvergent fanouts, black boxes, constrained nets, etc.)
Both Mentor and Synopsys tools provide ways to analyze low fault coverage in their ATPG tools. Also, some RTL analysis tools may be useful to find these kinds of problems
(see http://www.dftdigest.com/miscellaneous/rtl-design-for-test/)

How to toggle reset to get coverage?

Ans: If the reset is asynchronous (and properly bypassed during scan), you can declare the reset pin as a clock during ATPG, and ATPG will toggle it accordingly to get faults on reset pin.
If the reset is synchronous, you can treat the reset pin as a normal data pin, and ATPG should be able to cover faults on the reset.

Be careful, however, if you run transition fault ATPG. Reset usually cannot toggle at-speed, so you may not want to declare the reset as a clock when running transition fault ATPG.
You can also try to run the patterns that toggle the reset as a clock pin at a reduced speed on the tester, if you worry about transition fault coverage on reset.

During the process of ATPG, I encountered a term called clocked PO pattern. Could someone throw some light on what are these patterns?

Ans: Clock PO patterns are special patterns meant to test primary output values when those primary outputs are connected, directly or indirectly, to one of the scan clocks (usually through combinational logic or just buffers).

What is a BUS Primitive and clock_PO pattern?

Ans: A bus primitive is just a DFT model of a bus - a net that has more than one driver. It's important that you constrain it during test.
A clock PO pattern is a pattern that measures a primary output that has connectivity to a clock. So if a clock signal propagates through combinational logic to a primary output (PO), an ATPG vector can be created to measure the results of that propagation.

What are all the advantages and disadvantages of LBIST over Scan?

Ans: Logic BIST lack the full controllability and observability that scan provides, which may be essential for debug, or coverage improvement.

I understood that there should be seperate clock control circuitry to select two at-speed clock pulses from free-running PLLs for transition fault testing. What about stuck-at testing, in this case we need only one pulse from PLLs to capture response. Will there be any signal to control this behavior of clock control circuitry?

Ans: Well, it's not ***strictly*** necessary to have on-chip control. You can source the clock from an I/O, just as long as your ATE can handle the speed you need, and the device can distribute the clock well enough. The advantage of having on-chip control is that you can use a slow tester.

As far as the stuck-at clock goes, remember, you're using the slow external clock to shift data through the scan chains. For stuck-at, you can bypass the at-speed clock control and use only the slow external clock. Yes, you'd need a control signal. You can also, if you want to increase your flexibility, and get better fault coverage, design your clock control to be able to output 2, 3, or even 4 pulses, to catch faults that only get caught with sequential patterns. I've not done it myself, but I've read about such designs.

Could I know what is the major difference between transition and path delay fault models and which of them is an industrial standard?

Ans: Transition delay and path delay fault models both seek to describe defects which cause slow transitions on gate inputs or outputs. The difference is that a path delay is more specific - the whole path, from point a to point b is specified. It's a more stringent case than a transition fault. To detect a transition fault, it is only required that the ATPG generates a pattern to propagate that fault to a scanned flop. The path selected may not be the worst case path.

Most people in the industry begin with transition faults, because the ATPG can generate patterns with decent coverage more easily. Then if desired, those patterns are augmented with a much smaller set of

path delays that are determined to be critical (maybe those with the least margin as determined by the timing tool).

What all things in DFT can be of a concern whenever there is a technology node change?

Ans: There are many changes in DFT with the nodes.
In fact DFT is expanding as the technology is shrinking.
Some of the things to take care of are :

1. Power : This is the biggest concern now-a-days. As the requirements to decrease power increases, the DFT gets more complex. Its part of DFT to make sure that DFT coverage has no impact with all the added logic due to new logic added to save power, adding different power domains and retaning values of logic during power down. Apart from that power consumption during ATPG could also lead to generation of Patterns for each power domain/clock domain separately.

2. Faults : As the technology shrinks new types of faults starts playing a role. Some of the faults which industry is working on now-a-days is Bridging faults, fast-to-rise/fast-to-fall faults, transient faults, many yet unknown memory faults are to name a few.

3. Programmability : The biggest ask for DFT today is programmability, be it IOs, logic or memory tests.

4. Meeting standards : As chip gets integrated with other chips on board, it becomes inevitable to meet various standards floating around.

5. Reliability Checks : Many old methodologies are getting redundant with time, e.g. with increased leakage IDDQ got redundant. Many new methodologies are developed now and then to check for reliability of circuits.

6. Starting early : May be moving to RTL for even pattern generation may help. With so much to do in DFT, its never too early to start. Many companies are working to come up with many new solutions.

Can I get some information regarding the IO BIST?

Ans: IO BIST is the terminology normally used for High Speed I/Os with a loop back circuitry on TX/RX.
Since capturing data from such high speed I/Os is not possible for low cost tester, a BIST logic which transfers some data through TX and then reads back the data looped back at I/O level through RX, is added. Based on this received data it activates the pass/fail flag.

Memory cell is not observable or controllable during a normal scan mode. how do we test the logic around memory , i.e how is this coverage taken care of ? are the Ram sequential patterns used for this ? what is done in these patterns ?

Ans: RAM sequential patterns are normally used to test very small memories for which you do not want to put memory BIST logic. It can also be used to test logic around memories, but normally a wrapper around memory, such that Memory output is bypassed with Memory Input signals, is used to test logic around memories.

During RAM sequential patterns, you need to make sure that memories can be enabled, read enable can be set and Valid data can be put on data and address bits using SCAN Flops. Moreover it should also be made sure that during shift phase data written in memory should not get corrupted, i.e. Write Enable should not toggle during scan shifting.
Generating Ram seq patterns can be tricky at times - as the user needs to define the read and write cycles properly....

As an alternative to Ram sequential patterns you can try generating clock seq patterns to test the shadow logic around the memories.
You can do the following things

1. Increase the sequential depth for the design

2. Also (for fastscan) you need to define the read and write clocks for the memories
Commands:
add read control
add write control
The clock_name mentioned above is the clock that is used to clock the memories during scan mode. for more details you can refer to the tool document.

Another important thing to take care before generating clock seq patterns ..... make sure you have the ATPG (functional) models for the memories.

How to generate patterns if we have both compressed chains (of size say 100 flops each) and compressed chains (of size 1K flops each) in the same design
1. Do we generate patterns for each of these chains together?
In this case the compression would be lost
2. Do we generate patterns compressed chains only and then generate for uncompressed chains?
While generating patterns for compressed chains should the uncompressed chains be defined or not?

Ans: preferred way of generating patterns would be:
For final Tester Run (To have minimum number of patterns, hence minimum test time):
a. Generate patterns with compressed chains.
b. Save above fault list
c. Generate top-up patterns with uncompressed chains just to cover faults not covered by above run (load fault list from above run and target au faults only)

For debugging purposes you will need to generate whole set of patterns in both modes. In case anything fails on tester it is always easier to debug in uncompressed mode.

How does the ATPG compression tool take care of head/tail flops inserted outside the compression engine(e.g flops residing outside decompressor and compactor in Test Compress)?

Ans: In case of Test compress specially, the following commands take care of the registers outside the chains:

set edt pins input_channel -pipeline_stages
set edt pins output_channel -pipeline_stages

I am assuming the flops inserted outside the compression engine are in form of shift register.

Why do we need more patterns in At-speed testing compared to DC-scan. is it because of achieving the same coverage requires more patterns ?

Ans: Regarding your question, the difference in pattern count between Stuck-at fault testing and At-Speed Fault testing exists due to the difference in type of faults.
For Stuck-at fault testing you need to set a node to value opposite to the fault you are testing and propagate it to a flop, however for transition fault testing you first need to set the node to particular value and then toggle it to opposite value and capture it at a flop. Hence you can see that in stuck-at, only setting a node at particular value and making sure other nodes are set to such a value that fault can propagate to a flop is enough. While for at-speed testing you need to make sure that data reaches next flop within the desired clock period, so you need to toggle it to opposite value during capture to check for timing.
So you can see that generating patterns for At-Speed testing is little bit more tricky than at-speedfault testing and therefore results in more patterns.
To reduce your total pattern count (transition + stuck-at) you can fault grade your patterns.
1. use the patterns generated to transition faults and simulate them to get the the coverage for stuck-at fault model
2. save the fault list after simulating transition faults for stuck-at fault
3. load the fault list and generate the patterns for the remaining stuck-at faults

The final result would lead to a reduction in total pattern count.

Why do we need more patterns in At-speed testing compared to DC-scan. is it because of achieving the same coverage requires more patterns?
Or
Why at-speed(LOC) pattern count is more than stuck-at pattern count?
Ans: For a full scan design, stuck-at ATPG looks at generating patterns in the combinational logic blocks in between scan flip-flops. This is because every scan flip-flop can be treated as a primary input and a primary output for stuck-at ATPG purpose.
However, when testing for at-speed failures, 2 patterns are needed to launch a transition and capture the effect into a scan flip-flop. Therefore, ATPG needs to trace back beyond one level of scan flip-flops to figure out how to get the appropriate "2nd pattern" in a 2-pattern test, which means it has less degree of freedom in how to assign the scan flip-flops to detect a fault, which leads to higher pattern count.

LOC does indeed result in many more patterns that stuck-at patterns. However at-speed patterns can also be applied using a launch-on-shift (LOS) timing protocol. This does not suffer from the sequential ATPG drawbacks as the LOC protocol and therefore results in little to no pattern increase over stuck-at. LOS can be more tricky to implement due to the need for at-speed scan-enable signals (these are necessary as LOS requires going from shift-mode to functional mode within one system clock cycle).

Regarding your question, the difference in pattern count between Stuck-at fault testing and At-Speed Fault testing exists due to the difference in type of faults.
For Stuck-at fault testing you need to set a node to value opposite to the fault you are testing and propagate it to a flop, however for transition fault testing you first need to set the node to particular value and then toggle it to opposite value and capture it at a flop. Hence you can see that in stuck-at, only setting a node at particular value and making sure other nodes are set to such a value that fault can propagate to a flop is enough. While for at-speed testing you need to make sure that data reaches next flop within the desired clock period, so you need to toggle it to opposite value during capture to check for timing.
So you can see that generating patterns for At-Speed testing is little bit more tricky than at-speedfault testing and therefore results in more patterns

Why scan chain is deleted before placement and reconnected after routing ?
Ans: It's called scan chain reordering. It is an optimisation to ensure the scan chain is connected in the most efficient way - based upon the placement of the flip-flops.

At initial stage , we dont have the placement information. So we just stich the flops register by register. But after placement it might be possible (and generally in pratice happens) that the two flops stiched at initial stage of a different block sits far or too far from each other when the placement is done. So if we keep the scan chanin stiched same as a initial stage, very long and wearied and inefficient scan path results(i.e. one flop at this end of chip and next flop in chain is on ohter corener of chip). So before we do placement, remove the scan chain stiching, and once you done with the placement we re-stich the whole scan chain. (called scan chain re-ordering as well). So tool will systematically stich the scan flops.

Please let me know the Launch and capture path in Launch On Capture(LOC) and Launch On Shift(LOS) in At-Speed mode?
Ans: For at-speed fault there is a requirement that we launch the fault. By launching it means that we give a transition at the fault site. So for slow-to-rise fault, the launch will be to change the value from 0 to 1. Once the fault has been launched , the changed value is captured At-speed in a scan flop.

Now there are two ways to launch a fault. Either when the scan chain is still getting loaded or once the scan chain load is over.

If the scan chain is still getting loaded, then last shift is used to launch the fault and then in the capture mode, the transition on the fault location is captured. The thing to note here the last but 1 shift should be able to put a value of 0 at the fault and with the last shift the value at the fault site should change to 1.

In the launch on capture at the end of the shift , the fault site will have a value of 0, and then there will be two clock pulses, one to change the fault site location to 1 and the other to capture the transition....

what is Compression ratio and how to fix the compression ratio for any design?
Ans: The compression ratio in DFT is basically used for TAT and TDV

TAT : Tester application Time
TDV : Test data volume. (Size of the patterns)
It is the reduction in these two number when compared to a design which has just the scan chains and no compression techniques.

The scan compression technique available with most of the comercial tool today is to have multiple scan chains inside the core with a limited number of top level scan ports used for loading and unloading these chains. So, you require hardware in your design to support. This , there will be a decompressor to feed many internal chains from limited number of top level scan input ports and a compressor to unload the value from many internal scan chains to limited number of top scan outputs.

The TAT and TDV is achieved by lesser number of cycles needed to load the internal chains. In most curde form the compression ratio for such technique is

External chain = #internal chains * compression ratio * β

Where β = to account for some pattern inflation. You need to know this β and then can control the compression ratio.

Can we have the shift frequency same as functional frequency?
Ans: The maximum shift frequency will depend on what is the maximum peak power, the chip can support.

When you are shifting the data on the scan chain, all the flops in the chains are operational the same time. The peak power requirment goes high, also too much of heat is generated , if you dont account for it the chip will burn up.

In the functional mode , you will never get a chance where all the flops will be operational at the same time...

What are the factors which decide the number of scan chains?

Ans: Factors such as availability of chip I/O pins, available tester channels, and on-chip routing congestion caused by chaining storage elements in test mode limit the number of scan
chains.

LOS is combinational ATPG algorithm and LOC is Sequential ATPG algorithm. Why?

Ans: LOC is sequential, since it is essentially a double capture, and the ATPG tool needs to be able to store the state of the circuit after the last shift and first clock pulse of the capture in order to know what is expected after the second capture clock.

LOS is essentially the same as a simple stuck-at, since there is only one clock pulse during the capture. No second state needs to be stored by the ATPG to determine how the circuit will react after the second clock pulse.

Ques:----- Can any body tell me

1) What causes HOLD VIOLATIONS in DESIGN.

2) How it effects DESIGN.

3) What changes need to be done to make DESIGN work.

Ans:----
Hi Friend,

1) What causes **HOLD VIOLATIONS** in DESIGN.

Simply, data should be **hold** for some time (**hold** time) after the edge of the clock. So, if the data changes with the **hold** time might cause violation. In general, **hold** time will be fixed during backend work (during PNR) while building clock tree. If u r a frontend designer, concentrate on fixing **setup** time **violations** rather than **hold violations**.

2) How it effects DESIGN.
If a chip is done with some **setup violations** it can work by reducing the frequency.
If achip is done with **hold violations**, JUST DUMP the chip. This is how it effects at the end of the day. **Hold** vilations needs to be fixed.

3) What changes need to be done to make DESIGN work.
PNR tools will route and place the cells in such a way that no timing **violations** will occur. If still u face **hold violations**, u can manully work on it to fix. Manually place the cells to avoid **hold violations**, or in the worst case, u can keep some buffers in the datapath to avoid **hold violations** (but be sure **setup** timing is not effected.)

you said "If achip is done with hold violations, JUST DUMP the chip. "
>why can't reducing the frequency to settle the hold violation as setup violation ?
>could you explain it clearer ?

Equation for Setup Time
Tclk > Tclktoq + Tlogic + Tsetup + Tskew + Tjitter

Equation for Hold Time
Tclktoq + Tlogic - Tskew > Thold

Note that **Hold** Time equation is independent of clk frequency(i.e Time period Tclk)

key things to **note** from above equations
a) once the silicon comes back , if u have **setup** time problem , u can increase the clock period (Tclk) to fix it , whereas if u have **hold** time problem , its a more serious problem and u will need a new metal fix tapeout . (But u can still test the current chip using Low supply voltage, or High temperature or SS corner part that decrease **hold** time violation

Pls. make a **note** that **HOLD violations** are **dangerous** than **SETUP**. To keep it simple way, **SETUP** timing depends on the frequency of operation. But **HOLD** time is not. Let us see the equations here.

T = Frequency of operation (can be variable)
Tcq = Flop clock to Flop output delay (fixed/constant)
Tcomb = Delay od the combinational logic between the Flops (can be variable)
Tsetup = **Setup** time of a Flop (fixed/constant)
Thold = **Hold** time of a Flop (fixed/constant)
Tskew = Delay between clock edges of two adjacent flops (delay offered by clock path) (can be variable)

For **SETUP**,

T >= Tcq + Tcomb + Tsetup - Tskew

If you have **setup** time means u r violating the above rule. i.e some how the equation becomes
T < Tcq + Tcomb + Tsetup - Tskew

Now let us consider two cases.

Case1: During the Design development phase itself.

Now, you have three variables (T, Tcomb, Tskew.) to avoid the **setup** violation.
T : Reduce the frequency such that u saticify T >= Tcq + Tcomb + Tsetup - Tskew. But do u think it is the correct solution. Obviously, NO. This is because we have other options to avoid **setup violations** right.
Tcomb : If you reduce the combinational delay (between the Flops of violated path) such a way that T < Tcq + Tcomb + Tsetup - Tskew will become T >= Tcq + Tcomb + Tsetup - Tskew. So, the **SETUP** violation is avoided. How do u reduce the combinational delay??? Try different logic structure without effecting the functionality. or try to reduce the more fanout nets within the logic. Or upsize or downsize the cells. If it worked out thats fine.
Tskew: If u increase the skew, u can change T < Tcq + Tcomb + Tsetup - Tskew to T >= Tcq + Tcomb + Tsetup - Tskew. How to increase the Tskew? Just keep buffers in the clock path. But be sure doesnt effect the **HOLD** timing.

Case2: After the CHIP is manufatured and is in your hand.

In this case, one cannot access the Tcomb and Tskew. Only the variable that can handle is T.
So, Just reduce the frequency (T) such that the violated equation, T < Tcq + Tcomb + Tsetup - Tskew becomes violation free equation T >= Tcq + Tcomb + Tsetup - Tskew.

So, if u have **setup violations** on a manufatured chip, u can make it work by reducing the frequency.

For **HOLD**,
Thold + Tskew <= Tcq + Tcomb

If you have **setup** time means u r violating the above rule. i.e some how the equation becomes
Thold + Tskew > Tcq + Tcomb and ur aim is to make Thold + Tskew <= Tcq + Tcomb

Now let us consider two cases.

Case1: During the Design development phase itself.

You have two variables in hand (Tcomb, Tskew) to avoid **HOLD violations**.
Tcomb: Increase the Tcomb by adding buffers in the data path. Thus u can change the situation from Thold + Tskew > Tcq + Tcomb to Thold + Tskew <= Tcq + Tcomb. But this might effect the **SETUP** time as you are increasing the delay of combinational path. So this may not be the perfect solution always.

Tskew : Reduce the clock skew so that you will land on Thold + Tskew <= Tcq + Tcomb. To reduce the clock skew, the best solution is to take the help of your PNR engineer.

Case2: After the CHIP is manufatured and is in your hand.

Do you see any variables that will fix the **hold violations** after manufaturing?????!!!!!! NO right. So, its time to DUMP

the chip as we dont deliver malfunctioning chips to the customers.

So becareful with the **HOLD violations**.

Note: One can get those equations if u put those scenarios on a paper and develop the timing diagrams.

It is nice information. Could you please give us more information on

" u will need a new metal fix tapeout . (But u can still test the current chip using Low supply voltage, or High temperature or SS corner part that decrease **hold** time violation)"

what i meant was , when u have **hold** time violation , u dont need to throw away chip and wait for 3 months for fixed chip to come back. in the meanwhile , by playing with voltage and temperature , u can do other functional tests on the chip . since normally **hold** time simulation are done at " FF corner , high voltage, low temperature " which is the pessimistic case for **hold** time , by decreasing voltage , using high temperature, and a SS corner chip , we may be lucky enough to find a part that works , to do other functional tests to catch any other bugs before next tapeout.

Ques:- Suggest the design techniques for improving the testability?
Ans:
Prevention is better than Cure...

The above saying tells everything related to this information.

The scan design or design-for-testability (DFT) has been a standard in ASIC design flow for most designers, Obtaining high fault coverage for a design depends on the quality of the implemented DFT logic, and most \real-world" designs suffer from a variety of DFT related issues, which if left unsolved, result in reduced fault coverage. Designs with gated clocks and resets may not be good for testing, but the design style is used frequently. Since the clock pins of scan cells with gated clocks are not controlled by the primary input clock ports, a scan test is impossible without repair. The situation is the same for gated resets. Repair has been performed by inserting an additional primary input port `test-mode' and test logic in order to feed the top level clock (or reset) signal to the scan cells. This modication results in untestable faults due to the DFT constraints on the `test-mode' port, which lowers fault coverage.

There are some common reasons due to which we generally observe DRC violations. Here I am discussing those common areas and possible fixes if they are un-avoidable reasons.

In order to ensure a successful scan-based design (i.e., achieve high fault coverage), one must design with testability in mind throughout the design process. If the designer is aware of design structures that can derail testing efforts, he or she can adopt alternative design techniques whenever possible.

1. Avoid Bus Contention

One of the bigest hurdle in system on chip (SOC) with respect to ATPG is controlling the internal tristate bus structures.

Bus contention occurs when two drivers are driving different values on the same bus. Since bus contention causes severe damage to the chip, it is important to prevent bus conflicts during normal operation as well as during scan. So, one should handle internal tristates with care and avoid bus contention by design.

In most designs, a bus has more than one driver. To avoid bus contention in normal operation, designers must make sure only one tristate gate at a time is selected. To satisfy testability rules, designers must set the signals for the bus drivers so that contention is eliminated in the test phase.

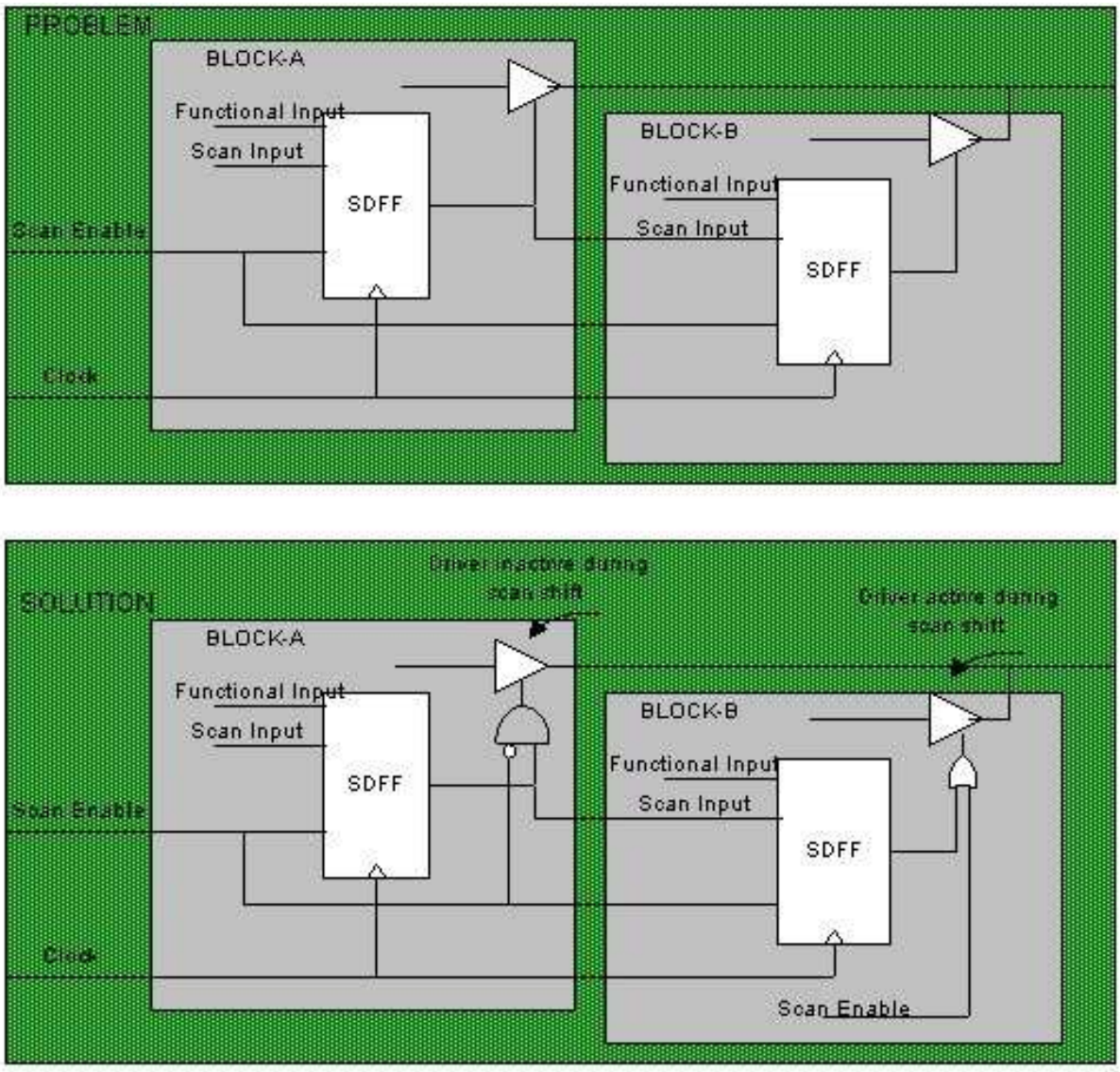
I can't say do not design internal tristate bus structures bus structures. But, I can suggest, if possible, never implement designs with internal tristate bus structures. If you can’t follow this rule, then implement the fewest possible internal tristate-bus structures and guarantee by design that no bus contention can occur on any internal bus during scan
testing.

Be careful of two important while handling contention issues.

1.Ensure that there is no contention on the tristate buses during scan-shift operations.
2. ensure that there is no possible contention on the internal tristate buses during the capture cycles during scan testing.

Most of the DFT tools are intelligent enough to take care of internal tri state buses during test logic insertion and will made proper fixes. And, some fixes has to be made manually. Even most of the ATPG tools will warn about the bus-contentions. Ultimately, test engineer has to take decision whether to generate the vectors that cause contention or to avoid it.

It is suggestable to avoid it or fully-decode the busses else, it may effect the chip productivity.



Three-State Output With one enable and rest of the drivers disabling logic

2. Avoid Gated Clocks

Gated clocks are generally used as one of the power saving techniques. These clocks fan-in through at least one combinational gate apart of buffers and inverters. Enable signal of the gated clocks makes the followed sequential elemets in ON/OFF states. Unfortunately, if the clocks of these flip-flops cannot be controlled from primary inputs, these gated clocks make it impossible to scan in data. Removing these flip-flops from the scan chain is one solution, but this results in a loss of fault coverage.

One can employ one of the following solutions as per the design and/or requirements.

Solution-1 is just excluding DFF from the scan chain. This solution costs test coverage loss.

In Solution-2 the “Scan Mode” input controls a multiplexer that changes/bypass the clock source for DFF. In this case, output of AND gate is un-observable and all faults into the AND gate are ATPG un-testable. To overcome this situation, one can add test-point at the output of AND gate as shown below. By adding this logic to the solution-2 provide better ATPG test coverage.

3. Bypass Pulse generators and internally generated clocks

Here comes the internally generated clocks/pulse generators (PLLs, frequency dividers, pulse generators). Before going into the topic, I strongly suggest that DFT Engineer should always question him/her self that whether he/she can control a particular internal logic element via. primary inputs or not. It not add test logic to control that element through primary inputs. Here the thumb rule is that the primary inputs (scan clocks, reset etc.) should be controllable from any primary input. Derived clocks can render a large part of the design untestable because the flip-flops are not controllable from any primary input.

So, do not use clock dividers in ATPG test mode. If your design contains clock dividers, bypass them in ATPG test mode. Use the Scan Mode signal to control the source of the internal clocks, so that in ATPG test mode you can bypass the clock divider and source the internal clocks from the primary CLK output.

On the bottom line, do not use phase-locked loops (PLLs) as clock sources and bypass the clocks while in ATPG test mode.For pulse generators in the design, one can disable the pulse generators by adding disabling logic or can bypass the pulse generator output with access from a top-level input port as mentioned below.

4. Asynchronous Set/Reset Signals

It is strongly recommended to avoid asynchronous resets in the design. If it is unavoidable one can go ahead with it bearing some test coverage loss. There are different ways to handle asynchronous resets during test logic insertion to get a better test coverage.

There are different situations generally one would face with reset signals.

1.Reset used as data:

In general, reset signal goes to the reset pin of the sequential elements. We discuss in the next point what if the reset is asynchronous reset. In some designs, reset is used as data. i.e. Reset pin goes to the data input of the sequential elements. In such situations, there are two different ways to handle this.

Option-1:
Place a MUX logic at the Reset net, which is synchronously driven to the sequential cell. One input of the MUX would be an actual reset signal; the other input is constant 1|0, which is the inactive state of the Reset. The select line is the ScanMode signal. But the problem with this case is the Reset logic is still not observed, which reduces the coverage. This method provides direct control over the asynchronous reset for scan testing. This method will make the reset controllable and will detect faults on the asynchronous reset of all registers. The disadvantage of the method is that it blocks the functional reset and therefore faults in the reset logic circuitry might be untestable because of the ScanMode constraint. Also, an extra port has to be identify for the external reset pin.

Option-2:
This is very tricky job. It is just to play with the reset signal making it either active high or active low based on the type of flops (active high or active low reset).

Assume, flops have active high reset.

During Shift Mode:

Control the reset signal fed to the flop such that it is active low and keep the scan flop in the active state. And the operation follows the regular scan operation where the data is first shifted into the scan flip-flops holding at active low.

During Hold & Capture Mode:

In this case the vectors are generated without applying any clocks during the capture operation, and the logic feeding into the SET/RESET inputs (the Asynchronous path) of the scan flip-flops is tested. The operation follows the regular scan operation where the data is first shifted into the scan flip-flops holding at active state. Then during hold and capture, de-activate those scan flops by proper controlling of the reset signal fed into it. so that the signals are allowed to propagate through the reset logic and modify the values in the scan flip-flops. Finally the results are shifted out for comparison on the tester. This second option will improve the test coverage either.

Below figure shows you how to control the asynchronous reset signal fed into the flops.
Place an OR gate with one input as Scan Enable signal and the other input of OR gate is the async. reset signal fanout from the reset logic. One can use a dedicated signal in place of the Scan Enable signal.

This method provides control to disable asynchronous resets during scan shift only. The method makes the reset controllable and will detect faults on the asynchronous reset of all registers as well as test the reset logic circuitry. The only disadvantage of this method is that the asynchronous resets will always be controlled by the internal logic and can't be disabled during scan capture. This could cause unwanted resets while attempting to test for other faults. As explained above, one can avoid these unwanted resets by using a dedicated external signal to control the disabling of the asynchronous resets during shift instead of using the main ScanEnable signal.
Note: To follow the 2nd option, one should be familiar with the corresponding tool command flow on which ATPG vectors are generating. All most all the ATPG tools will have a proper command flow to handle the asynchronous resets.

5. Avoid Cross coupled Gates

In some designs, one can observe SR latches. Though they are in expensive way of implementing latches, they create testability problems when both the inputs of cross coupled NAND gates are ant logic 1.

Add logic to make each cross-coupled NAND/NOR gate behave like a buffer
during testing as shown below. When the ScanMode signal is set to ‘1’ during test, the circuit converts the cross-coupled gates into two inverters

If possible, it is better to implement the latches using D-latches instead of cross-coupled SR latches. On the bottom line, as latches reduce the test coverage, avoid inferring latches in the design if possible.

6. Fix Bi-Directional ports

One should fix the bi-directional ports of the design. Ensure that the direction of Bi-directional ports are fixed to either input or output by controlling the enable signal of the bi-directional port. If the enable signal is generated internally, proper DFT logic has to be employed. Or else, if the controlling signal is a primary port, provide proper logic value on the bi-di controlling port during the test to make the bi-di ports unidirectional.

If the primary DFT pads (scan enable, scan-ins, scan-outs, scan-clks etc.) are bi-directional, fix them and make the respective pads, uni-directional accordingly with the ScanMode signal. For the pads, other than DFT ports, fix them using ScanEnable signal. This will improve the test coverage.

Note: If Scan Mode it self is bi-directional, be sure that the bi-di control pin is accessable to the ATE. So that, one can make the ScanMode signal to input by providing the proper value to the control pin during test.

If it is scan clock, scanenable, scanmode, scan-in ports fix their direction to input. If scan-out ports are bi-directional, fix their direction to output.

Following figure shows the fix for scan-clock which is bi-directional with internally generated control logic.

7. Avoid contention due to non tri-state inputs (Wired Logic)

Wired-AND Wired-OR logics leads to contention issues and should be avoided during the test. Most of the DFT tools have the ability to perform contention ability checks on buses and wired logic.

When a net has more than one driving object, wired-AND or wired-OR logic is introduced. A circuit re-modeling will subsequently be performed where wired-ANDs are replaced by AND gates and wire-ORs are replaced by OR gates.

8. Remove Redundant Logic and Test Point Addition

Remove Redundant Logic
This sounds un-reasonable. But still, if the design is not properly optimized and left with some redundant logic, be sure to optimize the logic properly. Most of the DFT tools will take care of optimizing the design.

Test Point Addition:
After test patterns and test coverage is generated by the ATPG tool, be sure to verify the pin-point locations where controllability and/or observability are insufficient. Insert test points at these locations when I/O pad limit is allowed and again rerun the ATPG for better coverage

9. Provide proper time margin for DFT inserted design

To improve a circuit's fault coverage, one may have to add a DFT circuitry in the silicon.
If the designer does not leave room for DFT insertion, the newly created design may violate timing constraints. Thus, during the early design stage, it is important to leave, say
15%-20% timing margin for future DFT insertion. It is strongly recommended to re-simulate the DFT inserted design before submitting it to ATPG and layout

10. Floating Nets

Avoid floating nets in the design. This may cause unwanted leakage currents that leads to power failures. So, it is strongly recommended to verify for floating nets in the design before and after the DFT insertion.

11. Avoid Latches

Latches cannot be scanned unless they are converted to scan latches. Such a conversion creates an extremely large area overhead. If the latches are part of an embedded memory, it is possible to model all the latches together as a memory. Using a memory model can reduce the difficulty of generating patterns for faults near the latches; however, special
tests are required for the latches.

When latches are not part of the memory, a more general solution is to make the latches transparent during test mode. This takes care of the problem of propagating fault effects through the latches, but the faults on the enable logic of latches become completely untestable, reducing fault coverage. The best solution is to avoid using latches in designs as much as possible.

12. Try avoiding combinational feedback loops

What is a combinational loop?
It is a simple form of D-Latch.

Designs that contain combinationalfeedback loops have inherent testability problems. Combinationfeedback loops may introduce internal logic states to a design that scan-storage elements cannot control.

ATPG tools have a few methods for breaking combinational feedback loops. Some of these methods are less harmful to fault coverage than others, but all of them result in some loss of coverage. Therefore, you should avoid combinational feedback loops whenever possible. Most ATPG tools inform users of all the combinational-feedback loops present in a design. If you cannot avoid these feedback loops, then you should break the feedback loop by inserting an additional flip-flop that is present in the feedback path only during scan-test mode. This modification results in the highest fault coverage. If you cannot insert a flip-flop, then insert a multiplexer in the feedback path that drives a constant value during scan-test mode. This approach results in lower coverage than the flip-flop option but higher coverage than if you allow the tool to break the loop by assuming an unknown value as a result of the
loop.

Combinational feedback loops introduce sequential behavior into otherwise combinational logic, making it difficult to use combinational ATPG tools. In addition, such loops may lead to races and hazards, resulting in unpredictable circuit behavior. These loops are generally delay-dependent and therefore can't be tested with any ATPG algorithm.

Typically, designers will omit combinational feedback loops from their designs. However, the loops can be created unknowingly when two portions of a design done by different designers are tied together.

If creating combinational feedback loops are unavoidable, designers should add test logic to the design. This fix allows combinational feedback loops to be broken during testing by using a scan flip-flop. One should break this loop during scan mode for the ATPG tool to predict the operation of the circuit.

Ques:- Why latches (other than lock-up and clock gating) in design are bad for ATPG? We always loose coverage by putting them on transparent mode by constraining enable signal. How can we increase this coverage?

Ans:- I think you answered your own question regarding why we lose coverage,
"putting them on transparent mode by constraining enable signal".

Any logic driving the enable signal will lose some coverage due to this constraint.

If the enable signal is driven by a large cone of logic, you can increase the coverage by adding a "force transparent on" gate close to the enable of the latch, and add an observe flip-flop just before the added gate, so that most of the faults in the logic cone can be recovered.
Without modifying the circuitry, recovering fault coverage in ATPG really depends on how these latches are used in the functional path.

Ques:- when both the Transition-delay and path_delay target for transition faults, why we should have two different methods to detect the same?
Ans:- Transition delay is similar to stuck-at atpg, except that it attempts to detect slow-to-rise and slow-to-fall nodes, rather than stuck-at-0 and stuck-at-1 nodes. A slow-to-rise fault at a node means that a transition from 0 to 1 on the node doesnot produce the correct results at the maximum operating speed of the design. Similarly a slow-to-fall fault means that a transition form 1 to 0 on a node doesnot produce the correct results at the maximum speed of the design.Transition delay fault targets single point defects.

The Path delay fault model is useful for testing and characterizing critical timing paths in the deisgn. Path delay fault tests exercise critical paths at-speed to detect whether the path is too slow because of manufacturing detects or variations.
Path delay fault testing targets physical defects that might affect distributed region of a chip. For example, incorrect field oxide thicknesses could lead to slower signal propogation times, which could cause transitions along a critical path to arrive too late.
Note:----------
If we attain 100% coverage with transition atpg test then we dont need to do the path delay test.
But this is not the case in most of our designs. Then how can we GUARANTEE that all the critical paths are targeted by Transition ATPG
so we give the critical path to the atpg tool and tell it to generate patterns for it
this is called path delay

Ques:- What is Burn in test? Why is it done?
Ans:- Burn-in is the process by which device infant mortality failures are accelerated through application of temperature and stress voltages for specific periods of time. The goal of burn-in is to stress the device as much as possible thereby accelerating device infant mortality rate. Since scan based test has higher coverage, scan test vectors are used to stress the logic portion of the device. This can either be done by performing device level ATPG by applying external test patterns on onchip LBIST. Memory BIST is used for memories
Note :----------

High toggled atpg vectors are generated to sensitize the internal nodes of the chip so that chip will get more stress

Ques:-- If for a DFT production set we have both PATH Delay and Transition patterns which scheme should be done first ?
Ans:- Its always better to do Path_delay first and then the transition delay.

Path_delay- start flop and end flop are given by the user.(i.e) transition through a well defined path-(More effective)

Transition-Start and end flop is decided by the tool,so trasition may happen through a shorter or longer path-(Less effective)

1)path delay basically targets for all the critical paths in the design.
2)Generate the path_delay patterns for all the critical paths in the design and write down the faults for the same.
3) Generate the transition delay pattern by loading the faults on the path_delay, such that the faults detected in the path_delay are not Re-targeted.
4) The major reason to follow the above sequence is in trans-delay we are not sure weather a transition has really happened through the critical path.

(A)-If we do the transition-ATPG first, we are not sure whether the transition has happened through the critical path,In such case we may not have pattern which may catch the fault through the critical path,but the fault may be detected through some other shorter paths which may add the faults to the detected list.
(B)If we run a path_delay after the above step(A) then we may have a pattern for the critical path,but it leads to the pattern redundancy because we have already a transition pattern for the same fault.

Ques:-- What is IDDQ Testing?
Ans:- IDDQ Testing can detect certain types of circuit faults in CMOS circuits that are difficult or impossible to detect by other methods. IDDQ testing, when used with standard functional or scan testing, provides an additional measure of quality assurance against defective devices.

IDDQ testing refers to the integrated circuit (IC) testing method based upon measurement of steady state power-supply current. Iddq stands for quiescent Idd, or quiescent power-supply current. Majority of IC'sare manufactured using complementary metal–oxide–semiconductor (CMOS) technology. In steady state, when all switching transients are settled-down, a CMOS circuit dissipates almost zero static current. The leakage current in a defect-free CMOS circuit is negligible (on the order of few nanoamperes). However, in case of a defect such as gate-oxide short or short between two metal lines, a conduction path from power-supply (Vdd) to ground (Gnd) is formed and subsequently the circuit dissipates significantly high current. This faulty current is a few orders of magnitude higher than the fault-free leakage current.Thus, by monitoring the power-supply current, one may distinguish between faulty and fault-free circuits.

Ques:- Why do IDDQ Testing?

Ans:- For functional testing, a tester applies a sequence of input data and detects the results in the sequence of output data. Then, the output sequence is compared against the expected behavior of the device. An advantage of functional testing is that it exercises the device as it would actually be used in the target application. However, this type of testing has only a limited ability to tests the integrity of a device's internal nodes.

with functional testing only, an internal defect could slide by undetected.

The methodology for scan testing is all the sequential elements of the device are connected into chains and used as primary inputs and primary outputs for testing purposes. Using automatic test-pattern generation (ATPG) techniques, you have the capability to test a much larger number of internal faults than with functional testing alone. The goal of ATPG is to set all nodes of the circuit to both 0 and 1, and to propagate any defects to nodes where they can be detected by test equipment.

Using both functional and scan testing you greatly increases your odds at finding an internal defect, but what if the defect is not controllable or can't be observed? That is where IDDQ testing can help.

Ques:- why do we need to put first negative edge flops followed by positive edge flops in the scan chain?

Ans:- The rule is there should not be 2 shift during one clock period. So if you put +ve edge flop followed by -ve edge flop, there is a chance of 2 shift (if the clock skew between 2 clocks is small) in one clock period. But if you put -ve edge flop then +ve edge flop, then there is no chance of that. because the +ve edge come in the next period. Or if ur design needs that +ve edge then -ve edge then you need to insert a lock up latch (if skew is small).

This will depend on the nature of clock involved in your scan design.
If Clock is RTZ then chain should be : negedge ->posedge
If clock is non RTZ then it should be vice-versa

reason is value loaded on first flop should not pass on to next FF in same cycle.

Ques:- How to toggle reset to get coverage ?

Ans:- If the reset is asynchronous (and properly bypassed during scan), you can declare the reset pin as a clock during ATPG, and ATPG will toggle it accordingly to get faults on reset pin.

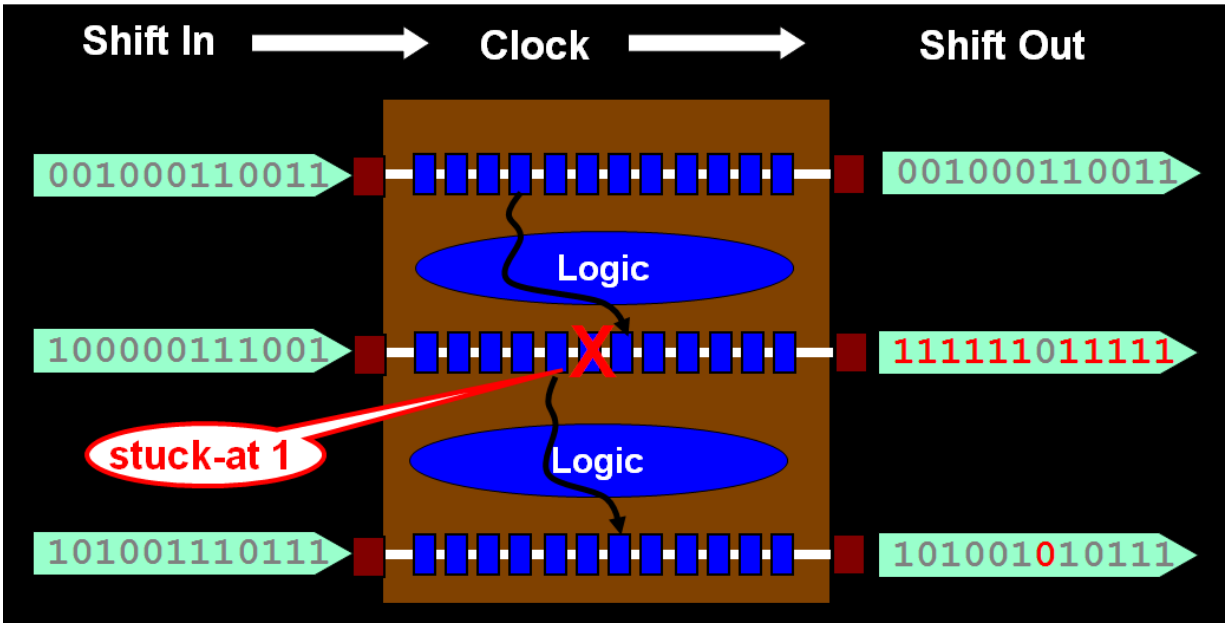
If the reset is synchronous, you can treat the reset pin as a normal data pin, and ATPG should be able to cover faults on the reset. If you run transition fault ATPG. Reset usually cannot toggle at-speed, so you may not want to declare the reset as a clock when running transition fault ATPG. You can also try to run the patterns that toggles the reset as a clock pin at a reduced speed on the tester, if you worry about transition fault coverage on reset.

Why scan chain patterns fail ?

A chain fail is when one or more scan chains in non-operational, that is to say data cannot be accurately shifted through the flip-flops arranged as shift registers that constitute a scan chain. The reasons they suck are many, but here are my Top 5:

1. Scan chain fails usually dominate the logic yield loss early in the technology ramp, right when it's most critical to get some actionable data
2. Scan chain fails require a huge amount of data to perform fault simulation based chain diagnosis for defect localization
3. Scan chain fails require specialized ATE hardware to perform defect localization without resorting to simulation based chain diagnosis
4. Scan chain fails have all the same possible root cause explanations as logic fails (logic fails = scan chains are functional, silicon is not) and several additional 'chain fail only' root causes
5. Scan chain fails are often mixed in with logic fails and product/yield engineers disregard this important distinction.

What Causes a Chain Fail?



Defects That Cause Chain Fails

A chain fail occurs when the data being shifted out of a scan chain is found by the Automatic Test Equipment (ATE) to be corrupted for a reasonably higher percentage of cycles. Defects that cause chain fails come from:

- A defect in the scan flip-flop that effects shift functionality. Note that it is possible to have a defect in a flip-flop that only effects functional operation, and this would not be a chain fail.
- A defect on the net that connects the scan out signal from one flop to scan in signal on another flop

- A defect in the scan enable tree that feeds one or more flops. The scan enable tree is huge (high critical area) and usually not signed off for timing performance.
- A defect in the clock tree that feeds one or more flops. The clock tree is also extremely high critical area, but is obviously signed off for timing performance.

Parametrics That Cause Chain Fail

There is another equally important list of chain fails to consider. This class of fails can't really be considered defects in the traditional sense because a parametric problem can't typically be imaged in a failure analysis Scanning Electron Microscope (SEM) or Transmission Electron Microscope (TEM). However parametric effects can be just as devastating to yield, and often are responsible for yield loss early in the process or product ramp. Examples include:

- Setup or Hold time faults. These can often be blamed on a timing mismatch that was either found by Static Timing Analysis (STA) and ignored, or missed by STA. I once worked on a yield problem where a setup fault could only be identified by STA when the design is simulated at the slow timing, high voltage, low temperature corner (no one simulates that corner for signoff) and yet the flop was failing in silicon.
- Power droop can cause a flop to lose it's state. This may only effect a certain flop or set of flops in the design, but this lost state will effect the whole chain as each value is shifted through a weak flop. Power droop is especially problematic during scan testing and logic BIST testing because there is abnormally high activity on the chip (half the flops toggling simultaneously). I remember one time my classmate and I were tying to figure out why a part running logicBIST was running below its intended frequency. We would check the max frequency, solder a capacitor onto the loadboard, and recheck the max frequency. It went up every time we added a capacitor: power droop in action.
- Connection between ATE and scan chain inside the chip. It is possible to have a chain fail occurring because of effects outside of the silicon. Not usually for digital pins like scan chains, but it's possible to have a bad calibration on the test head. Also I've observed chain fails that were induced because of a resistive connection between the probecard and the chip pads. You also can't rule out the possiblility of a bad package connection if you are working on final test units. Note that there is an extension of this where all chains are failing because power is not being supplied to the chip, like when I would forget to touch the probecard down on the wafer before I started testing.

Some Details on Why Chain Fails Suck

1. Chain Fails are often the dominant yield loss mechanism during product or process ramp. There are several reasons why this is true and they all serve to confuse the issue as well as the engineer. Firstly, lets note the fact that scan chains are usually heavily routed in the upper metal layers. This is a natural consequence of the fact that scan enable and clock-trees are for the most part considered global routes and a scan out net can connect fairly distant flops because it's a slow speed and non-functional connection. All of this ends up meaning that there is a whole lot of upper metal layer critical area tied up nets that, if defective, would induce a chain fail. Early in a process ramp the defectivity is always high. Combining the high critical area of the chain fails with the high defectivity of the process ramp and you can see why there are so many chain fails at the beginning of a process. It's not because they are systematic, it's because they catch a lot of defects. Also, a chain fail will almost always mask the existence of a logic fail (the exception being compound diagnosis which can separate the two), so when there are a lot of defects it will look like a lot of chain fails. *Do not be*

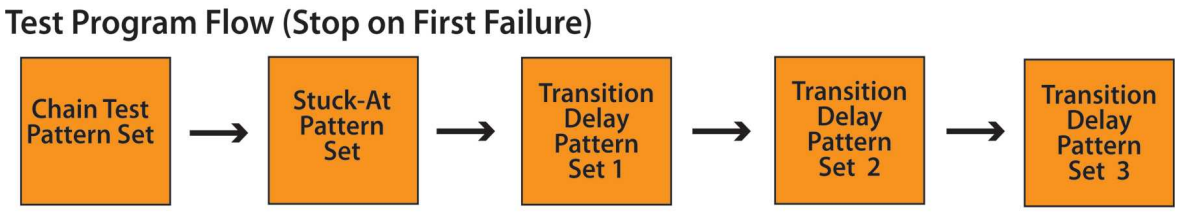
fooled into thinking that a high number of chain fails can only be the result of a systematic defect. Every design is different and it's possible to have 30-70% of logic defects result in a chain fail with perfectly random defectivity.

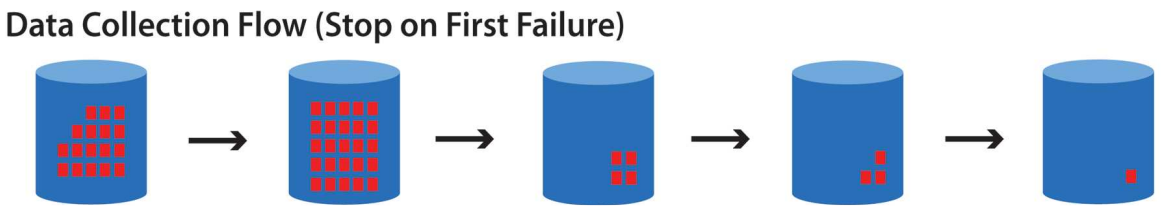
2. Scan chain fails require a huge amount of data to perform fault simulation based chain diagnosis for defect localization. Why is this? It's because there are so many freaking failing bits! In actuality a lot of those bits could be ignored (we already know the chain is Stuck-at-0), but the tester isn't smart enough to know which bits to ignore. Scan chain diagnosis, like logic diagnosis, needs a certain number of failing patterns (entire shifts through the chain) in order to have good resolution. Therefore the longer the scan chain the more data needs to be collected. As an interesting sidenote, the opposite is also true (*). An interesting shift in the industry is occurring in this space with the emergence of zero overhead datalogging like theTeseda V520 and its kin. What used to be an impossible amount of data to collect for chain fail diagnosis (often 10's of thousands of cycles) doesn't really take that long with a symmetric capture buffer. This is leading to a more widespread adoption of chain diagnosis (both hardware and software) as a way of understanding yield loss. Unfortunately, for many existing ATE fleets this rapid data collection capability isn't the case, which is why is stays on my list of things that suck.

3. Scan chain fails require specialized ATE hardware to perform defect localization without resorting to simulation based chain diagnosis. It is possible to perform scan chain defect localization using only hardware. The company that I know of doing this is Verigy with their ChainAnalyzer, part of the Inovys acquisition. From what I have heard about this, it seems to be compelling technology which is able to automatically apply patterns to the scan input pins, and check values on the scan output pins, in order to localize which flip-flop is failing. The only reason I would call this a downside is that it requires specialized hardware, and if you don't have the hardware, you can't use it (obviously). To those of you that have the hardware and can use it, Kudos!

4. Scan chain fails have all the same possible root cause explanations as logic fails and several additional 'chain fail only' root causes. Continuing on the line of thinking in 1. above, chain fails can come from random defects. For random defects, chain fails and logic fails have the same basic root cause. The problem comes when there really are systematic defect related chain fails that are buried in amongst all of the random defect related chain fails. Thus in order to tackle the systematic chain fails, one must first separate the random and systematics. This is much easier said than done. Take for example a systematic defect that affects the chain functionality in a scan-flip flop. Since the flip-flop can be placed anywhere on the design, the defect will seem to occur randomly on the design; and yet it is a systematic chain fail. Separating random from systematic generally involves either zonal analysis (ITC05, ISTFA11) or design normalization (let me know if you have a great reference to this). The problem with design normalization for chain fails is that it's not typically clear which locations result in a logic fail vs. a chain fail. I've seen a lot of people normalize by chain length, but they really should be normalizing by chain critical area (at the very least).

5. Scan chain fails are often mixed in with logic fails and product/yield engineers disregard this important distinction. I actually consider this one to be the biggest reason chain fails suck, and yet the easiest to solve. The overwhelming majority of the time product and test engineers have no idea what % of the yield loss is related to chain fails vs. logic fails. They will end up datalogging 2 parts, both will be chain fails, and they will wrongly conclude that they have 100% systematic chain fallout. The right answer is to separate the Chain Test Pattern Set from the Stuck-At Pattern Set, test them individually, and assign each it's own softbin. This will be a ~zero test time impact approach that will provide visibility on how much of the scan yield loss is chain related. To me this is a no-brainier. Even if the data never gets used, it's free to collect: no datalogging required!.





By now I hope I've convinced you that chain fails suck, but can be conquered with patience and hard work (and some sort of tester). I'd be happy to hear about any stories you have to support my suckness conclusion, or solutions you have for additional desuckifying.

(*) When scan chains are compressed, like with Mentor Graphics TestKompress and Synopsys DFTMAX, less data volume is required to be collected for chain diagnosis because the number of patterns required is the same but the chain length is reduced by the compression factor!

IJTAG automated chip-to-system-level IP integration



Mentor Graphics announced the full interoperability between the Tessent IJTAG chip-level IP integration product and ASSET InterTech's ScanWorks platform for embedded instruments, which includes chip, circuit board and system-level IJTAG tools.

This combination delivers a comprehensive chip-to-system-level automated IJTAG IP integration solution, greatly simplifying the user's ability to leverage chip-level resources to the printed circuit board (PCB) or system levels.

Chip IP that is compliant with the IJTAG standard (IEEE P1687), such as self-test or diagnostic functions, can now be seamlessly accessed from board debug, validation and test systems or in the field from system software.
Tessent IJTAG combined with ScanWorks lets engineers access the operational and diagnostic features of all IP blocks in the design from a top-level interface, greatly simplifying the job of integrating the hundreds of IP blocks in a typical system. Interoperability between the two solutions revolves around the IEEE P1687 standard's Instrument Connection Language (ICL) and Procedural Description Language (PDL).

The Tessent IJTAG tool reads ICL and PDL code delivered with third-party IP blocks and verifies that it is IEEE P1687 (IJTAG)-compliant. It then generates a logic network and associated ICL to integrate all the IP blocks in a design and processes the PDL for each IP to create composite chip-level PDL. The ScanWorks product then reads chip-level ICL and PDL for use in chip debug and also retargets the PDL to a board or system level interface.

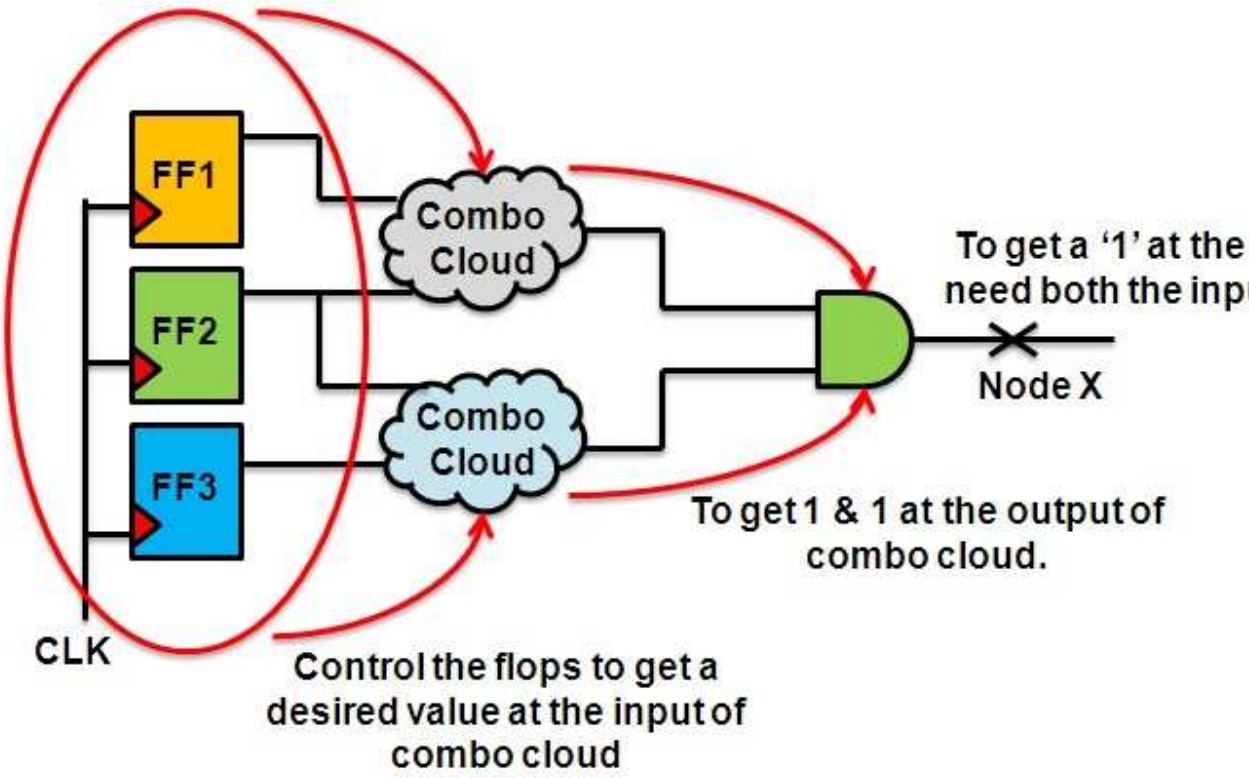
Two Pillars of DFT: Controllability & Observability

Just like Timing is built on two pillars: ***Setup & Hold***, entire DFT is built on two pillars:***Controllability & Observablity***. Very often you would find DFT folks cribbing that they can't control a particular node, or don't have any mechanism to observe a particular node in question. You may like to review the previous post: DFT Modes: Perspective before proceeding further.

Shifting our attention to the pillars of DFT, let's define the two.

- ***Controllability:*** ***It is the ability to have a desired value (which would be one out of 0 or 1) at any particular node of the design. If the DFT folks have that ability, they say that that particular node is 'controllable'. Which in turn means that they can force a value of either 0 or 1, on that node!***
- ***Observability:*** ***It is the ability to actually observe the value at a particular node whether it is 0 or 1 by forcing some pre-defined inputs. Note that, unlike the circuit that we make on paper, the SoC is a colossal design and one can observe a node only via the output ports. So, DFT folks actually need a mechanism to excite a node and then fish that value out of the SoC via some output port and then 'observe' it!***

Ideally, it is desired to have each and every node of the design controllable and observable. But reality continues to ruin the life of DFT folks! (Source: Calvin & Hobbes). It is not always possible or rather practical to have all the nodes in a design under your control, because of the sheer complexity that modern SoCs possess. And therefore, it is the reason you would hear them talk about ***'Coverage'.*** Let's say coverage is 99%, this means that we have the ability to control and observe 99% of the nodes in the design (A pretty big number, indeed!).
Now let's take some simple examples.

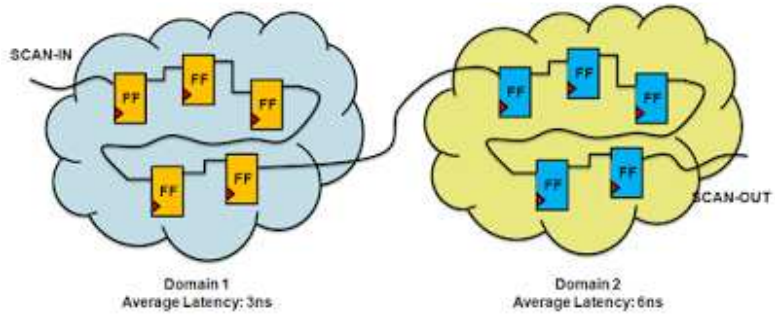


In the above example, if we have control the flops such that the combo cloud results in 1 at both the inputs of AND gate, we say that the node X is ***controllable*** for 1. Similarly, if we can control any input of AND gate for 0, we say that node X is ***controllable*** for 0. Similarly, let's say we wish to observe the output of FF1. If we can somehow replicate the value of FF1 by making the combo clouds and AND gate transparent to the value at FF1, we say that output of FF1 is observable. Intuition tells us that for AND gate to be transparent, we should have the controllability of other node for 1. Because when one input of AND gate is 1, whatever is the value at the other input, it is simply passed on!!

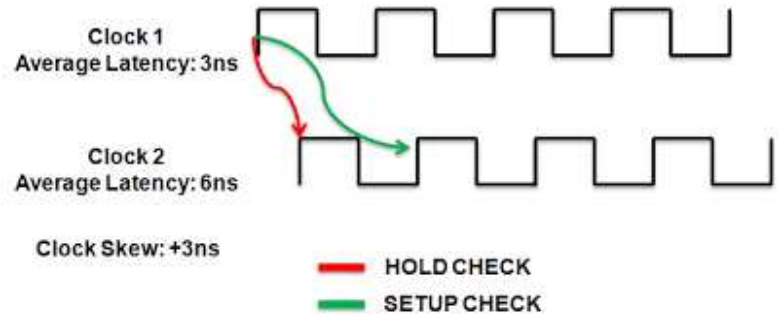
Lock-Up Latch: Implication on Timing

Lock-Up Latches are important elements for any STA engineer while closing timing on their DFT Modes: specially the Hold Timing Closure of the Shift Mode. While shifting, the scan chains come into picture, which are nothing but the chains of Flops involving the output pin of one flop, connected to the Scan-Input or Test-Input pin of the other flop, and so on, forming a chain.

Now, imagine that we have two functionally asynchronous domains 1 and 2. Functionally asynchronous means that during the normal mode of operation (Functional Mode), the two domains do not interact with each other. However, very rarely do the designers have the liberty to make a separate scan chain for functionally asynchronous domains. Let's consider the following scenario: where domain 1 has the average clock latency of 3ns, and domain 2 has the average latency of 6ns. And the time period of the test clock is, let's say, 10ns.



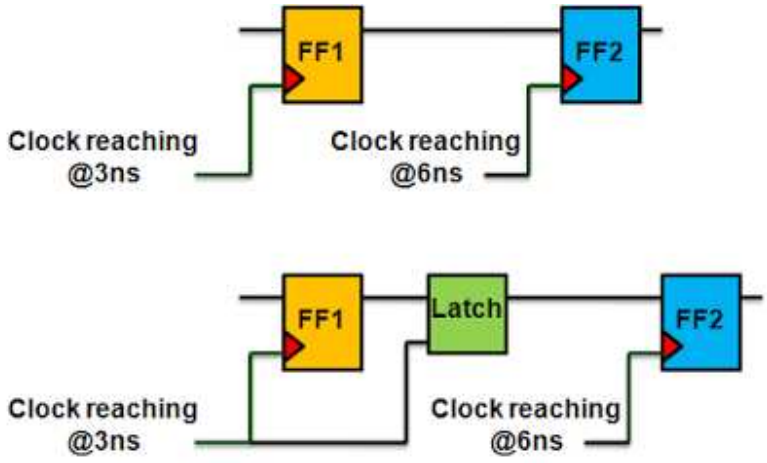
Now, let's see the timing checks for this scenario. The output of the last flop of the domain 1 is in scan-chain and connected to the Test-Enable input of the first flop of domain 2. The check would be like:



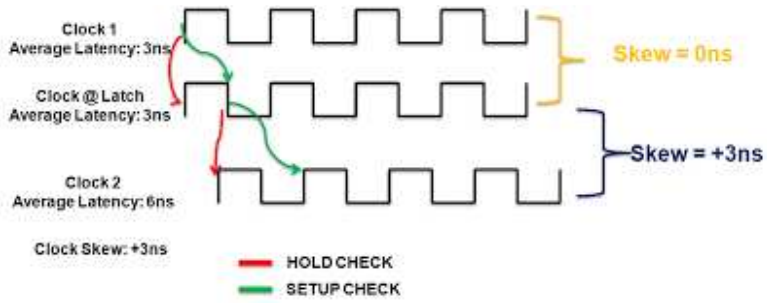
Owing to the positive clock skew, the setup check would be relaxed, but hold would be critical. Two possible options to fix the hold timing:

- *Insert the buffers, to add sufficient delay, so that hold is finally met.*
- *To add the Lock-Up Latch between the two flops where scan chain crosses the functional domains.*

The first might not be a robust solution because the delay of the buffers would vary across the PVT Corners and RC Corners. In the worse case, there might be a significant variation in the delays across corners and you might witness closure of timing in one corner, but violation in other corner!
Second solution is a more robust solution because it obviates the above scenario. Hence see how it does it.



Timing Check would be like:



Hence, both setup and hold checks are relaxed!!

[Please note that in the above circuit, the latch is a negative level triggered latch.]

DFT Modes: Perspective

As more number of transistors are finding their way onto a single SoC, Design for Testability (DFT) is becoming an increasing important function of the SoC design cycle. As the technology nodes are shrinking consistently, the probability of the occurrence of the faults is also increasing which makes DFT an indispensable function for modern sub-micron SoCs. What are the possible faults within an SoC and whhat all ways are possible to detect them? We will take them up briefly.

Imagine that you own a chip manufacturing company for the automotive industry. The end application can be something meant for infotainment, engine management, rear view camera, ethernet connectivity, power glasses or for a critical application like collision detector or air-bag control etc. You wouldn't like to sell a faulty chip to your customers for two main reasons:

- ***Trust of the customer*** *which would impact the goodwill of the company.*
- ***Loss of business:*** *Maybe because the customer opted for some other semiconductor vendor or even worse, the chip failed at the user end and he sued your customer who ultimately sued you!*

Hence, it is pretty important to test the chip before shipping it out to the customers.

Types of fault and their detection:

- ***Structural Fault: Basically refers to the faults due to faulty manufacturing at the fabs. Even a tiny dust particle has cause shorts or opens in an SoC.***

Let's try to understand it from our example. Let's say you manufactured the chip but there is a fair probability that there might be some structural inadequacies in the form of shorts or opens. Imagine any digital circuit. Single short or a single open can cause the entire functionality of the device to go haywire. Structural testing is done during the DFT tests or modes called as ***Shift*** *and* ***Stuck-At Capture****. We'll discuss these in detail in the upcoming posts. Note that these tests are conducted after manufacturing, before shipping the part to the customer.*

- ***Transition Faults: Signal transitions are nothing but the voltage levels while they switch from either 'high' to 'low' or vice-versa. There is a designated time before the clock edge when all the signals should be stable at the input of the Flop (a very crude definition of setup time) and also a designated time after the clock edge when none of the signals should change their states at the input of the Flop (a very crude definition of hold time). Any such fault in the transition times (conversely: setup or hold violations) is referred to as a transition fault.***

Going back to our example. Suppose that you first filtered out the chips which had some structural fault. Now you would test the remaining chips for transition faults. What would happen if you ship a chip with a transition fault to a customer? If it had a setup violation, the chip will not be able to work at the specified frequency. However, it will be able to work at a slower frequency. If it had a hold violation, the chip will not be able to work at all! One possible consequence from our example could be that in an event of a collision you would expect a few micro- or nano- seconds for the air bag to open up, it might ending up taking seconds! Unfortunately, it would be too late.
The ***At-Speed*** *test is used to screen the chip for transition faults.*

Broadly speaking, there are only two types of the faults as discussed above. However, there's another possibility which can arise.

Imagine that your car has an SoC which senses a collision and opens the air bag within a few micro-seconds of the collision. You would expect it to open up if such a scenario arises. But what if your car is, let's say, 6 years old and the chip is now not functioning as expected. In this case, you would like to test the chip first. And if it is fine, you may proceed on to ignite the engine and start the car. Such a scenario would demand conducting a test while the chip is in operation. Such a DFT test is called**LBIST Test (Logical-Built-in-Self Test)**. In an LBIST test, one would be testing the entire chip or a sub-part of it for structural and/or transition faults. Such a test for memory is referred to as **MBIST Test (Memory-Built-in-Self-Test)**.

An important characteristic of a built in self test is that it is self sufficient. It would carry out the test internally, following a software trigger, without any external input; carry the test; and send out a signature in terms of PASS/FAIL.

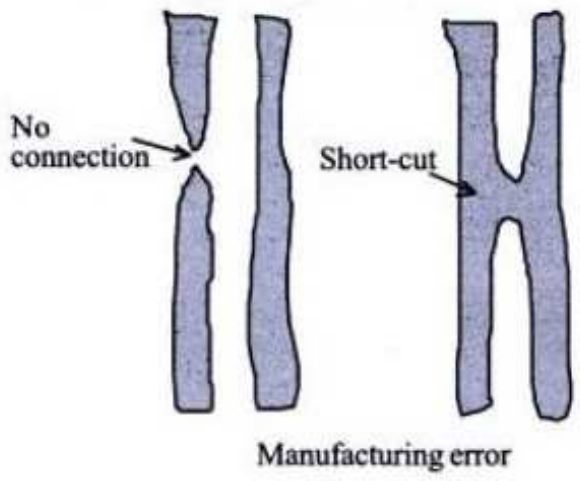
A failed LBIST test on the air-bag chip, might flash a warning and can prevent the user from starting the car engine! It might sound cruel, but it can surely save your life!

Design for Testability: The Need for Modern VLSI Design

DFT is the acronym for ***Design for Testability***. DFT is an important branch of VLSI design and in crude terms, it involves putting up a test structure on the chip itself to later assist while testing the device for various defects before shipping the part to the customer.

Have you ever wondered how the size of electronic devices is shrinking? Mobile phone used to be big and heavy with basic minimal features back in 90s. But nowadays, we have sleek phones, lighter in weight and with all sorts of features from camera, bluetooth, music player and not to forget with faster processors. All that's possible because of the ***scaling*** of technology nodes. Technology node refers to the channel length of the transistors which form the constituents of your device. Well, we are moving to reduced channel lengths. Some companies are working on technology nodes as small as 18nm. Smaller is the channel length, more difficult it is for the foundries to manufacture. And more are the chances of manufacturing faults.

Possible manufacturing faults are: ***Opens*** and ***shorts***.



The figure shows two metal lines one of which got "open" while other got "shorted". As we are moving to lower technology nodes, not only the device size is shrinking but that also enables to pack more transistors on the same chip and hence density is increasing. And manufacturing faults have become therefore indispensable. DFT techniques enable us to test these (other kinds as well) faults.

Kinds of defects:

- ***Opens and shorts***, as mentioned above, can cause functional failures. A kind of open and shorts, where any node might get shorted to ground is referred to as stuck-at-0 (SA0) fault or in cases where the node might get shorted to the power supply is referred to as stuck-at-1 (SA1) fault.
- ***Speed Defect***: May arise due to coupling of a net with the adjacent net and hence affecting the signal transition on it.
- ***Leakage Defect***: Where a path might exist between the power supply and ground and this would cause excessive leakage power dissipation in the device.

In a nutshell, DFT techniques are important especially for sub-deep micron technology nodes (i.e. below 90nm). And it can prevent shipping any defective part to be customer, which instead would have caused revenue and goodwill loss for the semiconductor design companies.

Multi-Cycle Paths: Perspective & Intent

Multi-Cycle Paths, as the name suggests, are those paths which need not be timed in one clock cycle. It is easier said than done! Before we discuss further, let's talk about what Multi-cycle paths does not mean!!

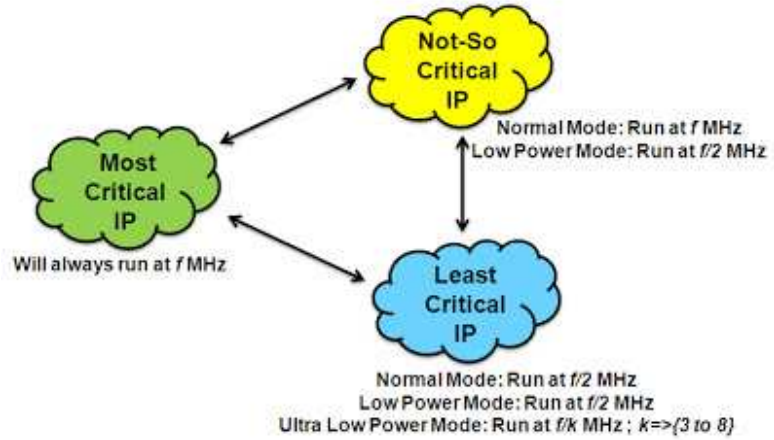
Myth 1: All those paths which the STA team is unable to meet at the highest clock frequency, are potential multi-cycle paths.

Reality: Multi-cycle paths are driven by the architectural and design requirements. STA folks merely implement or appropriately said, model the intent in their timing tools! A path can be a multi-cycle path even if the STA team is able to meet timing at the highest clock frequency.

Myth 2: It is by some magic that the design teams conjure up how many cycles it would be appropriate to for a path to function! <Apologies the hyperbole! :)>. And STA team follows the same in their constraints.

Reality: MCPs are driven by the intent. And implementation is governed by that intent which includes but is not limited to the number of run modes a particular SoC should support.

Consider the following scenario:

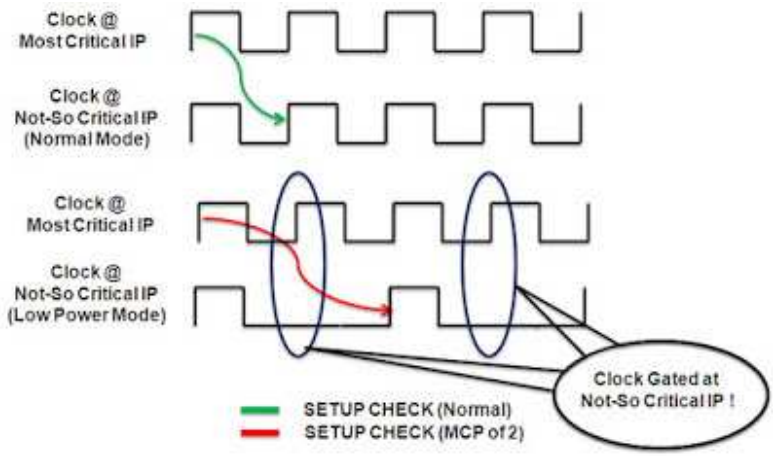


Normal mode, Low Power Mode and Ultra Low Power Modes can be considered to be the different run modes of the SoC. You can say that the customer can choose at what time which run mode would be better. Example: when performance is not critical, or your device can go to 'hibernate' mode, you (or the software) can allow the non-critical parts of the SoC to go into a Low Power Mode, and hence save power!

Consider the specifications:

- **Normal Mode**: *Most Critical IP* & *Not-So Critical IP* would work at *f MHz. Least Critical IP* would work at *f/2 MHz*. Interaction between any two IPs would be at slower frequency.
- **Low Power Mode**: *Most Critical IP* would work at *f MHz. Not-So Critical IP* & *Least Critical IP* would work at *f/2 MHz*. Interaction between any two IPs would be at slower frequency.
- **Ultra Low Power Mode**: *Most Critical IP* would work at *f MHz. Not-So Critical IP*would work at *f/2 MHz.* And *Least Critical IP* would work at *f/k MHz; (k=> 3-8).* Interaction between any two IPs would be at slower frequency.

Consider the Low Power Mode. Any interaction within the *Not-So Critical IP* would be at slower frequency. However, any paths between the *Most Critical IP* and *Not-So Critical IP* would be Multicycle path of 2 in the low power mode. In this case, the clock at the *Not-So Critical IP* is gated selectively for every alternate clock cycle to implement an MCP. Hence data launched from*Most Critical IP* now effectively gets two clock cycles (of the faster clock) to reach the *Not-So Critical IP*. The following figure explains the intent:



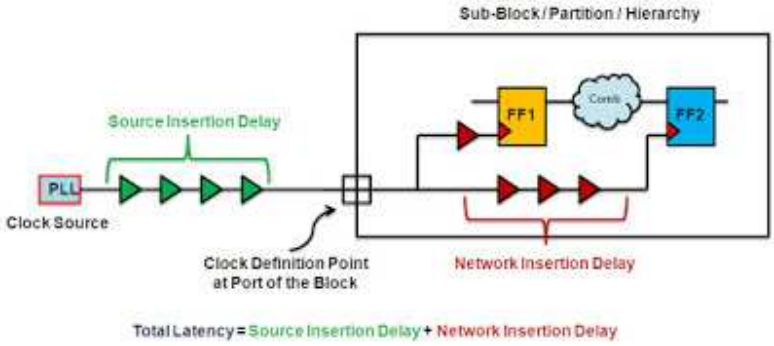
This much for the intent! However, as we mentioned that for the Least Critical IP, depending on the mode, would work at *f/k MHz => (k=3-8)* one might need an MCP of 2, 3, 4.... and so on. This calls for a need of a configurable implementation of multicycle paths.

Clock Jargon: Important Terms

Clock to an SoC is like blood to a human body. Just the way blood flows to each and every part of the body and regulates metabolism, clock reaches each and every sequential device and controls the digital events inside the SoC. There are many terms which modern designers use in relation to the clock and while building the Clock Tree, the backend team carefully monitors these. Let's have a look at them.

- **Clock Latency**: Clock Latency is the general term for the delay that the clock signal takes between any two points. It can be from source (PLL) to the sink pin (Clock Pin) of registers or between any two intermediate points. Note that it is a general term and you need to know the context before making any guess about what is exactly meant when someone mentions clock latency.
- **Source Insertion Delay**: This refers to the clock delay from the clock origin point, which could be the PLL or maybe the IRC (Internal Reference Clock) to the clock definition point.
- **Network Insertion Delay**: This refers to the clock delay from the clock definition point to the sink pin of the registers.

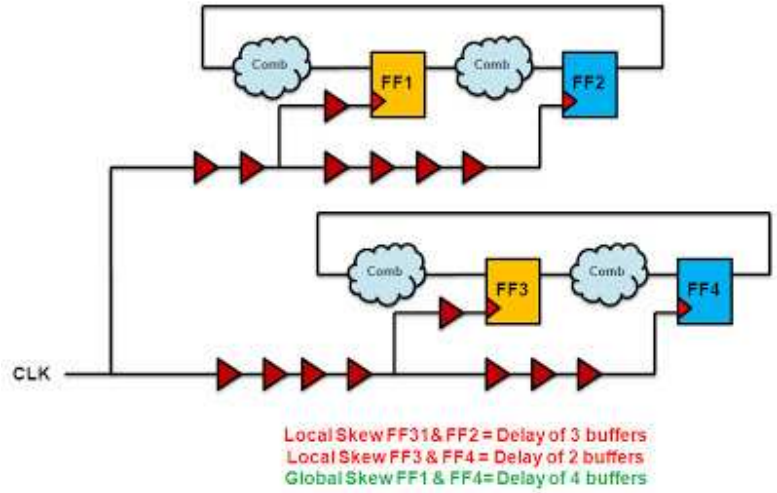
Consider a hierarchical design where we have multiple people working on multiple partitions or the sub-modules. So, the tool would be oblivious about the "top" or any logic outside the block. The block owner would define a clock at the port of the block (as shown below). And carry out the physical design activities. He would only see the Network Insertion Delay and can only model the Source Insertion Delay for the block.



Having discusses the latency, we have now focus our attention to another important clock parameter: The Skew.

We discusses the concept of skew and it's implication on timing in the post: Clock Skew: Implication on Timing. It would be prudent to go through that post before proceeding further. We shall now take the meaning of terms: Global Skew and Local Skew.

- **Local Skew** is the skew between any two related flops. By related we mean that the flops exist in the fan-in or fan-out cone of each other.
- **Global Skew** is the skew between any two non-related flops in the design. By non-related we mean that the two flops do not exist in the fan-out or fan-in cone of each other and hence are in a way mutually exclusive.



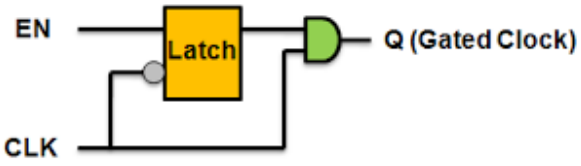
Integrated Clock and Power Gating

Clock Gating and Power Gating are two most commonly used design methods to save dynamic and leakage power respectively. How about integrating the two solutions such that they complement each other? In this post, I will talk about a simple way to do so.

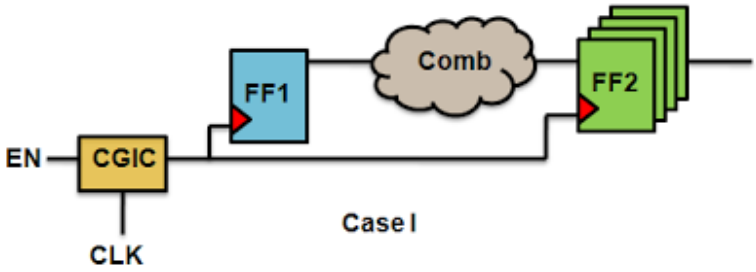
Clock Gating is accomplished by using Clock Gating Integrated Cell *(CGIC)* which gates the clock to the sequential elements present in its fan-out when the enable signal is logic 0. Power Gating structures may be of two types: Simple Power Gating and State Retention Power Gating. Using the former technique, the output of the logic gates slowly leaks the charge at the output and thereby when the *SLEEP* signal is de-asserted, one cannot predict the logic value at the output. The latter technique is able to retain the state at the output which was last present before asserting the *SLEEP* signal.

Let's take up a few plausible scenarios:

- Case I - Normal Case: Which employs only conventional clock gating. It is depicted in the figure.

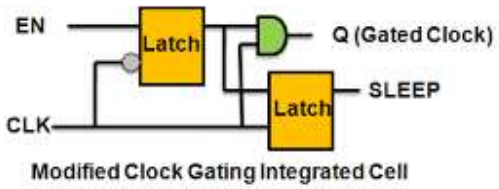


Clock Gating Integrated Cell

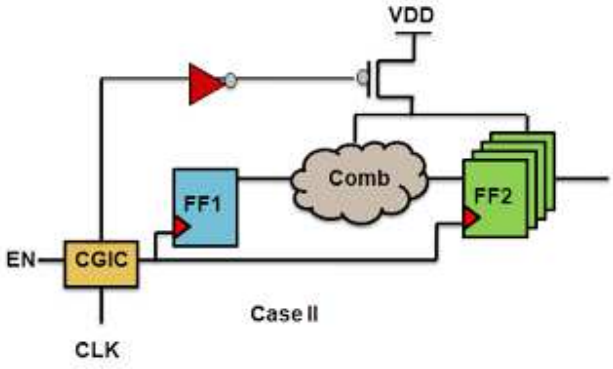


Case I

- Case II - When one does not need to retain the states of the combinatorial cells or the sequential elements. One possible scenario could be in the case of a standalone IP, which is not comunicating with any other IP on the SoC. Here one can use thee simple power gating where the SLEEP signal is derived from the CGIC itself using a latch, as depicted in the figure below. Doing so, we would save both dynamic and leakage powers.

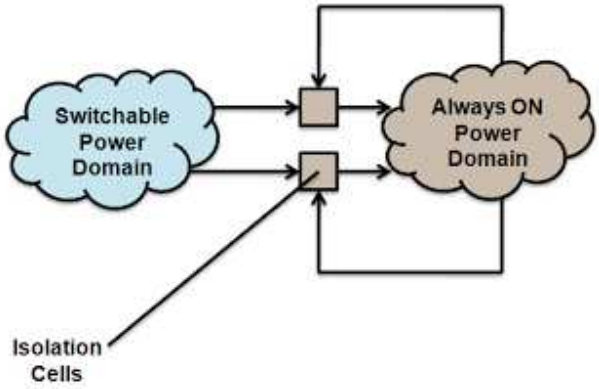


Modified Clock Gating Integrated Cell



Case II

- Case IIII - When one does not need to retain the states of the combinatorial cells, but the sequential outputs need to be safe-stated. Possible use-case could be where only the sequential outputs communicate with other IPs on the SoC. This can be accomplished by using State Retention Flip Flops instead of the conventional flip-flops.
- Case IV - When both the combinatorial cells and the sequential cells interact with other IPs. But the previous value need not be required. Since it is a classic case of interaction between "switchable power domain" with" always ON", it entails the use of isolation cells between such power domain crossings. It must be noted that in such a case, isolation cell would always be present in the always ON power domain, i.e., it would receive it's VDD supply from the always ON power domain supply. This is because, when the switchable power domain in OFF, the isolation cell can function only if receives the power supply!

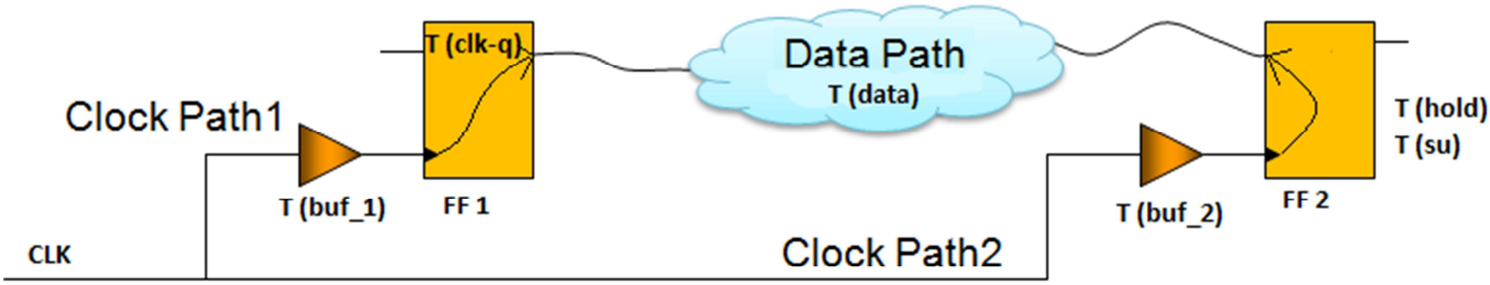


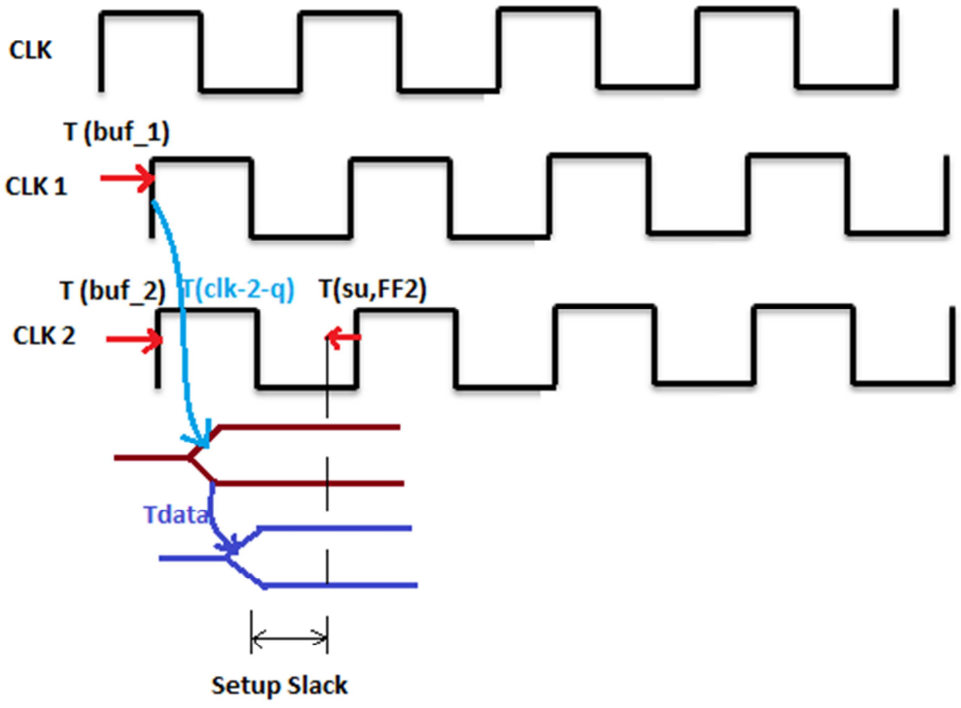
Isolation Cells can be simple cells like AND or an OR gate, which receive one input in a way that, irrespective of the second input coming from the switchable power domain, the value would be controllable. For example, logic 0 for AND gate and logic 1 for an OR gate. I will try to take this up in a separate post.

Sample Problem on Setup and Hold

In the post Timing: Basics, we discussed about the basics of setup and hold times. Why is it necessary to meet the setup and hold timing requirements. And how frequency affects setup but does not affect hold.

Let us understand the concept with an example:



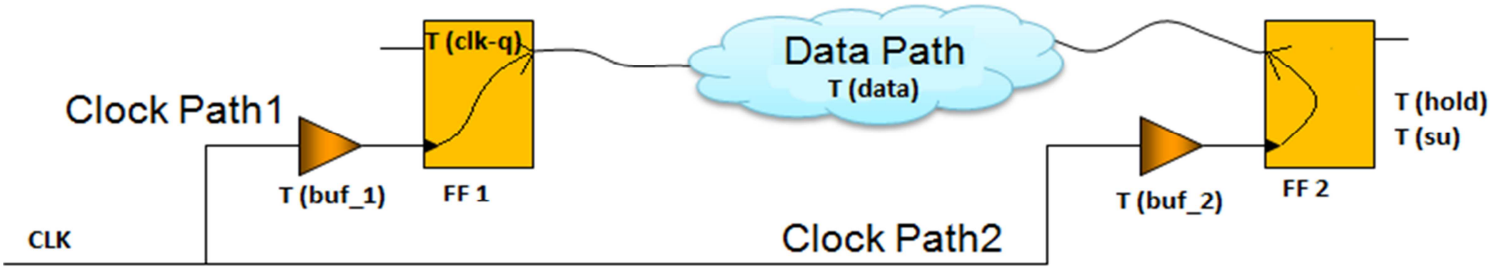


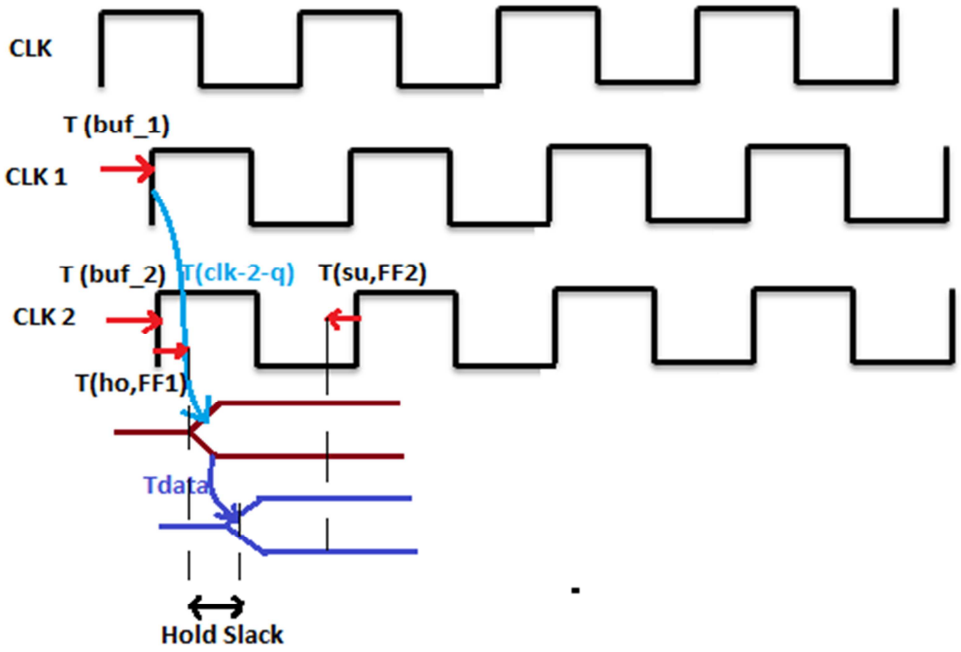
I hope the above waveforms are self explanatory.
Setup Slack in the above case (as inferred from the waveforms as well) is:

Setup Slack = Tclk - T(clk-2-q) - Tdata - T(su,FF2)

If this setup slack is positive, we say that *setup time constraint* is met. Note that setup slack depends upon the clock period and hence in turn frequency at which your design is clocked.

Let us consider hold timing:





Hold Slack = Tdata + T(clk-2-q) - T(ho,FF2)

As evident from the above equation, hold slack is independent of the frequency of the design.

Note:

- Setup is the next cycle check, we would take the setup time T(su,FF2) of FF2 into account while finding setup slack at input pin of FF2.
- Hold time is the same cycle check, we would take the hold time T(ho,FF2) of FF2 into account while computing the hold slack at input pin of FF2.

Try and grasp this example. I shall introduce the concept of clock skew next.

https:// http://vlsi-doubts.blogspot.in/